

LLMWiki

A Semantic Operating Layer for Local Knowledge

Technical Specification and Architecture Reference

Volume I – XI: Complete Technical Documentation

Jeoung Mingyu
jmkjmk1023@gmail.com



July 27, 2026

Version 1.0.0 | 

Mission Statement:

LLMWiki is not a Retrieval-Augmented Generation framework. It is a semantic operating layer that continuously transforms heterogeneous digital artifacts into an evidence-aware knowledge space.

Project Governance

Role	Name	Contact
Chief System Architect	Jeoung Mingyu	jmkjmk1023@gmail.com
Lead Researcher	Jeoung Mingyu	jmkjmk1023@gmail.com
Patent Attorney	TBD	TBD

This document is a living technical specification.
All design decisions are recorded in DESIGN_DECISIONS.md.
Never contradict prior architectural decisions without explicit review.

Contents

I	Foundation and Architecture	1
1	Architecture	3
1.1	Design Philosophy	3
1.2	Three-Layer Architecture	3
1.2.1	Layer 1: Ingestion Pipeline	4
1.2.2	Layer 2: Core Knowledge Space	4
1.2.3	Layer 3: Query and Execution	4
1.3	Filesystem as Write-Ahead Log	5
1.3.1	Directory Layout	5
1.4	Data Flow Architecture	5
1.4.1	Ingestion Pipeline	6
1.4.2	Query Pipeline	7
1.5	Core Invariants	7
1.6	The Filesystem as Lineage Recorder	7
1.7	Comparison with Prior Art	8
1.8	Patent-Relevant Novelty	8
1.9	Module Interface Contracts	9
1.10	Runtime Configuration	10
1.11	Extension Points	11
1.12	Summary	11
2	Semantic Parsing	13
2.1	Design Goals	13
2.2	Parser Architecture	13
2.2.1	Parser Trait	13
2.2.2	Parser Registry	14
2.3	Semantic Unit Model	14
2.3.1	Unit Type Taxonomy	15
2.3.2	Metadata Schema	15
2.3.3	Canonicalization	16
2.4	Language-Specific Parsers	16
2.4.1	Tree-sitter Based Parsers (Code)	16
2.4.1.1	Supported Languages	17
2.4.2	Markdown Parser	17
2.4.3	PDF Parser	17
2.4.4	OpenAPI / AsyncAPI Parser	18

2.4.5	SQL DDL Parser	18
2.4.6	Structured Data Parser (YAML/JSON/TOML)	18
2.5	Structure-Aware Chunking	18
2.5.1	Hierarchical Chunking Algorithm	19
2.5.2	Sliding Window (Fallback)	19
2.5.3	Semantic Chunking (Experimental)	19
2.5.4	Scale-Free Chunking by Graph Renormalization	20
2.6	Entity and Relation Extraction	21
2.6.1	Rule-Based Extraction	21
2.6.2	LLM-Assisted Extraction (Optional)	21
2.6.3	Entity Resolution	22
2.7	Incremental Parsing	22
2.8	Embedding Generation	23
2.8.1	Supported Models	23
2.8.2	Batch Inference	23
2.9	Parser Extensibility Guide	23
2.10	Summary	24
3	Knowledge Graph & Semantic Index	25
3.1	Knowledge Graph	25
3.1.1	Data Model	25
3.1.2	Graph Store Backend	26
3.1.2.1	Storage Layout (RocksDB)	26
3.1.3	Entity Resolution	27
3.1.4	Incremental Graph Updates	27
3.1.5	Graph Traversal Primitives	28
3.2	Semantic Index	28
3.2.1	HNSW Vector Index	28
3.2.2	BM25 Keyword Index	29
3.2.3	Graph Adjacency Index	29
3.2.4	Hybrid Search	29
3.2.5	Index Update Strategy	30
3.3	Query-Time Graph Expansion	30
3.4	Performance Characteristics	30
3.5	Patent-Relevant Novelty	30
3.6	Summary	31
4	Retrieval and Evidence Gate	33
4.1	Evidence Gate: Core Principle	33
4.2	Why Not Reranking?	33
4.3	Retrieval Pipeline	33
4.3.1	Stage 1: Query Embedding	34
4.3.2	Stage 2: Hybrid Search	34
4.3.3	Stage 3: Evidence Gate	34
4.3.4	Multi-Stage Proofreading Cascade	35
4.3.5	Stage 4: Synthesis with Forced Citations	36

4.4	Threshold Calibration	37
4.4.1	Calibration Methods	37
4.4.2	Label-Free Calibration by Discordant Pairing	37
4.4.3	Confidence Intervals	39
4.5	Global Consistency of Evidence	39
4.6	Query Planner Integration	40
4.7	Evidence Gate Config	41
4.8	Evaluation Metrics	41
4.9	Comparison with Prior Art	41
4.10	Patent-Relevant Novelty	42
4.11	Summary	43
5	Agent Runtime	45
5.1	Design Goals	45
5.2	Runtime Architecture	45
5.2.1	Core Components	46
5.3	Plan Execution Model	46
5.3.1	Query Plan Structure	46
5.3.2	Execution Algorithm	47
5.3.3	Step Kinds	47
5.4	Tool System	47
5.4.1	Tool Trait	47
5.4.2	Built-in Tools	47
5.4.3	Custom Tools	48
5.5	LLM Client Abstraction	48
5.6	State Management	48
5.7	Budget as Allocation, Not Truncation	49
5.8	Streaming Protocol	50
5.9	Error Handling and Retries	51
5.10	Conversation Management	51
5.11	Patent-Relevant Novelty	51
5.12	Summary	51
6	Evaluation Framework	53
6.1	Evaluation Philosophy	53
6.2	Benchmark Suite	53
6.2.1	Internal Benchmarks	53
6.2.2	External Benchmarks (Adapted)	54
6.3	Metric Definitions	54
6.3.1	Retrieval Metrics	54
6.3.2	Answer Quality Metrics	55
6.3.3	System Property Metrics	55
6.4	Evaluation Pipeline	55
6.4.1	Configuration Matrix	56
6.4.2	Statistical Rigor	56

6.5	Regression Detection	56
6.6	Qualitative Evaluation	57
6.7	Patent-Relevant Novelty	57
6.8	Summary	57
7	Deployment and Operations	59
7.1	Deployment Modes	59
7.2	Container Images	59
7.2.1	Image Variants	60
7.3	Kubernetes Deployment	60
7.3.1	Helm Chart Structure	60
7.3.2	Key Configuration	60
7.3.3	Stateful Considerations	61
7.4	Local-First Deployment	61
7.4.1	Single Binary	61
7.4.2	Docker Compose (Local Stack)	62
7.4.3	Air-Gapped Installation	62
7.5	Configuration Management	63
7.6	Monitoring and Observability	63
7.6.1	Metrics (Prometheus)	63
7.6.2	Distributed Tracing (OpenTelemetry)	64
7.6.3	Structured Logging	64
7.6.4	Health Checks	64
7.7	Backup and Recovery	64
7.7.1	Backup Strategy	65
7.7.2	Automated Backup (CronJob)	65
7.7.3	Disaster Recovery	65
7.8	Upgrades and Migrations	66
7.8.1	Version Compatibility	66
7.8.2	Index Migration	66
7.8.3	Graph Migration	66
7.9	Security Hardening	66
7.10	Performance Tuning	67
7.11	Patent-Relevant Novelty	67
7.12	Summary	67
II	Reference and Extension	69
8	API Reference	71
8.1	API Overview	71
8.2	Authentication	71
8.2.1	Bearer Token (OIDC)	71
8.2.2	API Key	71
8.2.3	Anonymous (Development)	72
8.3	Core Endpoints	72

8.3.1	Health	72
8.3.2	Query	72
8.3.3	Ingestion	73
8.3.4	Repository Management	74
8.3.5	Graph Operations	75
8.3.6	Index Operations	75
8.3.7	Unit Access	75
8.3.8	Evidence Gate	75
8.3.9	Administration	76
8.4	WebSocket API	76
8.5	gRPC API	77
8.6	Error Handling	77
8.7	Rate Limiting	77
8.8	SDKs	78
8.9	OpenAPI Specification	79
8.10	API Versioning Policy	79
8.11	Patent-Relevant Novelty	79
8.12	Summary	79
9	Developer Guide	81
9.1	Development Environment	81
9.1.1	Prerequisites	81
9.1.2	Repository Structure	81
9.1.3	Build and Test	82
9.2	Extension Points Deep Dive	82
9.2.1	Custom Parser	82
9.2.2	Custom Chunker	83
9.2.3	Custom Embedder	84
9.2.4	Custom Graph Store	85
9.2.5	Custom Index Backend	85
9.2.6	Custom Evidence Verifier	85
9.2.7	Custom Query Planner Strategy	86
9.2.8	Custom Agent Tool	86
9.2.9	Custom Auth Provider	87
9.3	Testing Extensions	87
9.3.1	Parser Tests	87
9.3.2	Integration Tests	88
9.3.3	Property-Based Testing	88
9.4	Debugging and Profiling	89
9.4.1	Logging	89
9.4.2	Profiling	89
9.5	Contributing	89
9.5.1	Code Style	89
9.5.2	PR Process	89
9.5.3	Adding a New Volume	90

9.6	Release Process	90
9.7	Summary	90
III Intellectual Property and Research		91
10 Patent Draft		93
10.1	Field of the Invention	93
10.2	Background	93
10.3	Summary of the Invention	93
10.4	Independent Claims	93
10.5	Dependent Claims	96
10.6	Design Decisions Supporting Novelty	97
10.7	Specification Support	97
10.8	Potential Design-Arounds and Defense	98
11 Research Paper: LLMWiki - A Semantic Operating Layer for Evidence-Aware Knowledge Management		101
11.1	Introduction	101
11.2	Related Work	102
11.2.1	Retrieval-Augmented Generation	102
11.2.2	Graph-Enhanced RAG	102
11.2.3	Evidence and Attribution	103
11.2.4	Local and Deterministic Systems	103
11.2.5	Methods Imported from Adjacent Sciences	103
11.3	System Architecture	104
11.3.1	Three-Layer Decomposition	104
11.3.2	Filesystem as Write-Ahead Log	104
11.3.3	Content Addressing	104
11.3.4	Evidence Gate	104
11.3.5	DAG Query Planner	104
11.3.6	Label-Free Significance via Discordant Pairing	104
11.3.7	Proofreading Cascade	105
11.3.8	Cohomological Evidence Closure	105
11.4	Evaluation	105
11.4.1	Benchmarks	105
11.4.2	Baselines	106
11.4.3	Metrics	106
11.4.4	Results	107
11.4.5	Ablation Studies	107
11.4.6	Calibration Analysis	107
11.5	Discussion	108
11.5.1	Why Evidence Gate Works	108
11.5.2	Local-First Implications	108
11.5.3	Limitations	108
11.6	Conclusion	108

IV	Appendices	111
.1	Mathematical Notation	113
.2	Complete Configuration Schema	113
.3	API Schemas	115
.3.1	Query Request	115
.3.2	Query Response	116
.4	Evaluation Protocol	117
.4.1	Dataset Construction	117
.4.2	Metrics Computation	117
.4.3	Statistical Rigor	118
.5	Glossary of Key Terms	118
A	Data Model Definitions	125
A.1	SemanticUnit	125
A.2	Chunk	125
A.3	Entity	126
A.4	Relation	126
A.5	QueryPlan	126
A.6	ToolSchema	127
B	Configuration Reference	129
B.1	Full config.toml	129
B.2	Environment Variable Overrides	130
C	Parser Grammar Specifications	131
C.1	Python (tree-sitter-python)	131
C.2	JavaScript/TypeScript	131
C.3	Rust	132
C.4	Go	132
C.5	SQL	132
C.6	Design Decision Log	134
D	Extended Related Work	137
D.1	Dense Retrieval	137
D.2	Sparse Retrieval	137
D.3	Hybrid Retrieval	137
D.4	Graph-Enhanced Retrieval	137
D.5	Recursive/Tree-Based Retrieval	137
D.6	Evidence and Attribution	137
D.7	Local/Private LLMs	137
D.8	Vector Databases	138
D.9	Graph Databases	138
D.10	Parsing and Embedding	138

List of Figures

1.1	Three-Layer Architecture of LLMWiki. Filesystem acts as write-ahead log (WAL) for all layers.	4
1.2	End-to-end data flow: ingestion pipeline (top) and query pipeline (bottom).	6
2.1	Parser pipeline: detection, routing, parsing, and unit emission.	14
4.1	Retrieval + Evidence Gate pipeline.	34
5.1	Agent Runtime components.	45
6.1	Automated evaluation pipeline.	55
11.1	Reliability diagrams for cross-encoder scores before and after isotonic calibration. Uncalibrated scores are systematically overconfident; isotonic regression brings empirical accuracy close to the diagonal.	108

List of Tables

1.1	LLMWiki vs. Prior Art: Architectural Positioning	8
1.2	LLMWiki Extension Points	11
2.1	Core Semantic Unit Types by Parser Domain	15
2.2	Tree-sitter Languages Supported (v0.1)	17
2.3	Local Embedding Models (Recommended)	23
3.1	Graph Store Backend Comparison	26
3.2	Expected Performance (Single Node, SSD)	31
4.1	Reranker vs. Evidence Gate	33
4.2	Evidence Gate Evaluation Metrics	41
4.3	Retrieval + Verification in Prior Art	41
5.1	Built-in Tools	48
6.1	LLMWiki Internal Benchmarks	53
7.1	LLMWiki Deployment Modes	59
7.2	Tuning Knobs by Workload	67
8.1	Error Codes	78
8.2	Official SDKs	78
10.1	Design Decisions Supporting Novelty	98
11.1	Main Results on LLMWikiBench (mean $\pm$ 95% CI, 1000 queries)	107
11.2	Ablation: Component Contribution (Evidence F1@10)	107
3	Notation Summary	113
C.1	Key Design Decisions	135

List of Algorithms

1	Ingestion Pipeline	6
2	Query Execution Pipeline	7
3	Unit Canonicalization	16
4	Hierarchical Chunking	19
5	Semantic Chunking via Embedding Similarity	20
6	Entity Resolution	22
7	Incremental Unit Diff	23
8	Entity Resolution	27
9	Incremental Graph Update	27
10	Hybrid Search	30
11	Evidence Gate	34
12	Calibration Procedure	37
13	False Alarm Rate Estimation without Relevance Labels	38
14	Consistency Gate	40
15	Plan Execution	47
16	Budget-Aware Step Scheduling	50
17	Evidence Recall@k	117

Acronyms

Part I

Foundation and Architecture

Architecture

LLMWiki is not a Retrieval-Augmented Generation (RAG) framework. It is a **semantic operating layer** that continuously transforms heterogeneous digital artifacts into an evidence-aware knowledge space. This volume establishes the architectural foundations: the three-layer decomposition, the filesystem-as-WAL principle, the data flow invariants, and the module interface contracts that all subsequent volumes build upon.

1.1 Design Philosophy

Design Principle 1.1 (Graph as Source of Truth). The property knowledge graph is the canonical knowledge representation. Vector indices are derived projections optimized for similarity search—not the primary store.

Design Principle 1.2 (Filesystem as Write-Ahead Log). The local filesystem (directory structure `.llmwiki/`) serves as the sole durability mechanism. No auxiliary transaction log service (Kafka, Redis Streams, etc.) is required.

Design Principle 1.3 (Evidence Gate as First-Class Module). Retrieval results must pass a mandatory cross-encoder verification stage before synthesis. This is not a reranker; it is an evidence filter with calibrated confidence thresholds.

Design Principle 1.4 (Local-First, LLM-Optional). All critical-path operations (parsing, chunking, embedding, graph update, indexing, evidence verification) run locally with zero external API dependencies. LLMs are used only for planning, entity extraction (optional), and answer synthesis.

Design Principle 1.5 (Content Addressability). Every semantic unit is identified by SHA-256 of its canonical form. Identical content implies identical identifier across all indices, enabling deduplication, caching, and verifiable lineage.

Design Principle 1.6 (Incremental Consistency). After any ingestion, the system state (G, I, U) - graph, index, units - satisfies: a query executed against the live state produces the same result as replaying the ingestion log.

Design Principle 1.7 (Deterministic Replay). Given identical `.llmwiki/` directory and identical query `q`, two executions produce bitwise-identical results (modulo non-deterministic LLM sampling, which is confined to planning/synthesis).

1.2 Three-Layer Architecture

The system decomposes into three layers with strictly unidirectional data flow: Ingestion to Core to Query.

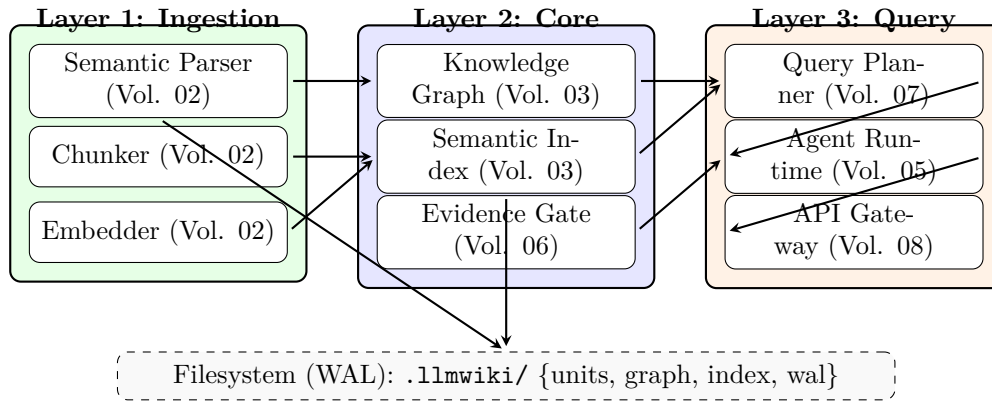


Figure 1.1: Three-Layer Architecture of LLMWiki. Filesystem acts as write-ahead log (WAL) for all layers.

1.2.1 Layer 1: Ingestion Pipeline

Responsible for parsing heterogeneous artifacts into semantic units - content-addressed, typed, and annotated fragments. Modules:

- Semantic Parser (Vol. 02): Language-aware parsing (code, markdown, PDF, OpenAPI, SQL DDL, etc.)
- Chunker: Hierarchical, structure-aware segmentation with overlap control
- Embedder: Local embedding models (BGE, E5, nomic-embed) with batch inference

1.2.2 Layer 2: Core Knowledge Space

The persistent, queryable knowledge representation. Three tightly coupled modules:

- Knowledge Graph (Vol. 03): Property graph with typed nodes/edges, entity resolution, and incremental updates
- Semantic Index (Vol. 03): Hybrid index (HNSW vector + BM25 keyword + graph adjacency)
- Evidence Gate (Vol. 06): Cross-encoder verification + citation extraction + confidence calibration

1.2.3 Layer 3: Query and Execution

Composable query execution with planning and observability:

- Query Planner (Vol. 07): DAG-based plan compilation, cost-based optimization, subquery decomposition
- Agent Runtime (Vol. 05): Tool execution, state management, streaming responses
- API Gateway (Vol. 08): REST/gRPC/WebSocket contracts, authentication, rate limiting

1.3 Filesystem as Write-Ahead Log

Design Decision 1.8. Decision: Use the local filesystem (directory structure `.llmwiki/`) as the write-ahead log instead of a dedicated WAL service (e.g., Kafka, Redis Streams, WAL-G).

- Design Rationale 1.9.**
1. Zero operational dependency: No separate service to deploy, monitor, or backup.
 2. Deterministic replay: `rsync` or `git` of `.llmwiki/` fully reproduces state.
 3. Content-addressed deduplication: Units stored as `units/<hash>.json` are naturally deduplicated.
 4. Atomicity via rename: Write to temp file, `rename(2)` for atomic commit - POSIX guarantee.
 5. Backup equals copy: `tar`, `rsync`, `borg`, `restic` work natively.

- Design Trade-off 1.10.**
- Concurrency: Single-writer per repository; multi-writer requires external locking (file lock or SQLite WAL mode).
 - Scale: Directory entry count grows with unit count. Mitigation: sharded subdirectories (`units/ab/cd/ef...`).
 - Durability: `fsync` on commit; configurable `sync_on_write` flag.

1.3.1 Directory Layout

```
.llmwiki/
|-- units/                # Content-addressed semantic units
|   |-- ab/cd/ef1234.json  # SHA-256 prefix sharding
|   |-- ...
|-- graph/                # Knowledge graph persistence
|   |-- nodes/            # Node partitions (by type)
|   |-- edges/            # Edge partitions (by relation type)
|   |-- indexes/          # Graph-local indexes (adjacency, labels)
|-- index/                # Semantic search indexes
|   |-- hnsw/              # HNSW vector index (disk-backed)
|   |-- bm25/              # BM25 inverted index (tantivy)
|   |-- adjacency/        # Graph adjacency for traversal
|-- wal/                  # Write-ahead log (append-only JSONL)
|   |-- 2024-01-15-001.jsonl # Daily rotation
|   |-- ...
|-- planner/              # Query plan cache
|   |-- plans/
|-- evidence/              # Evidence gate caches
|   |-- cross-encoder/
|-- config.toml            # Repository configuration
```

1.4 Data Flow Architecture

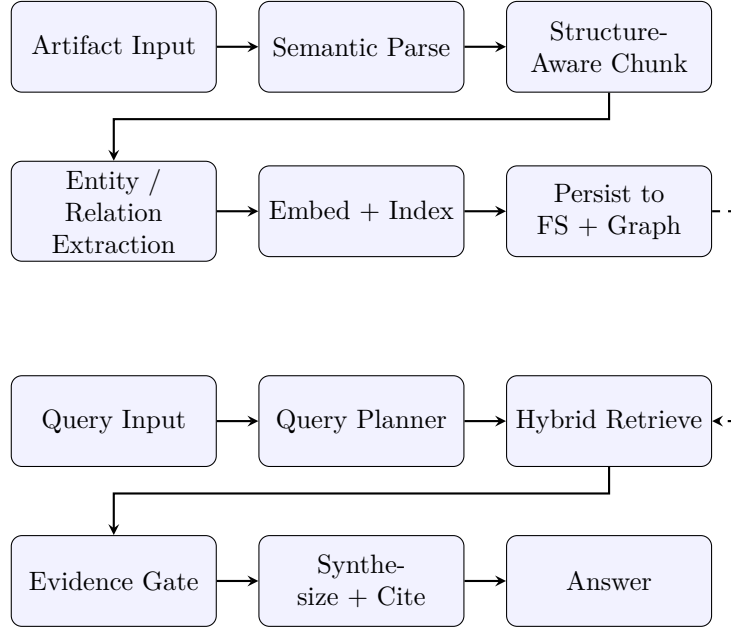


Figure 1.2: End-to-end data flow: ingestion pipeline (top) and query pipeline (bottom).

1.4.1 Ingestion Pipeline

Algorithm 1 Ingestion Pipeline

Require: Artifact A (file, API response, database row)

Ensure: Persisted semantic units U, graph updates Delta G, index updates Delta I

- | | |
|--|------------------------------------|
| 1: $U = \text{Parse}(A)$ | ▷ Language-aware parser |
| 2: $U = \text{Chunk}(U)$ | ▷ Hierarchical, overlap-controlled |
| 3: $\Delta G = \text{ExtractEntitiesRelations}(U)$ | ▷ Local LLM or rule-based |
| 4: $\Delta I = \text{Embed}(U)$ | ▷ Batch local inference |
| 5: $\text{WriteWAL}(\Delta G, \Delta I, U)$ | ▷ Atomic append to WAL |
| 6: $\text{ApplyGraph}(\Delta G)$ | ▷ Incremental graph update |
| 7: $\text{ApplyIndex}(\Delta I)$ | ▷ Incremental index update |
| 8: return U, Delta G, Delta I | |
-

1.4.2 Query Pipeline

Algorithm 2 Query Execution Pipeline

Require: Query q , context budget B , evidence threshold τ

Ensure: Answer a with citations C

```

1:  $P = \text{Plan}(q)$  ▷ Planner outputs DAG of subqueries
2:  $R = \emptyset$ 
3: for all subquery  $s$  in  $P$  do
4:    $R_s = \text{HybridRetrieve}(s)$  ▷ Vector + BM25 + Graph
5:    $R = R \cup R_s$ 
6:  $E = \text{EvidenceGate}(R, \tau)$  ▷ Cross-encoder verify
7:  $a = \text{Synthesize}(E, B)$  ▷ LLM with citations forced
8: return  $a, C(E)$  ▷ Answer + evidence citations

```

1.5 Core Invariants

Invariant 1.11 (Content Addressability). For all u in U : $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Identical content implies identical identifier across all indices.

Invariant 1.12 (Evidence Closure). For all a in Answers: $\text{citations}(a)$ subset of U and for all c in $\text{citations}(a)$: $\text{verify}(c) \geq \tau$.

Invariant 1.13 (Incremental Consistency). After any ingestion I , the triple (G, I, U) satisfies: $\text{query}(q, G, I, U) = \text{replay}(\text{queries}(q))$.

Invariant 1.14 (Local-First Determinism). Given identical `.llmwiki/` directory and identical query q , two executions produce bitwise-identical results (modulo non-deterministic LLM sampling, which is confined to planning/synthesis).

1.6 The Filesystem as Lineage Recorder

Invariants 1.13 and 1.14 assert that history can be replayed. They do not say how history is *reconstructed* when the record is incomplete, and every long-lived deployment produces an incomplete record: WAL segments are compacted, old snapshots are pruned, repositories are cloned from a mid-life state. A replay guarantee that holds only over an unbroken log is a guarantee that expires.

Developmental biology has spent a decade on exactly this reconstruction problem. CRISPR lineage recorders accumulate irreversible edits at engineered target sites across cell divisions, so that the division history of an organism can be inferred from the barcodes of its terminal cells alone — the intermediate states are gone, and the tree is recovered from the endpoint. Recent surveys note that recorders which log a *deterministic temporal ordering* of mutations enable precise reconstruction of deep lineages [Lineage tracing review authors, 2026], which is precisely the argument for an append-only WAL. More usefully, the field has formalised inference under a degraded record: Bayesian phylodynamic frameworks reconstruct lineage trees with calibrated uncertainty from barcode data with dependent target sites and missing observations [Phylodynamic inference authors, 2025].

Two failure modes from that literature transfer to LLMWiki without modification.

Homoplasy. An identical edit arising independently in two lineages falsely implies shared ancestry. Content addressability (Invariant 1.11) makes this endemic rather than occasional: two units produced by unrelated derivation paths that happen to canonicalise identically receive the same identifier *by design*. This is the deduplication property, and it is desirable for storage and indexing. It is also destructive of provenance, because it collapses distinct derivations into one node. Identity and ancestry are different relations and must not share a key.

Invariant 1.15 (Lineage Distinctness). Content identity does not imply derivation identity. For a unit u , $\text{id}(u)$ addresses content; the derivation of u is recorded separately as the set of WAL edges producing it. Provenance is therefore a directed acyclic graph over content identifiers, never a tree, and a query for the origin of u returns a set of derivations rather than a unique parent.

Dropout. A target site that fails to be read yields a missing character, not a wrong one, and treating the two alike corrupts the tree. The analogue is a compacted WAL segment: the derivation edge is unobserved, not absent. Recording compaction boundaries explicitly lets reconstruction distinguish *no derivation occurred* from *the derivation is unobservable*, which is the difference between a false claim and an honest one.

Invariant 1.16 (Replay Degradation is Explicit). Where the WAL has been compacted, replay returns a result annotated with the reconstruction’s uncertainty rather than an unqualified answer. A deployment that cannot verify a replay reports that it cannot, and never silently substitutes a plausible history.

This is the mechanism that makes the auditability claim survive contact with operations. A system that guarantees reproducibility only while nothing has been pruned has guaranteed reproducibility only in the laboratory; the useful statement is that reproducibility degrades in a declared, measurable way.

1.7 Comparison with Prior Art

Table 1.1: LLMWiki vs. Prior Art: Architectural Positioning

Property	RAG	GraphRAG	HippoRAG	LLMWiki
Graph as source of truth	No	Yes	Yes	Yes
Vectors as projection	No	No	No	Yes
Filesystem as WAL	No	No	No	Yes
Evidence gate (first-class)	No	No	No	Yes
Local-first (no API required)	No	No	No	Yes
Deterministic replay	No	No	No	Yes
Content-addressed units	No	No	No	Yes
Query planner (DAG)	No	No	No	Yes

1.8 Patent-Relevant Novelty

Patent Claim 1.17. Claim 1: A semantic operating layer characterized by: (a) content-addressed semantic units stored in a filesystem-backed write-ahead log, (b) a property

knowledge graph as the canonical knowledge representation, (c) a hybrid semantic index combining vector, keyword, and graph adjacency, (d) an evidence gate that verifies retrieval results via cross-encoder scoring before synthesis, and (e) a DAG-based query planner that decomposes queries into verifiable subqueries.

Patent Claim 1.18. Claim 2: The system of Claim 1 wherein the filesystem serves as the sole durability mechanism, enabling deterministic replay via directory synchronization without auxiliary transaction logs.

Patent Claim 1.19. Claim 3: The system of Claim 1 wherein the evidence gate operates as a mandatory pipeline stage, rejecting any retrieval result that fails cross-encoder verification against a calibrated threshold, thereby enforcing evidence closure.

Patent Discussion 1.20. Prior art distinctions:

- RAG (Lewis et al., 2020): Vector index is primary; no graph; no evidence gate; API-dependent.
- GraphRAG (Edge et al., 2024): Graph constructed via LLM; no content-addressed units; no filesystem WAL; no mandatory evidence gate.
- HippoRAG (Zhang et al., 2024): Personalized PageRank over graph; no hybrid index; no deterministic replay.
- LightRAG (Zeng et al., 2024): Dual-level retrieval; no evidence gate; no DAG planner.
- RAPTOR (Sarathi et al., 2024): Recursive summarization tree; no graph; no filesystem WAL.

1.9 Module Interface Contracts

```
Interface 1.21 {Parser}.
  fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>;
  fn supported_mimes(&self) -> Vec<MimeType>;
  fn schema(&self) -> ParserSchema;
}
```

```
Interface 1.22 {Chunker}.
  fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) ->
    ↪ Vec<Chunk>;
  fn config_schema(&self) -> ChunkConfigSchema;
}
```

```
Interface 1.23 {Embedder}.
  fn embed_batch(&self, texts: &[String]) -> Result<Vec<Vec<f32>>>;
  fn dimension(&self) -> usize;
  fn model_id(&self) -> ModelId;
}
```

```
Interface 1.24 {GraphStore}.
  fn upsert_nodes(&self, nodes: Vec<Node>) -> Result<()>;
  fn upsert_edges(&self, edges: Vec<Edge>) -> Result<()>;
}
```

```
fn traverse(&self, start: NodeId, pattern: TraversalPattern) ->
    ↪ Result<Vec<Path>>;
fn neighborhoods(&self, ids: &[NodeId], hops: u32) ->
    ↪ Result<Subgraph>;
}

Interface 25 (SemanticIndex).
fn upsert(&self, units: Vec<IndexedUnit>) -> Result<()>;
fn search_vector(&self, query: Vec<f32>, k: usize) ->
    ↪ Result<Vec<ScoredUnit>>;
fn search_bm25(&self, query: &str, k: usize) ->
    ↪ Result<Vec<ScoredUnit>>;
fn search_hybrid(&self, query: HybridQuery, k: usize) ->
    ↪ Result<Vec<ScoredUnit>>;
}

Interface 26 (VerificationGate).
fn verify(&self, query: &str, candidates: Vec<ScoredUnit>, threshold:
    ↪ f32)
    -> Result<Vec<VerifiedUnit>>;
fn calibrate(&self, dataset: CalibrationSet) ->
    ↪ Result<CalibrationCurve>;
}

Interface 27 (QueryPlanner).
fn plan(&self, query: Query, context: PlanContext) ->
    ↪ Result<QueryPlan>;
fn optimize(&self, plan: QueryPlan, stats: IndexStats) ->
    ↪ Result<QueryPlan>;
fn explain(&self, plan: QueryPlan) -> PlanExplanation;
}
```

1.10 Runtime Configuration

```
# .llmwiki/config.toml
[repository]
id = "llmwiki-main"
created_at = "2024-01-15T10:30:00Z"
version = "0.1.0"

[ingestion]
parsers = ["python", "markdown", "pdf", "openapi", "sql", "yaml", "json"]
chunker = { type = "ast-aware", max_tokens = 512, overlap = 50 }
embedder = { model = "bge-m3", batch_size = 32, device = "cpu" }

[graph]
store = "rocksdb"
entity_resolution = { enabled = true, threshold = 0.85 }
index_types = ["btree", "adjacency", "label"]
```

```

[index]
hns = { m = 16, ef_construction = 200, ef_search = 64 }
bm25 = { k1 = 1.2, b = 0.75 }
hybrid = { vector_weight = 0.6, keyword_weight = 0.3, graph_weight = 0.1 }

[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72
calibration = "isotonic"
batch_size = 16

[planner]
max_depth = 5
max_branching = 4
cost_model = "learned"
cache_ttl = "1h"

[api]
bind = "127.0.0.1:8080"
auth = "none"
rate_limit = 100

```

1.11 Extension Points

Table 1.2: LLMWiki Extension Points

Extension Point	Interface	Volume
Custom Parser	Parser	Vol. 02
Custom Chunker	Chunker	Vol. 02
Custom Embedder	Embedder	Vol. 02
Graph Store Backend	GraphStore	Vol. 03
Index Backend	SemanticIndex	Vol. 03
Evidence Verifier	EvidenceGate	Vol. 06
Query Planner Strategy	QueryPlanner	Vol. 07
Agent Tool	Tool	Vol. 05
Auth Provider	AuthProvider	Vol. 08

1.12 Summary

LLMWiki's architecture is defined by five differentiating choices:

1. **Graph-first, vectors-second:** The property graph is the canonical knowledge representation; vectors are derived projections for similarity search.
2. **Filesystem as WAL:** Zero operational dependencies; deterministic replay via directory sync.

3. **Content-addressed units:** SHA-256 identities enable deduplication, caching, and verifiable lineage.
4. **Evidence gate as mandatory stage:** No retrieval result reaches synthesis without cross-encoder verification.
5. **DAG query planner:** Queries decompose into verifiable subqueries with cost-based optimization.

These choices are not incremental improvements over RAG - they represent a different architectural category: a semantic operating layer.

The subsequent volumes detail each module's algorithms, data structures, APIs, and implementation considerations.

Semantic Parsing

This volume specifies the *semantic parsing* layer—the bridge between raw artifacts (source code, documents, schemas, APIs) and the content-addressed semantic units that populate the knowledge graph and search indices. Unlike traditional document chunking, semantic parsing extracts *typed, structured, hierarchically organized* units with explicit semantic annotations.

2.1 Design Goals

Design Principle 2.1 (Language Awareness). Parsers understand the syntax and semantics of their target language (Python, SQL, OpenAPI, Markdown, etc.), producing units that reflect logical constructs (functions, classes, tables, sections) rather than arbitrary text spans.

Design Principle 2.2 (Structure Preservation). The hierarchical structure of the source artifact is preserved in the unit tree. A Python class contains method units; a Markdown document contains section units; an OpenAPI spec contains operation units.

Design Principle 2.3 (Content Addressability). Each semantic unit is identified by SHA-256 of its canonical representation. Identical logical constructs yield identical identifiers regardless of source file location.

Design Principle 2.4 (Incremental Re-parse). When an artifact changes, only affected units are re-parsed. The parser emits a diff of (added, modified, removed) unit IDs.

Design Principle 2.5 (Extensibility). New parsers are added via the `Parser` trait (Vol. 01, §1.21) without modifying core ingestion logic.

2.2 Parser Architecture

2.2.1 Parser Trait

Every parser implements the `Parser` interface (Vol. 01, §1.21):

```
trait Parser {  
  fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>;  
  fn supported_mimes(&self) -> Vec<MimeType>;  
  fn schema(&self) -> ParserSchema;  
}
```

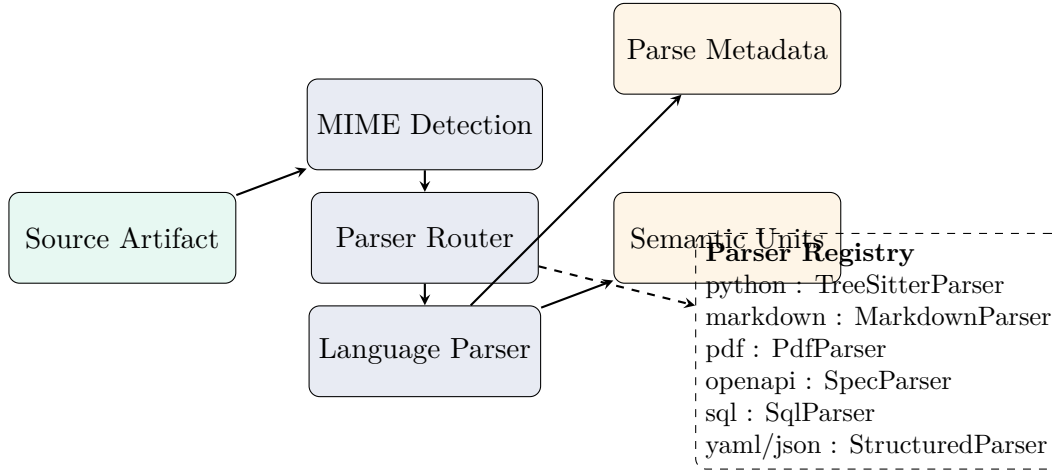


Figure 2.1: Parser pipeline: detection, routing, parsing, and unit emission.

Artifact encapsulates the raw bytes, path, MIME type, and optional metadata (encoding, language hints). **SemanticUnit** is the canonical output (defined below). **ParserSchema** describes the unit types this parser produces (e.g., `PythonFunction`, `SQLTable`, `MarkdownSection`).

2.2.2 Parser Registry

The registry maps MIME types (and file extensions as fallback) to parser instances. Parsers are loaded dynamically at startup from `config.toml [ingestion.parsers]`.

```
# Config example
[ingestion.parsers]
python = { type = "tree-sitter", grammars = ["python"] }
markdown = { type = "markdown", extract_frontmatter = true }
pdf = { type = "pdf", ocr = false }
openapi = { type = "openapi", version = "3.1" }
sql = { type = "sql", dialect = "postgres" }
```

2.3 Semantic Unit Model

Definition 2.6 (Semantic Unit). A *semantic unit* u is a tuple:

$$u = (\text{id}, \text{type}, \text{content}, \text{span}, \text{metadata}, \text{children}) \quad (2.1)$$

where:

- `id` = `SHA-256(canonical(content, type, span))`
- `type` $\in$ `{Function, Class, Section, Table, Operation, ...}`
- `content`: canonical text representation (whitespace-normalized)
- `span`: `(start_line, end_line, start_col, end_col)` in source
- `metadata`: typed key-value annotations (see §2.3.2)
- `children`: ordered list of child unit IDs (hierarchy)

2.3.1 Unit Type Taxonomy

Table 2.1: Core Semantic Unit Types by Parser Domain

Domain	Unit Types	Key Metadata
Python	Module, Class, Function, Method, Decorator, Import	signature, decorators, type_hints, docstring
JavaScript/TS	Module, Class, Function, ArrowFunction, Interface, Type	signature, jsdoc, exports
Markdown	Document, Section, CodeBlock, List, Table, Link	heading_level, language, alt_text
PDF	Page, Paragraph, Table, Figure, Header, Footer	bbox, font_info, page_num
OpenAPI	Info, Server, Path, Operation, Schema, Parameter	method, path, status_codes, auth
SQL DDL	Database, Schema, Table, Column, Index, Constraint	data_type, nullable, default, references
YAML/JSON	Document, Mapping, Sequence, Scalar	json_path, schema_ref
Dockerfile	Instruction, Stage, Arg, Env, Label	instruction, value, stage
Protobuf	Message, Enum, Service, Field, RPC	field_number, type, label

2.3.2 Metadata Schema

Each unit type defines a typed metadata schema. Examples:

```
# Python Function metadata
{
  "signature": "def foo(x: int, y: str = 'a') -> bool",
  "decorators": ["@staticmethod", "@cache"],
  "type_hints": {"x": "int", "y": "str", "return": "bool"},
  "docstring": "Returns True if...",
  "is_async": false,
  "is_generator": false,
  "complexity": 3
}

# SQL Table metadata
{
  "columns": [
    {"name": "id", "type": "BIGINT", "nullable": false, "primary_key":
      ↪ true},
    {"name": "email", "type": "VARCHAR(255)", "nullable": false, "unique":
      ↪ true}
  ],
  "indexes": [{"name": "idx_email", "columns": ["email"], "unique":
    ↪ true}],
  "foreign_keys": [{"column": "user_id", "references": "users.id"}],
  "engine": "InnoDB",
  "charset": "utf8mb4"
}

# OpenAPI Operation metadata
{
  "method": "POST",
  "path": "/api/v1/users",
  "operation_id": "createUser",
  "parameters": [{"name": "body", "in": "body", "schema": "UserInput"}],
  "responses": {"201": "User", "400": "Error"},
}
```

```
"security": [{"BearerAuth": []}],
"deprecated": false
}
```

2.3.3 Canonicalization

Content-addressability requires deterministic canonicalization:

Algorithm 3 Unit Canonicalization

Require: Unit u with fields ($type, content, span, metadata, children$)

Ensure: Canonical byte string C for hashing

1: $C \leftarrow type \parallel :$	▷ Type prefix
2: $C \leftarrow C \parallel \text{NORMALIZEWHITESPACE}(content)$	▷ Canonical text
3: $C \leftarrow C \parallel \text{SORTKEYS}(metadata)$	▷ Deterministic JSON
4: $C \leftarrow C \parallel \text{SORT}(children)$	▷ Sorted child IDs
5: return C	

This ensures: (a) semantically identical units in different files produce identical IDs, (b) formatting changes that don't affect semantics (whitespace, comment reordering) don't change IDs if metadata excludes them.

2.4 Language-Specific Parsers

2.4.1 Tree-sitter Based Parsers (Code)

For programming languages, we use [tree-sitter](#) for robust, incremental, error-tolerant parsing.

Design Decision 2.7. Decision: Use tree-sitter as the primary parsing engine for all supported programming languages.

Design Rationale 2.8. 1. **Incremental re-parse:** tree-sitter reuses syntax tree nodes for unchanged regions ($O(\log n)$ edits).

2. **Error tolerance:** Recovers gracefully from syntax errors; produces partial trees.

3. **Unified API:** Same query language (SCM) across 40+ languages.

4. **Bindings:** Rust ([tree-sitter](#)), Python ([tree-sitter-languages](#)), Node.js available.

5. **Grammar maintenance:** Community-maintained grammars; we vendor specific versions.

Tree-sitter Query Patterns. Semantic units are extracted via SCM queries. Example for Python functions:

```
(function_definition
  name: (identifier) @name
  parameters: (parameters) @params
  return_type: (type)? @return_type
```

```

body: (block) @body
decorators: (decorator)* @decorators
) @function

```

The query captures are mapped to `SemanticUnit` fields. A `QueryMapper` per language handles this transformation.

2.4.1.1 Supported Languages

Table 2.2: Tree-sitter Languages Supported (v0.1)

Language	Grammar Version	Unit Types
Python	0.20.1	Module, Class, Function, Method, Decorator, Import, TypeAlias
JavaScript	0.20.0	Module, Class, Function, ArrowFunction, Interface, Type, Import
TypeScript	0.20.0	(extends JS) + TypeAnnotation, Generic
Rust	0.20.1	Module, Struct, Enum, Fn, Impl, Trait, Macro, Use
Go	0.20.0	Package, Struct, Func, Method, Interface, Type, Import
Java	0.20.1	Class, Interface, Method, Field, Constructor, Annotation
C/C++	0.20.1	Function, Struct, Class, Macro, Include, Namespace
C#	0.20.0	Class, Method, Property, Interface, Attribute, Using
SQL	0.20.0	(see §2.4.5)

2.4.2 Markdown Parser

Uses `pulldown-cmark` (Rust) or `markdown-it` (Python) with extensions:

- GitHub Flavored Markdown (GFM): tables, task lists, strikethrough
- Frontmatter (YAML/TOML/JSON) extraction
- Heading anchoring and TOC generation
- Code block language detection
- Link/reference resolution

Units produced: `Document`, `Section(level)`, `Paragraph`, `CodeBlock(lang)`, `Table`, `List`, `Link`, `Image`.

2.4.3 PDF Parser

Uses `pdfplumber` (Python) or `lopdf` (Rust) with optional OCR (`tesseract`):

- Text extraction with position (bbox, font, size)
- Table detection (via `pdfplumber` table extraction)
- Figure/image extraction (bbox, optional OCR)
- Reading order reconstruction (column-aware)

Units: `Page`, `Paragraph`, `Table`, `Figure`, `Header`, `Footer`.

2.4.4 OpenAPI / AsyncAPI Parser

Parses OpenAPI 3.0/3.1 and AsyncAPI 2.x/3.x specs. Produces units for every reusable component:

- Info, Server, Path, Operation
- Schema (with JSON Schema resolution)
- Parameter, RequestBody, Response
- SecurityScheme, Tag, ExternalDocs

Cross-references (`$ref`) are resolved and stored as graph edges in the semantic units.

2.4.5 SQL DDL Parser

Uses `sqlglot` (Python) for multi-dialect parsing (PostgreSQL, MySQL, SQLite, BigQuery, Snowflake, T-SQL). Produces units for: Database, Schema, Table, Column, Index, Constraint, View, Function, Trigger.

Column-level lineage is extracted: Column units reference source columns in SELECT clauses.

2.4.6 Structured Data Parser (YAML/JSON/TOML)

For configuration and data files:

- JSON Schema / OpenAPI schema detection (if `$schema` present)
- Path-based units: each mapping key becomes a unit with JSON path metadata
- Array elements indexed with position
- Merge with schema annotations when available

2.5 Structure-Aware Chunking

After parsing, semantic units may exceed embedding model context windows. The `Chunker` splits units while preserving semantic boundaries.

Interface 2.9 (`Chunker`).

```
fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) ->  
    ↪ Vec<Chunk>;  
fn config_schema(&self) -> ChunkConfigSchema;  
}  
  
struct ChunkConfig {  
    max_tokens: usize,  
    overlap_tokens: usize,  
    respect_boundaries: bool,    // Don't split inside function/class  
    preserve_hierarchy: bool,    // Include parent context  
    strategy: ChunkStrategy,     // "hierarchical", "sliding",  
    ↪ "semantic"  
}
```

2.5.1 Hierarchical Chunking Algorithm

Default strategy for code and structured documents:

Algorithm 4 Hierarchical Chunking

Require: Unit u with children $c_1, \dots, c_n$, config cfg

Ensure: Chunks $\mathcal{C}$ each $\leq cfg.max_tokens$

```

1:  $\mathcal{C} \leftarrow \emptyset$ 
2:  $current \leftarrow \emptyset, current\_tokens \leftarrow 0$ 
3: for all child  $c \in children(u)$  in order do
4:    $child\_tokens \leftarrow TOKENCOUNT(c.content)$ 
5:   if  $current\_tokens + child\_tokens > cfg.max\_tokens$  and  $current \neq \emptyset$  then
6:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{current\}$ 
7:     if  $cfg.overlap\_tokens > 0$  then
8:        $current \leftarrow TAIL(current, cfg.overlap\_tokens)$ 
9:        $current\_tokens \leftarrow TOKENCOUNT(current)$ 
10:    else
11:       $current \leftarrow \emptyset, current\_tokens \leftarrow 0$ 
12:     $current \leftarrow current \parallel c.content$ 
13:     $current\_tokens \leftarrow current\_tokens + child\_tokens$ 
14: if  $current \neq \emptyset$  then
15:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{current\}$ 
16: return  $\mathcal{C}$ 

```

2.5.2 Sliding Window (Fallback)

For plain text or when hierarchical structure is unavailable:

```

# Sliding window with sentence boundary awareness
strategy = "sliding"
window_size = 512      # tokens
overlap = 50           # tokens
split_on = ["\n\n", "\n", ". ", "! ", "? ", "; "]

```

2.5.3 Semantic Chunking (Experimental)

Uses embedding similarity to find natural boundaries:

Algorithm 5 Semantic Chunking via Embedding Similarity**Require:** Sentences $s_1, \dots, s_n$, threshold θ , `max_tokens`**Ensure:** Chunks $\mathcal{C}$

```

1:  $embs \leftarrow \text{EMBED}(s_1, \dots, s_n)$ 
2:  $boundaries \leftarrow \{1\}$ 
3: for  $i = 2$  to  $n$  do
4:    $sim \leftarrow \text{COSINE}(embs[i-1], embs[i])$ 
5:   if  $sim < \theta$  then
6:      $boundaries \leftarrow boundaries \cup \{i\}$ 
7:  $boundaries \leftarrow boundaries \cup \{n+1\}$ 
8:  $\mathcal{C} \leftarrow \emptyset$ 
9: for all consecutive  $(b_j, b_{j+1}) \in boundaries$  do
10:   $chunk \leftarrow \text{MERGESENTENCES}(s_{b_j}, \dots, s_{b_{j+1}-1})$ 
11:  if  $\text{TOKENCOUNT}(chunk) > \text{max\_tokens}$  then
12:     $\mathcal{C} \leftarrow \mathcal{C} \cup \text{SLIDINGWINDOW}(chunk)$ 
13:  else
14:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{chunk\}$ 
15: return  $\mathcal{C}$ 

```

2.5.4 Scale-Free Chunking by Graph Renormalization

The three strategies above share a defect that no amount of tuning removes: each commits to a scale. `window_size = 512` is a claim that the natural unit of meaning in the corpus is 512 tokens wide, and that claim is false for every corpus — a call site, a function, a module, and a subsystem are all legitimate retrieval targets at different query granularities. Fixing the scale at ingestion time forces the retrieval stage to reconstruct the levels that chunking destroyed.

Coarse-graining across scales without committing to one is the subject of the renormalization group. The construction repeatedly integrates out short-range degrees of freedom, generating a flow through a family of effective descriptions in which structures that survive many steps are the physically meaningful ones. Recent work carries this to graphs directly: [Spectral coarse graining authors \[2025\]](#) define a renormalization procedure over the coarse-grained Laplacian that produces reduced representations across scales while preserving both dynamics and large-scale topology, and note explicitly that this differs from clustering methods such as Louvain — renormalization captures characteristic structure *across* scales rather than partitioning at one.

Since LLMWiki already maintains a knowledge graph over units (Vol. 03), the machinery applies without a new data structure.

Definition 2.10 (Renormalization Chunk Ladder). Let G_0 be the unit graph with edges weighted by containment, adjacency, and reference. A chunk ladder is the sequence $G_0 \rightarrow G_1 \rightarrow \dots \rightarrow G_L$ where each $G_{\ell+1}$ is obtained by spectral coarse-graining of G_ℓ , and the chunk set at level ℓ is the set of vertices of G_ℓ , each carrying the concatenated content of the units it subsumes.

Observation 2.11 (Persistence Selects Boundaries). A chunk boundary that survives many renormalization steps separates parts of the corpus that remain weakly coupled under coarse-graining; a boundary that dissolves after one step was an artifact of local token statistics. Boundary quality is therefore measurable as persistence across the ladder,

replacing the unfalsifiable question “is 512 the right window?” with the answerable one “at which level does this boundary disappear?”

Retrieval indexes every level of the ladder, and the query planner selects the level rather than the ingestion configuration doing so: a query naming a specific symbol resolves against G_0 , while a query about a subsystem’s responsibilities resolves against a coarse level whose vertices correspond to that subsystem. The cost is index size, which grows as $\sum_{\ell} |V(G_{\ell})|$ — bounded by a small multiple of $|V(G_0)|$ because coarse-graining contracts the vertex set geometrically.

Status. This is the least mature of the mechanisms specified in this document. Spectral coarse-graining of the unit graph is well-defined and implementable, but the claim that renormalization-persistent boundaries yield better retrieval than tuned fixed-width chunking is an empirical one that the evaluation framework of Vol. 06 must settle before it is relied upon. It is specified here because the ladder subsumes the existing strategies as special cases — $L = 0$ recovers hierarchical chunking exactly — and can therefore be adopted without discarding the current pipeline.

2.6 Entity and Relation Extraction

After chunking, lightweight extraction identifies entities and relations within and across units.

2.6.1 Rule-Based Extraction

Zero-LLM approach using tree-sitter queries and regex patterns:

- **Code:** Function calls, imports, class inheritance, type references
- **Markdown:** Links, cross-references, image references
- **OpenAPI:** Schema references, parameter bindings, auth requirements
- **SQL:** Table/column references, foreign keys, join conditions
- **Config:** Environment variable references, file path references

Produces (source_unit, target_unit, relation_type, confidence) tuples.

2.6.2 LLM-Assisted Extraction (Optional)

For complex relations (e.g., implicit semantic links, cross-file call graphs), a local LLM can be invoked:

```
# Extraction prompt template (Vol. 05 Agent Runtime)
EXTRACT_ENTITIES_RELATIONS = """
Given the following semantic units from a {language} codebase,
extract entities and relations in JSON format.
```

```
Units:
{units_json}
```

```
Entity types: {config.entity_types}
```

Relation types: {config.relation_types}

Output format:

```
{
  "entities": [{"id": "...", "type": "...", "name": "...", "unit_id":
    ↪ "..."}],
  "relations": [{"source": "...", "target": "...", "type": "...",
    ↪ "confidence": 0.0-1.0}]
}
```

2.6.3 Entity Resolution

Extracted entities are resolved against the knowledge graph (Vol. 03):

Algorithm 6 Entity Resolution

Require: Extracted entity e with name n , type t ; graph G

Ensure: Resolved node ID or new node

```
1:  $candidates \leftarrow \text{GRAPHLOOKUP}(G, name = n, type = t)$ 
2: if  $candidates = \emptyset$  then
3:   return  $\text{CREATENODE}(G, e)$  ▷ New entity
4: else
5:    $best \leftarrow \arg \max_{c \in candidates} \text{SIMILARITY}(c, e)$ 
6:   if  $\text{SIMILARITY}(best, e) \geq \tau_{er}$  then
7:     return  $best.id$  ▷ Merge
8:   else
9:     return  $\text{CREATENODE}(G, e)$  ▷ Distinct entity
```

Similarity combines: name fuzzy match, type compatibility, context overlap (shared neighbors), embedding cosine similarity.

2.7 Incremental Parsing

When a file changes, re-parsing only affected regions is critical for latency.

Tree-sitter Incremental Parse. Tree-sitter accepts the old tree and the edit (byte range + new text). It returns a new tree with reused nodes. We compute the diff at unit level:

Algorithm 7 Incremental Unit Diff**Require:** Old units U_{old} , new tree T_{new} , edit range E **Ensure:** (*added*, *modified*, *removed*) unit ID sets

- 1: $U_{new} \leftarrow \text{EXTRACTUNITS}(T_{new})$
- 2: $affected \leftarrow \{u \in U_{old} \mid u.span \cap E \neq \emptyset\}$
- 3: $unchanged \leftarrow U_{old} \setminus affected$
- 4: $modified \leftarrow \{u_{new} \in U_{new} \mid \exists u_{old} \in affected : u_{new}.id = u_{old}.id \wedge u_{new} \neq u_{old}\}$
- 5: $added \leftarrow \{u_{new} \in U_{new} \mid u_{new}.id \notin U_{old}.ids\}$
- 6: $removed \leftarrow \{u_{old} \in affected \mid u_{old}.id \notin U_{new}.ids\}$
- 7: **return** (*added*, *modified*, *removed*)

Only *added* $\cup$ *modified* units are re-embedded and re-indexed. *removed* units are tombstoned in indices.

2.8 Embedding Generation

The **Embedder** trait (Vol. 01, §1.23) abstracts local embedding models.

2.8.1 Supported Models

Table 2.3: Local Embedding Models (Recommended)

Model	Dim	Max Tok	Speed (CPU)	MTEB	License
BGE-M3	1024	8192	~500 tok/s	62.3	MIT
E5-Large-v2	1024	512	~400 tok/s	61.5	MIT
Nomic-Embed-Text-v1.5	768	8192	~600 tok/s	60.1	Apache-2.0
BGE-Small-En-v1.5	384	512	~1200 tok/s	58.2	MIT
Multilingual-E5-Large	1024	512	~350 tok/s	63.1	MIT

2.8.2 Batch Inference

All embeddings use batched inference for throughput:

```
# Embedder config
batch_size = 32
max_seq_len = 512
normalize = true
device = "cpu" # or "cuda", "mps"
onnx_runtime = true # optimized inference
```

Embeddings are stored as `float32` in the HNSW index (Vol. 03) and as `float16` in the unit metadata for memory efficiency.

2.9 Parser Extensibility Guide

To add a custom parser:

1. Implement `Parser` trait 2. Register in `config.toml` 3. Define unit types and metadata schemas 4. (Optional) Add tree-sitter grammar or SCM queries 5. Write tests against `testdata/` fixtures

```
# Example: Custom parser for GraphQL SDL
[ingestion.parsers.graphql]
type = "custom"
module = "my_parsers.graphql_parser"
class = "GraphQLParser"
supported_mimes = ["application/graphql", "text/x.graphql"]
extensions = [".graphql", ".gql"]
```

2.10 Summary

The semantic parsing layer transforms heterogeneous artifacts into a uniform representation of content-addressed, hierarchically structured, metadata-rich semantic units. Key properties:

1. **Language-aware:** Tree-sitter for code, specialized parsers for structured formats
2. **Hierarchical:** Units form trees reflecting source structure
3. **Content-addressed:** SHA-256 identifiers enable deduplication and lineage
4. **Incremental:** Tree-sitter's incremental parsing + unit-level diff
5. **Extensible:** Parser trait + registry for new formats
6. **Metadata-rich:** Typed annotations per unit type for downstream use

The next volume (Vol. 03) details how these units populate the knowledge graph and semantic indices.

Knowledge Graph & Semantic Index

This volume specifies the *knowledge graph* (property graph with typed nodes/edges) and the *semantic index* (hybrid search: vector + keyword + graph adjacency). These two modules form Layer 2 Core (Vol. 01, §1.2.2) and are tightly coupled: the graph provides structural semantics, the index provides fast retrieval, and both share the same content-addressed unit identifiers.

3.1 Knowledge Graph

3.1.1 Data Model

Definition 3.1 (Property Graph). A *property graph* $G = (V, E, \lambda_V, \lambda_E, \sigma_V, \sigma_E)$ where:

- V : set of nodes (entities, concepts, units)
- $E \subseteq V \times V$: set of directed edges (relationships)
- $\lambda_V : V \rightarrow \mathcal{L}_V$: node label function (type)
- $\lambda_E : E \rightarrow \mathcal{L}_E$: edge label function (relation type)
- $\sigma_V : V \rightarrow \mathcal{P}(\text{Key} \times \text{Value})$: node properties
- $\sigma_E : E \rightarrow \mathcal{P}(\text{Key} \times \text{Value})$: edge properties

Node Types.

- **Unit**: A semantic unit from parsing (Vol. 02). Properties: `id` (SHA256), `type`, `span`, `file_path`, `language`, `metadata`(JSON).
- **Entity**: Resolved entity (class, function, table, concept). Properties: `name`, `type`, `canonical_id`, `description`, `embedding_ref`.
- **File**: Source artifact. Properties: `path`, `hash`, `size`, `mtime`, `mime`.
- **Module/Package/Namespace**: Logical grouping. Properties: `name`, `parent_id`.

Edge Types.

- **CONTAINS:** File $\rightarrow$ Unit, Module $\rightarrow$ Unit, Unit $\rightarrow$ Unit (nesting)
- **DEFINES:** Unit $\rightarrow$ Entity (unit defines entity)
- **REFERENCES:** Unit $\rightarrow$ Entity (unit references entity)
- **CALLS:** Entity $\rightarrow$ Entity (function call)
- **INHERITS:** Entity $\rightarrow$ Entity (class inheritance)
- **IMPORTS:** Module $\rightarrow$ Module
- **RELATES:** Entity $\leftrightarrow$ Entity (semantic similarity, co-occurrence)
- **NEXT_TO:** Unit $\rightarrow$ Unit (sequential proximity in source)

Content-Addressed Identity. Every Unit node has `id = SHA256(canonical_form)`. Entity nodes have `canonical_id` derived from name + type + context hash. This ensures deterministic identity across re-ingestion.

3.1.2 Graph Store Backend

The `GraphStore` trait (Vol. 01, §1.24) admits multiple backends:

Table 3.1: Graph Store Backend Comparison

Backend	Embedded	Scale	Transactions	License
RocksDB (custom)	Yes	10B+ edges	Yes (WAL)	Apache-2.0
SQLite (with ext)	Yes	100M edges	Yes (WAL)	Public Domain
Redwood (Rust)	Yes	1B edges	Yes	MIT
Neo4j Embedded	No	100B edges	Yes	GPL/Commercial
Kuzu	Yes	10B edges	Yes	MIT

Default: RocksDB-based custom store (`llmwiki-graph`). Nodes and edges stored as separate column families with prefix encoding for efficient prefix scans (by type, by partition).

3.1.2.1 Storage Layout (RocksDB)

Column Families:

```

|-- nodes:{type}      # Key: node_id (fixed 32 bytes), Value: serialized
  ↳ props
|-- edges:{rel_type}  # Key: src_id(32) | dst_id(32), Value: edge props
|-- adj:out:{type}    # Key: src_id, Value: list of (rel_type, dst_id) -
  ↳ adjacency
|-- adj:in:{type}     # Key: dst_id, Value: list of (rel_type, src_id) -
  ↳ reverse adj
|-- idx:label:{label} # Key: label | node_id, Value: empty (label index)
|-- idx:prop:{key}    # Key: prop_value | node_id, Value: empty (property
  ↳ index)
`-- meta              # Key: "stats", "version", "config", Value: JSON

```

3.1.3 Entity Resolution

When a parser emits a **Unit** that **DEFINES** or **REFERENCES** an entity, the graph layer resolves it to a canonical **Entity** node.

Algorithm 8 Entity Resolution

Require: Unit u with entity mention $m = (name, type, context)$

Ensure: Resolved **Entity** node e

```

1:  $candidates \leftarrow \text{LOOKUPBYNAMETYPE}(G, m.name, m.type)$ 
2: if  $candidates = \emptyset$  then
3:   return  $\text{CREATEENTITY}(m, u)$ 
4:  $best \leftarrow \arg \max_{c \in candidates} \text{SCORE}(c, m, u)$ 
5: if  $\text{SCORE}(best, m, u) \geq \tau_{er}$  then
6:    $\text{UPDATEENTITY}(best, u)$  ▷ Merge context, update embedding
7:   return  $best$ 
8: else
9:   return  $\text{CREATEENTITY}(m, u)$  ▷ New distinct entity

```

Scoring function:

$$\begin{aligned}
 \text{SCORE}(c, m, u) = & \alpha \cdot \text{FuzzyMatch}(c.name, m.name) \\
 & + \beta \cdot \text{TypeCompat}(c.type, m.type) \\
 & + \gamma \cdot \text{ContextOverlap}(c.context, u.context) \\
 & + \delta \cdot \text{CosineSim}(c.embedding, u.embedding)
 \end{aligned}$$

Default weights: $\alpha = 0.3, \beta = 0.2, \gamma = 0.2, \delta = 0.3$. Threshold $\tau_{er} = 0.85$ (configurable).

3.1.4 Incremental Graph Updates

On ingestion diff ($added, modified, removed$) units (Vol. 02, Alg. 7):

Algorithm 9 Incremental Graph Update

Require: Diff sets A (added), M (modified), R (removed)

Ensure: Updated graph G'

```

1: for all  $u \in R$  do
2:    $\text{DELETENODE}(u.id)$  ▷ Tombstone
3:    $\text{DELETEEDGES}(u.id)$  ▷ All incident edges
4: for all  $u \in A \cup M$  do
5:    $\text{UPSERTUNITNODE}(u)$ 
6:    $entities \leftarrow \text{EXTRACTENTITIES}(u)$ 
7:   for all  $e \in entities$  do
8:      $resolved \leftarrow \text{RESOLVEENTITY}(e, u)$  ▷ Alg. 8
9:     if  $e.relation = \text{DEFINES}$  then
10:       $\text{UPSERTEDGE}(u.id, resolved, \text{DEFINES})$ 
11:     else
12:       $\text{UPSERTEDGE}(u.id, resolved, \text{REFERENCES})$ 
13:    $\text{UPSERTADJACENCY}(u)$  ▷ Contains, Next_To
14:  $\text{COMMITTRANSACTION}()$ 
15: return  $G$ 

```

All operations within a single ingestion are batched in one RocksDB write batch (atomic).

3.1.5 Graph Traversal Primitives

The `traverse` and `neighborhoods` methods (Vol. 01, §1.24) support:

- **Pattern matching:** Cypher-like declarative patterns (e.g., `(a:Function)-[:CALLS*1..3]->(b:Function)`)
- **Bounded BFS/DFS:** k -hop neighborhoods with edge-type filters
- **Personalized PageRank:** For HippoRAG-style retrieval (Vol. 04)
- **Shortest path:** Between two entities
- **Subgraph extraction:** Induced subgraph for context packing

```
# TraversalPattern examples
pattern = {
  "start": { "type": "Function", "name": "process_order" },
  "hops": 3,
  "edge_types": ["CALLS", "REFERENCES"],
  "direction": "out",
  "filter": { "node_type": ["Function", "Class"] },
  "limit": 100
}
```

3.2 Semantic Index

The `SemanticIndex` (Vol. 01, §1.25) is a hybrid index combining three retrieval signals:

1. **Vector (HNSW):** Dense embedding similarity
2. **Keyword (BM25):** Sparse lexical matching
3. **Graph Adjacency:** Structural proximity in knowledge graph

3.2.1 HNSW Vector Index

Implementation: `hnswlib` (C++ with Python/Rust bindings) or `faiss` (with disk-backed index). Disk-backed mode: memory-mapped file for vectors, metadata in separate store.

Configuration:

```
hnsw:
  M: 16                      # max connections per node
  ef_construction: 200       # build-time search width
  ef_search: 64              # query-time search width
  dim: 1024                  # embedding dimension
  space: "cosine"            # or "l2", "ip"
  disk_backed: true
  mmap: true
```

Content-Addressed Vectors. Vectors are stored keyed by `unit_id` (SHA256). The index maps `unit_id` $\rightarrow$ internal HNSW label. On upsert: if unit exists, update vector; if new, add. On delete: mark tombstone (lazy cleanup during compaction).

3.2.2 BM25 Keyword Index

Implementation: `tantivy` (Rust) or `OpenSearch` (if distributed). Tantivy is embedded, fast, and supports incremental updates.

Schema:

```
fields:
- unit_id: Keyword (stored, indexed) # SHA256
- content: Text (analyzer: language-specific)
- file_path: Keyword (stored, indexed)
- unit_type: Keyword (stored, indexed)
- language: Keyword (stored, indexed)
```

Analyzer: Language-specific (e.g., `code` analyzer for source: split on punctuation, camel-Case, `snake_case`; `standard` for prose).

3.2.3 Graph Adjacency Index

For query-time graph expansion (Vol. 04), we maintain a lightweight adjacency index separate from the full graph store:

```
Adjacency Index (RocksDB column family "adj:index"):
Key: unit_id (32 bytes)
Value: serialized adjacency list
[
  { "rel": "CONTAINS", "target": "sha256...", "weight": 1.0 },
  { "rel": "REFERENCES", "target": "sha256...", "weight": 0.8 },
  { "rel": "NEXT_TO", "target": "sha256...", "weight": 0.3 }
]
```

Updated incrementally alongside graph. Used for `search_graph` in hybrid retrieval.

3.2.4 Hybrid Search

The `search_hybrid` method combines three signals:

Algorithm 10 Hybrid Search**Require:** Query q (text + optional embedding), k , weights w_v, w_k, w_g **Ensure:** Top- k scored units

```

1:  $q_v \leftarrow \text{EMBED}(q.\text{text})$  ▷ If not provided
2:  $R_v \leftarrow \text{HNSWSEARCH}(q_v, k \times 3)$ 
3:  $R_k \leftarrow \text{BM25SEARCH}(q.\text{text}, k \times 3)$ 
4:  $R_g \leftarrow \text{GRAPHEXPAND}(q, k \times 2)$  ▷ Adjacency traversal
5:  $\text{scores} \leftarrow \emptyset$ 
6: for all  $r \in R_v \cup R_k \cup R_g$  do
7:    $s_v \leftarrow r \in R_v ? \text{Score}(r, R_v) : 0$ 
8:    $s_k \leftarrow r \in R_k ? \text{Score}(r, R_k) : 0$ 
9:    $s_g \leftarrow r \in R_g ? \text{Score}(r, R_g) : 0$ 
10:   $\text{scores}[r.\text{id}] = w_v \cdot s_v + w_k \cdot s_k + w_g \cdot s_g$ 
11: return  $\text{TOPK}(\text{scores}, k)$ 

```

Default weights: $w_v = 0.6, w_k = 0.3, w_g = 0.1$ (configurable per query type).

Score normalization: Reciprocal Rank Fusion (RRF) or min-max normalization per signal before weighted sum.

3.2.5 Index Update Strategy

Vector (HNSW): Incremental add/update. HNSW supports dynamic insertion. Deletes = tombstones. Periodic `rebuild_threshold` triggers full rebuild.

BM25 (Tantivy): Incremental commit. Segments merged in background. Delete = term tombstone.

Graph Adjacency: Updated in same transaction as graph (RocksDB batch).

3.3 Query-Time Graph Expansion

Used by retriever to expand candidate set via graph structure:

1. **Seed:** Start from vector/keyword hits S
2. **Expand:** For each $s \in S$, fetch k -hop neighborhood $N_k(s)$
3. **Score:** Personalized PageRank from S over induced subgraph
4. **Merge:** Add top- m PPR nodes to candidate pool

This implements the “graph traversal” retrieval path complementary to vector/keyword.

3.4 Performance Characteristics

3.5 Patent-Relevant Novelty

Patent Claim 3.2. Claim 4: A knowledge graph system wherein: (a) all nodes are content-addressed via SHA-256 of their canonical representation, (b) entity resolution

Table 3.2: Expected Performance (Single Node, SSD)

Operation	Latency (p99)	Throughput
Node upsert (batch 1000)	< 50 ms	20k nodes/s
Edge upsert (batch 5000)	< 100 ms	50k edges/s
k -hop traversal ($k=2$, 100 seeds)	< 10 ms	10k queries/s
PPR (100 seeds, 3 hops)	< 50 ms	2k queries/s
HNSW search ($k=10$, $ef=64$)	< 5 ms	2k queries/s
BM25 search ($k=10$)	< 3 ms	3k queries/s
Hybrid search ($k=20$)	< 15 ms	1k queries/s

merges mentions using a weighted combination of name similarity, type compatibility, context overlap, and embedding similarity, (c) the graph is persisted in a column-family store with prefix-encoded keys enabling type-partitioned scans, and (d) a separate graph adjacency index enables sub-millisecond k -hop expansion for hybrid retrieval.

Patent Claim 3.3. Claim 5: The system of Claim 4 wherein incremental updates to the graph and indices are executed atomically within a single write batch, and deterministic replay is achieved by re-applying the ingestion log against an empty store.

3.6 Summary

The knowledge graph and semantic index form the queryable substrate of LLMWiki:

1. **Graph:** Property graph with content-addressed nodes, entity resolution, and incremental updates
2. **Vector Index:** HNSW with disk-backed storage, SHA-256 keyed
3. **Keyword Index:** Tantivy BM25 with language-aware analyzers
4. **Adjacency Index:** Lightweight graph expansion for structural retrieval
5. **Hybrid Search:** RRF-weighted fusion of three signals

All indices share the same unit identifiers, enabling seamless cross-index joins and evidence tracing (Vol. 06).

The next volume (Vol. 04) details the retrieval pipeline including the Evidence Gate.

Retrieval and Evidence Gate

This volume specifies the *retrieval pipeline*—the hybrid search mechanism that fuses vector, keyword, and graph signals—and the *Evidence Gate*, the mandatory verification stage that ensures every retrieved unit cited in an answer is semantically relevant to the query. This is the critical differentiator from prior RAG systems: retrieval is necessary but not sufficient; evidence verification is mandatory.

4.1 Evidence Gate: Core Principle

Design Principle 4.1 (Evidence Closure). No retrieval result reaches the synthesis stage without passing cross-encoder verification against a calibrated threshold. This enforces *evidence closure*: every citation in an answer traces to a verified unit.

Definition 4.2 (Evidence Gate). The Evidence Gate is a function $\mathcal{G} : \mathcal{Q} \times \mathcal{U}^* \times \Theta \rightarrow \mathcal{U}^*$ that takes a query q , a candidate set $\mathcal{R} \subseteq \mathcal{U}$, and a threshold $\tau \in \Theta$, and returns only those units $u \in \mathcal{R}$ where $\text{score}(q, u) \geq \tau$, where score is computed by a cross-encoder.

4.2 Why Not Reranking?

The Evidence Gate is **not** a reranker. Key distinctions:

Table 4.1: Reranker vs. Evidence Gate

Property	Reranker	Evidence Gate
Output	Reordered top- k	Filtered subset (size $\leq k$)
Threshold	Implicit (top- k)	Explicit calibrated τ
Goal	Improve ranking	Enforce relevance / reject noise
Failure mode	Bad results still returned	Empty set $\rightarrow$ “no evidence”
Citation policy	Top- k cited	Only verified cited
Calibration	Rarely calibrated	Mandatory calibration

This distinction is fundamental: a reranker always returns k results; the Evidence Gate may return 0 (triggering “insufficient evidence” response), which is the correct behavior when no unit actually supports the query.

4.3 Retrieval Pipeline

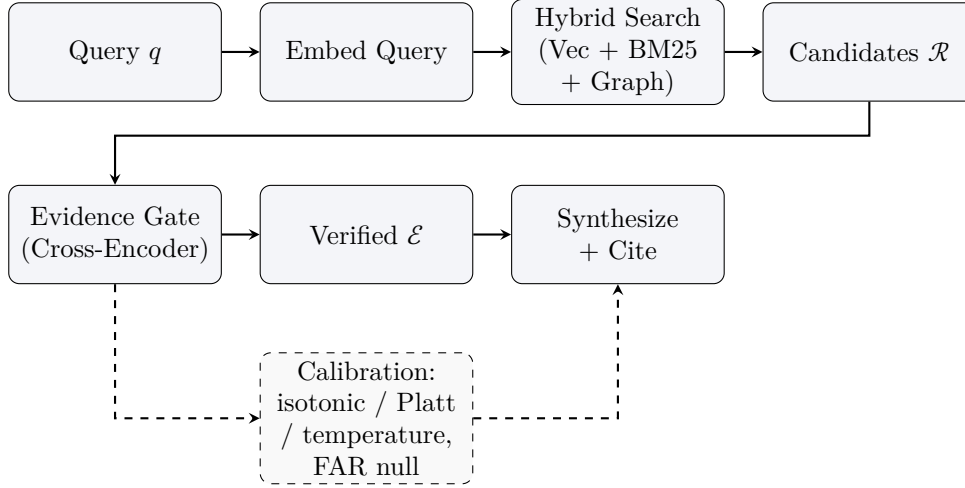


Figure 4.1: Retrieval + Evidence Gate pipeline.

4.3.1 Stage 1: Query Embedding

Query text q is embedded using the same model as ingestion (config: `[index.hnsw.dim]`). For multi-vector queries (e.g., ColBERT-style), each token embedding is preserved.

4.3.2 Stage 2: Hybrid Search

Detailed in Vol. 03 §3.2.4. Returns candidate set $\mathcal{R}$ of size k_{cand} (default 50).

4.3.3 Stage 3: Evidence Gate

Cross-Encoder Model. A transformer cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`, `jinaai/jina-reranker-v2-base`) scores (q, u) pairs:

$$\text{score}(q, u) = \text{sigmoid}(\text{CrossEncoder}([q; \text{SEP}; u.\text{content}])) \quad (4.1)$$

Batch Processing. Candidates are scored in batches (config: `[evidence_gate.batch_size]`).

Thresholding. Units with score $< \tau$ are discarded. τ is calibrated per model/dataset (see §4.4).

Algorithm 11 Evidence Gate

Require: Query q , candidates $\mathcal{R}$, threshold τ , batch size B

Ensure: Verified units $\mathcal{E}$

```

1:  $\mathcal{E} \leftarrow \emptyset$ 
2: for  $i = 1$  to  $\lceil |\mathcal{R}|/B \rceil$  do
3:    $batch \leftarrow \mathcal{R}[(i-1)B : iB]$ 
4:    $scores \leftarrow \text{CROSSENCODERBATCH}(q, batch)$ 
5:   for all  $(u, s) \in \text{ZIP}(batch, scores)$  do
6:     if  $s \geq \tau$  then
7:        $\mathcal{E} \leftarrow \mathcal{E} \cup \{(u, s)\}$ 
8: return  $\text{SORTBYScoreDESC}(\mathcal{E})$ 

```

4.3.4 Multi-Stage Proofreading Cascade

Algorithm 11 performs *single-step* discrimination: one score, one threshold. Its achievable error rate is bounded by the separation of the score distributions of relevant and irrelevant units under a single measurement. This is the direct analogue of equilibrium binding discrimination in molecular recognition, where selectivity cannot exceed the ratio of dissociation constants.

Biological systems face an identical constraint and defeat it. A T cell must discriminate agonist from self peptide despite very small differences in receptor binding affinity, yet achieves error rates far below the equilibrium bound. The mechanism, due to Hopfield [1974] and Ninio [1975], is *kinetic proofreading*: the recognition event is decomposed into n sequential intermediate states, each of which may irreversibly discard the candidate, and each of which is driven out of equilibrium by energy dissipation. Recent work has refined this picture, showing that parallel reaction threads on a single receptor improve discrimination fidelity without lengthening the sequential chain [Kirby, D. and collaborators, 2025], and that multimolecular schemes can escape the classical activity–fidelity trade-off [Cell Systems authors, 2026].

We adopt this architecture directly.

Design Principle 4.3 (Proofreading over Thresholding). Evidence verification is a cascade of n irreversible discard stages of increasing cost and decreasing throughput, not a single thresholded score. Fidelity is purchased with computation, and the exchange rate is quantifiable.

Definition 4.4 (Proofreading Cascade). A proofreading cascade of depth n is a sequence of verifiers $g_1, \dots, g_n$ with thresholds $\tau_1, \dots, \tau_n$ and unit costs $c_1 \leq \dots \leq c_n$. A candidate u survives stage j iff $g_j(q, u) \geq \tau_j$, and is discarded irreversibly otherwise. The gate output is the set of candidates surviving all n stages.

Theorem 4.5 (Multiplicative Error Suppression). Let ε_j denote the false-acceptance probability of stage j on an irrelevant unit, and ρ_j its true-acceptance probability on a relevant unit. If the stage verdicts are conditionally independent given relevance, the cascade attains

$$\varepsilon_{casc} = \prod_{j=1}^n \varepsilon_j, \quad \rho_{casc} = \prod_{j=1}^n \rho_j. \quad (4.2)$$

For homogeneous stages ($\varepsilon_j = \varepsilon$, $\rho_j = \rho$) the false-acceptance rate decays as ε^n while recall decays only as ρ^n ; since $\rho \gg \varepsilon$ by construction, the discrimination ratio $\rho_{casc}/\varepsilon_{casc} = (\rho/\varepsilon)^n$ grows exponentially in depth.

The conditional independence premise is the load-bearing assumption, and it is not free: stacking n copies of the same cross-encoder yields perfectly correlated verdicts and no gain whatsoever. Independence is engineered by making the stages read *different evidence channels*, which is the retrieval analogue of the parallel reaction threads of Kirby, D. and collaborators [2025]:

1. **Stage 1 — Lexical admissibility** (c_1 negligible): BM25 span overlap between query terms and unit content. Rejects candidates retrieved purely by embedding drift.
2. **Stage 2 — Cross-encoder relevance** (c_2 moderate): Algorithm 11 as specified above.

3. **Stage 3 — Structural admissibility** (c_3 low): the unit must be reachable in the knowledge graph from at least one other surviving unit within k hops, or be the unique survivor. Rejects semantically plausible but contextually orphaned units.
4. **Stage 4 — Entailment verification** (c_4 high): a natural language inference model must find the candidate answer span entailed, not merely topically related, by the unit.

Cost. Naively a cascade is n times more expensive. It is not, because survival is exponentially rare: with per-stage survival fraction s_j the expected cost per initial candidate is

$$C = c_1 + \sum_{j=2}^n c_j \prod_{i < j} s_i, \quad (4.3)$$

so the expensive terminal stages are evaluated on a vanishing fraction of the candidate set. Ordering stages by increasing $c_j/(1-s_j)$ minimises C — cheap, highly selective filters first.

Observation 4.6 (Fidelity Requires Dissipation). In the thermodynamic setting, proof-reading fidelity beyond the equilibrium bound is possible only if the cycle consumes free energy; a cascade run at equilibrium provides no discrimination gain. The computational analogue is exact: the discard steps are irreversible, and the cascade’s advantage over a single threshold is paid for strictly in compute. This yields an accuracy-versus-compute frontier for evidence verification rather than an unbounded accuracy claim, and we regard the existence of that frontier as a design constraint rather than a limitation to be hidden.

4.3.5 Stage 4: Synthesis with Forced Citations

The verified set $\mathcal{E}$ is packed into the LLM context with explicit citation markers:

```
[EVIDENCE]
Source: units/ab/cdef1234.json (Function: process_payment)
Content: def process_payment(amount: Decimal, currency: str) -> bool:
    "\"\"Process a payment transaction...\"\"\"
    validate_currency(currency)
    ...
Relevance: 0.94
[/EVIDENCE]

[EVIDENCE]
Source: units/ef/5678abcd.json (Class: PaymentProcessor)
Content: class PaymentProcessor:
    def __init__(self, gateway: PaymentGateway):
        self.gateway = gateway
    ...
Relevance: 0.87
[/EVIDENCE]

ANSWER THE QUERY USING ONLY THE EVIDENCE ABOVE.
IF EVIDENCE IS INSUFFICIENT, SAY: "Insufficient evidence to answer."
CITE SOURCES AS [E1], [E2], ...
```

This forces the LLM to ground every claim in the provided evidence.

4.4 Threshold Calibration

The threshold τ must be calibrated to achieve desired precision/recall tradeoff. Un-calibrated cross-encoder scores are not probabilities.

4.4.1 Calibration Methods

1. **Isotonic Regression:** Non-parametric, fits piecewise constant function. Requires validation set $\mathcal{D}_{val} = \{(q, u, y)\}$ where $y \in \{0, 1\}$ is relevance judgement.
2. **Platt Scaling:** Fits logistic regression $\sigma(As + B)$ to scores s . Simpler, less flexible.
3. **Temperature Scaling:** Single parameter $T > 0$, calibrated $\sigma(s/T)$. Most efficient.

Algorithm 12 Calibration Procedure

Require: Validation set $\mathcal{D}_{val} = \{(q_i, u_i, y_i)\}$, cross-encoder f

Ensure: Calibrated threshold τ^* for target precision P_{target}

- 1: $scores \leftarrow [f(q_i, u_i) \text{ for } (q_i, u_i, y_i) \in \mathcal{D}_{val}]$
 - 2: $labels \leftarrow [y_i \text{ for } (q_i, u_i, y_i) \in \mathcal{D}_{val}]$
 - 3: Fit calibrator C on $(scores, labels)$ $\triangleright$ Isotonic/Platt/Temperature
 - 4: $cal_scores \leftarrow C(scores)$
 - 5: $\tau^* \leftarrow$ threshold achieving P_{target} on $\mathcal{D}_{val}$
 - 6: **return** τ^*, C
-

Calibration artifacts are stored in `.llmwiki/evidence/cross-encoder/calibration.json` and loaded at startup.

4.4.2 Label-Free Calibration by Discordant Pairing

Algorithm 12 requires $\mathcal{D}_{val}$ with human relevance judgements y_i . This is the binding practical constraint on every deployment: relevance labels are expensive, corpus-specific, and stale the moment the corpus changes. A threshold that cannot be recalibrated without re-annotation is a threshold that will silently drift.

Gravitational-wave astronomy confronts a structurally identical problem — assigning significance to candidate events when no labelled catalogue of true detections exists — and solves it without labels. The technique is *time slides* [Was et al., 2010, Nitz et al., 2018]: triggers from one detector are shifted in time by more than the inter-detector light travel time, so that any coincidence formed from the shifted streams *cannot* correspond to a real astrophysical signal. Repeating the shift many times yields a large empirical sample of pure-noise coincidences, from which the false alarm rate (FAR) is read off as a function of the ranking statistic. Detection thresholds are then quoted in operational units — LIGO’s public alert threshold corresponds to approximately one false alarm per two months.

We construct the same empirical null for evidence scoring.

Definition 4.7 (Discordant Pair). A discordant pair (q, u^-) is a query paired with a unit drawn such that genuine relevance is excluded by construction, not by judgement. Admissible constructions are:

1. **Corpus displacement:** u^- is drawn from a repository disjoint in domain from the one q was issued against.

2. **Temporal displacement:** u^- is drawn from a WAL snapshot predating the creation of every entity referenced by q , so no unit in that snapshot can describe them.
3. **Graph displacement:** u^- is drawn from a connected component of the knowledge graph containing no entity mentioned in q .

Algorithm 13 False Alarm Rate Estimation without Relevance Labels

Require: Query sample Q , corpora $\{\mathcal{U}_1, \dots, \mathcal{U}_m\}$, scorer f , displacement count N

Ensure: Empirical null $\hat{F}_0$, threshold τ^* for target FAR*

```

1:  $S_0 \leftarrow \emptyset$ 
2: for  $i = 1$  to  $N$  do
3:   for all  $q \in Q$  do
4:      $u^- \leftarrow \text{DRAWDISCORDANT}(q, \{\mathcal{U}_j\})$  ▷ Def. 4.7
5:      $S_0 \leftarrow S_0 \cup \{f(q, u^-)\}$ 
6:    $\hat{F}_0(s) \leftarrow |\{s' \in S_0 : s' \geq s\}| / |S_0|$  ▷ Empirical survival function
7:    $\tau^* \leftarrow \min\{s : \hat{F}_0(s) \cdot \bar{k} \leq \text{FAR}^*\}$  ▷  $\bar{k}$  = mean candidates per query
8: return  $\hat{F}_0, \tau^*$ 

```

Reporting. Every admitted unit carries not a raw score but a false alarm rate: “*evidence of this strength arises from an unrelated corpus once per $1/(\hat{F}_0(s)\bar{k})$ queries.*” This is an operationally meaningful quantity that a threshold of 0.72 is not.

Invariant 4.8 (Operational Significance). No unit enters the synthesis context without an associated FAR computed against the current empirical null. Recalibration is triggered whenever the WAL advances past a configured number of commits, and requires no human annotation.

Limitations, stated plainly. Two are inherited directly from the source domain and one is specific to ours.

- **Null contamination.** Time slides work because the light travel time bound is a physical guarantee. Our displacements are weaker guarantees: a unit from a “disjoint” repository may still be generically relevant (a logging idiom, a licence header). Contaminated discordant pairs inflate the null’s upper tail and make the gate conservative. Domain disjointness must be asserted from repository metadata, never assumed.
- **Estimator saturation.** The gravitational-wave literature reports that FAR estimation error saturates with the number of time shifts [Was et al., 2010]; beyond a point, additional displacements re-use the same triggers and add no independent information. The same ceiling applies to N in Algorithm 13, and we therefore treat N as tuned to the saturation point rather than maximised.
- **Null $\neq$ alternative.** FAR bounds the rate of *spurious* admission. It says nothing about recall. FAR calibration therefore complements, and does not replace, the labelled conformal procedure of §4.4.3: the empirical null fixes the operating point without labels, and conformal prediction supplies coverage guarantees when labels are available.

4.4.3 Confidence Intervals

Beyond point threshold, we provide confidence intervals per prediction:

- **Conformal Prediction:** Split conformal for distribution-free coverage guarantees
- **MC Dropout:** Enable dropout at inference, multiple forward passes $\rightarrow$ uncertainty
- **Ensemble:** Multiple cross-encoders $\rightarrow$ variance as uncertainty

Default: Temperature scaling + conformal prediction with 95% coverage.

4.5 Global Consistency of Evidence

Everything specified so far verifies each unit *independently* against the query. This leaves a hole that no amount of per-unit accuracy can close: two units may each be strongly relevant to q and yet assert incompatible things about it. A gate built from pairwise scores admits both, the synthesis stage receives a contradictory evidence set, and the resulting answer is fluently grounded in mutually exclusive sources. Evidence closure as defined in §4.1 is a statement about provenance, not about coherence, and is therefore not yet closed.

The obstruction to assembling locally valid data into a globally valid whole is the subject of sheaf theory, where it is not a metaphor but the definition. A sheaf assigns data to open sets and specifies restriction maps between them; the global sections H^0 are exactly the consistent global assignments, and the first cohomology H^1 measures the obstruction to gluing local data that is pairwise compatible into a global section. Cellular sheaves make this computable over graphs, and recent work has applied the construction to distributed computation — where terminating solutions to a task are precisely the global sections of a task sheaf [Sheaf-theoretic distributed systems authors, 2025] — to consistency constraints on networks [Network sheaf authors, 2025], and to characterising irreducible error patterns in predictive coding networks [Predictive coding sheaf authors, 2025].

Definition 4.9 (Evidence Sheaf). Let $\mathcal{E}$ be the set of units surviving the proofreading cascade, and let $G_{\mathcal{E}}$ be the graph with vertices $\mathcal{E}$ and an edge $\{u, v\}$ whenever u and v share an entity or stand in a graph relation. The evidence sheaf $\mathcal{F}$ over $G_{\mathcal{E}}$ assigns

- to each vertex u , the stalk $\mathcal{F}(u)$ of admissible truth assignments to the atomic claims extracted from u ;
- to each edge $\{u, v\}$, the stalk $\mathcal{F}(uv)$ of assignments to the claims shared between them;
- restriction maps $\mathcal{F}_{u \leq uv} : \mathcal{F}(u) \rightarrow \mathcal{F}(uv)$ induced by natural language inference between the claim sets.

Definition 4.10 (Evidence Closure, Cohomological Form). An evidence set $\mathcal{E}$ is *closed* for query q iff the evidence sheaf $\mathcal{F}$ admits a global section supporting the answer, i.e. $H^0(G_{\mathcal{E}}; \mathcal{F}) \neq \emptyset$ and the answer’s claims lie in its image. The set is *obstructed* iff $H^1(G_{\mathcal{E}}; \mathcal{F}) \neq 0$.

This supersedes the provenance-only definition. A nonzero H^1 certifies that no consistent reading of the retrieved evidence exists, regardless of how high the individual relevance scores are.

Algorithm 14 Consistency Gate**Require:** Verified units $\mathcal{E}$, query q **Ensure:** Coherent subset $\mathcal{E}^*$ or abstention with conflict witness

```

1:  $G_{\mathcal{E}} \leftarrow \text{BUILD OVERLAP GRAPH}(\mathcal{E})$ 
2:  $\mathcal{F} \leftarrow \text{BUILDEVIDENCESHEAF}(G_{\mathcal{E}})$  ▷ Def. 4.9
3:  $Z \leftarrow \text{COMPUTE } H^0(G_{\mathcal{E}}, \mathcal{F})$  ▷ Consistent global assignments
4: if  $Z \neq \emptyset$  then
5:   return  $\mathcal{E}$  ▷ Closed; proceed to synthesis
6:  $B \leftarrow \text{COMPUTE } H^1 \text{ SUPPORT}(G_{\mathcal{E}}, \mathcal{F})$  ▷ Edges carrying the obstruction
7:  $\mathcal{E}^* \leftarrow \text{MAXCONSISTENTSUBSET}(\mathcal{E}, B)$ 
8: if  $\text{SUPPORTS}(\mathcal{E}^*, q)$  then
9:   return  $\mathcal{E}^*$  with conflict report  $B$ 
10: return  $\text{ABSTAIN}$ ("contradictory evidence", witness  $B$ )
```

Why cohomology rather than pairwise conflict detection. A pairwise contradiction check finds edges where two units disagree. It cannot detect *cyclic* inconsistency, in which every pair is individually compatible but no assignment satisfies the cycle — the situation where u is compatible with v , v with w , and w with u , yet the three cannot hold simultaneously. This is precisely the regime H^1 was invented to measure, and it is the regime in which a system reporting high pairwise confidence is most confidently wrong. The cohomology computation additionally *localises* the failure: the support of the obstruction class identifies which relations carry it, so the system can report a conflict witness rather than an unexplained refusal.

Cost. For the linear sheaves used here, H^0 and H^1 reduce to kernel and cokernel computations on the sheaf Laplacian, polynomial in $|\mathcal{E}|$. Since $|\mathcal{E}|$ after the cascade is small by construction (§4.3.4), the consistency gate is not the bottleneck; the NLI evaluations that populate the restriction maps are, and they are bounded by the number of overlap edges.

4.6 Query Planner Integration

The Query Planner (Vol. 07) decomposes complex queries into subqueries, each executed through the retrieval + evidence gate pipeline:

Query: "How does the payment module handle refunds?"

Plan:

```

Subquery 1: "refund processing logic" -> Search + Gate -> [E1, E2]
Subquery 2: "refund API endpoints" -> Search + Gate -> [E3]
Subquery 3: "refund error codes" -> Search + Gate -> [E4, E5]
```

Synthesis: Combine [E1..E5] with citations

Each subquery has its own budget and can use different retrieval parameters (e.g., more graph hops for API questions, more vector weight for logic questions).

4.7 Evidence Gate Config

```
[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72                # calibrated threshold
calibration = "isotonic"        # "none" | "platt" | "isotonic" |
    ↪ "temperature"
batch_size = 16
max_candidates = 50              # max inputs to gate
max_verified = 20               # max outputs from gate
timeout_ms = 2000               # per-query gate timeout

# Conformal prediction
conformal = true
coverage = 0.95
calibration_set = "eval/evidence_calibration.jsonl"
```

4.8 Evaluation Metrics

The Evidence Gate is evaluated on:

Table 4.2: Evidence Gate Evaluation Metrics

Metric	Definition	Target
Precision@k	$\frac{ \{u \in \text{top-}k(\mathcal{E}): y_u=1\} }{k}$	> 0.85
Recall@k	$\frac{ \{u \in \mathcal{E}: y_u=1\} }{ \{u: y_u=1\} }$	> 0.75
F1	Harmonic mean of P/R	> 0.80
AUC-ROC	Area under ROC curve	> 0.90
ECE (Expected Calibration Error)	$\sum_b \frac{ B_b }{N} acc(B_b) - conf(B_b) $	< 0.05
Conformal Coverage	Empirical coverage of prediction sets	≈ 0.95
Empty Rate	$\frac{ \{q: \mathcal{E}(q)=\emptyset\} }{ Q }$	< 0.10

4.9 Comparison with Prior Art

Table 4.3: Retrieval + Verification in Prior Art

System	Hybrid Search	Cross-Enc.	Mandatory Gate	Label-Free FAR	Global Consistency
Standard RAG	✗(vec only)	✗	✗	✗	✗
RAG with Reranker	✓(vec)	✓	✗	✗	✗
GraphRAG	✓(vec+graph)	✗	✗	✗	✗
HippoRAG	✓(graph PPR)	✗	✗	✗	✗
LightRAG	✓(dual-level)	✗	✗	✗	✗
REALM/RA-CM3	✗	✗	✗	✗	✗
LLMWiki	✓(3-signal)	✓(cascade)	✓	✓	✓(H^1)

The final two columns are the ones without prior occupants. No deployed retrieval

system reports significance against an empirical null constructed without relevance labels, and none certifies that its returned evidence set is mutually consistent rather than merely individually relevant.

4.10 Patent-Relevant Novelty

Patent Claim 4.11. Claim 6: A retrieval system comprising: (a) a hybrid searcher combining vector similarity, keyword matching, and graph adjacency traversal, (b) an evidence gate that applies a cross-encoder to every retrieval candidate, (c) a calibrated threshold that rejects candidates below a precision-targeted score, and (d) a synthesis stage that only cites units passing the evidence gate, ensuring evidence closure.

Patent Claim 4.12. Claim 7: The system of Claim 6 wherein the evidence gate operates as a filter (not a reranker), such that zero candidates may be returned when no evidence meets the threshold, triggering an explicit “insufficient evidence” response rather than hallucinated synthesis.

Patent Claim 4.13. Claim 8: The system of Claim 6 wherein the evidence gate comprises a cascade of $n \geq 2$ verification stages reading mutually distinct evidence channels, each stage irreversibly discarding candidates below a stage-specific threshold, ordered by increasing unit cost, such that the false-acceptance rate decays multiplicatively in the number of stages while the expected cost per candidate remains bounded by the survival-weighted sum of stage costs.

Patent Claim 4.14. Claim 9: The system of Claim 6 wherein the threshold is derived without relevance annotations by (a) constructing query–unit pairs for which relevance is excluded by corpus, temporal, or graph displacement, (b) accumulating scorer outputs over said pairs into an empirical null distribution, and (c) selecting the threshold attaining a target false alarm rate under said distribution, the false alarm rate being reported with each admitted unit.

Patent Claim 4.15. Claim 10: The system of Claim 6 further comprising a consistency gate that constructs a cellular sheaf over the verified evidence set with restriction maps induced by inference between overlapping claims, and abstains from synthesis when the first cohomology of said sheaf is nonzero, emitting the support of the obstruction class as a conflict witness.

Patent Discussion 4.16. Prior art (rerankers, RAG, GraphRAG) never enforces evidence closure: they always return top- k for synthesis. The mandatory gate with calibrated threshold is a structural invariant, not a heuristic.

Claims 8–10 narrow the invention away from the “combination of known components” objection that a bare hybrid-search-plus-reranker claim invites. Claim 8 recites a specific cascade discipline — irreversible discard, channel-distinct stages, cost ordering — with a stated quantitative consequence, rather than generic multi-stage filtering. Claim 9 recites a construction procedure for the null distribution, which is the non-obvious element: the displacement conditions are what make the estimate valid, and no retrieval prior art constructs relevance nulls at all. Claim 10 recites a topological consistency criterion that is not reducible to pairwise contradiction checking, since cyclic inconsistency is invisible to any pairwise test.

4.11 Summary

The retrieval pipeline transforms a query into a verified, mutually consistent evidence set:

1. **Hybrid Search:** Vector (HNSW) + Keyword (BM25) + Graph (PPR/adjacency)
2. **Proofreading Cascade:** n -stage irreversible verification over distinct evidence channels; error suppression multiplicative in depth
3. **Calibration:** label-free FAR from an empirical null (§4.4.2), complemented by con-formal coverage when labels exist
4. **Consistency Gate:** sheaf cohomology over the surviving set; $H^1 \neq 0 \Rightarrow$ abstain with conflict witness
5. **Forced Citations:** Synthesis only from verified units
6. **Evidence Closure:** cohomological form (Def. 4.10); every citation traces to a verified unit admitting a consistent global reading

This pipeline is the enforcement mechanism for LLMWiki’s core invariant: *answers are evidence-bounded*. The strengthening over the volume’s opening statement is deliberate — evidence closure begins as a provenance property and ends as a verifiable topological one.

Agent Runtime

This volume specifies the *agent runtime*—the execution environment that carries out query plans, orchestrates tool calls, manages conversation state, and streams results. The runtime is the bridge between the Query Planner (Vol. 07) and the external world (API, CLI, LLM providers).

5.1 Design Goals

Design Principle 5.1 (Stateless Execution). Each query execution is a self-contained episode. No persistent agent state across queries; all context is passed explicitly in the plan and conversation history.

Design Principle 5.2 (Tool-First Architecture). LLMs are not the primary actors; tools are. The LLM plans and decides; tools execute deterministically. This inverts the typical “agent = LLM + tools” model.

Design Principle 5.3 (Streaming by Default). All long-running operations (search, synthesis, tool calls) stream partial results. The API responds with Server-Sent Events (SSE) or WebSocket frames.

Design Principle 5.4 (Budget Enforcement). Every execution has configurable budgets: token budget, time budget, tool call budget, cost budget. Exceeding any budget terminates with a partial result.

Design Principle 5.5 (Observability Native). Every decision (plan step, tool choice, retrieval result, synthesis token) is logged with structured traces. OpenTelemetry compatible.

5.2 Runtime Architecture

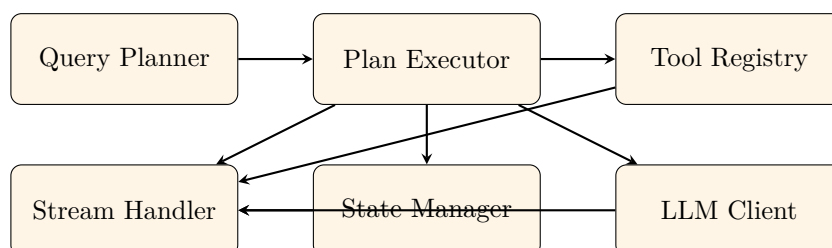


Figure 5.1: Agent Runtime components.

5.2.1 Core Components

- **Plan Executor:** Traverses the query plan DAG, executes nodes in topological order, handles parallelism and dependencies.
- **Tool Registry:** Maps tool names to implementations, handles invocation, timeouts, retries, and result normalization.
- **LLM Client:** Abstracts LLM providers (local: llama.cpp, vLLM, ollama; remote: OpenAI, Anthropic, etc.) with unified streaming interface.
- **State Manager:** Maintains scratchpad, conversation history, intermediate results, and budget counters.
- **Stream Handler:** Emits SSE/WebSocket events for partial results, progress, errors, and final answer.

5.3 Plan Execution Model

5.3.1 Query Plan Structure

The planner (Vol. 07) produces a `QueryPlan`:

```
struct QueryPlan {
    id: PlanId,
    query: String,
    steps: Vec<PlanStep>,
    edges: Vec<(StepId, StepId)>, // DAG dependencies
    budget: ExecutionBudget,
    config: PlanConfig,
}

struct PlanStep {
    id: StepId,
    kind: StepKind,           // Search, Synthesize, ToolCall, Aggregate,
    ↪ Branch
    tool: Option<ToolName>,
    input: StepInput,         // Template with variable interpolation
    output: StepOutput,       // Expected output schema
    depends_on: Vec<StepId>,
    parallel: bool,
    retry: RetryPolicy,
    timeout_ms: u64,
}
```

5.3.2 Execution Algorithm

Algorithm 15 Plan Execution

Require: Plan P , initial state S_0

Ensure: Final state S_f with answer

```

1:  $S \leftarrow S_0$ ,  $completed \leftarrow \emptyset$ 
2:  $ready \leftarrow \{s \in P.steps \mid s.depends\_on \subseteq completed\}$ 
3: while  $ready \neq \emptyset$  do
4:   if  $\exists s \in ready : s.parallel$  then
5:      $batch \leftarrow \{s \in ready \mid s.parallel\}$ 
6:      $results \leftarrow EXECUTEPARALLEL(batch, S)$ 
7:   else
8:      $s \leftarrow POP(ready)$ 
9:      $results \leftarrow EXECUTESTEP(s, S)$ 
10:   $S \leftarrow UPDATESTATE(S, results)$ 
11:   $completed \leftarrow completed \cup \{s.id \mid s \in batch \vee s\}$ 
12:   $ready \leftarrow \{s \in P.steps \setminus completed \mid s.depends\_on \subseteq completed\}$ 
13:  CHECKBUDGET( $S$ ,  $P.budget$ ) ▷ May raise BudgetExceeded
14:  EMITPROGRESS( $S$ ,  $completed$ ) ▷ Stream update
15: return  $S$ 

```

5.3.3 Step Kinds

- Search: Invoke hybrid search + evidence gate (Vol. 04)
- Synthesize: Call LLM with evidence context, stream tokens
- ToolCall: Invoke registered tool (see §5.4)
- Aggregate: Merge results from parallel branches
- Branch: Conditional execution based on prior result

5.4 Tool System

5.4.1 Tool Trait

```

trait Tool {
  fn name(&self) -> &str;
  fn description(&self) -> &str;
  fn schema(&self) -> ToolSchema;           // JSON Schema for input
  fn output_schema(&self) -> ToolSchema;    // JSON Schema for output
  fn execute(&self, input: ToolInput) -> Result<ToolOutput>;
  fn execute_stream(&self, input: ToolInput) -> Stream<ToolOutput>;
  fn timeout_ms(&self) -> u64 { 30000 }
  fn retry(&self) -> RetryPolicy { RetryPolicy::default() }
}

```

5.4.2 Built-in Tools

Table 5.1: Built-in Tools

Tool	Input	Output
search	SearchInput{query, k, filters}	SearchOutput{results[], citations[]}
graph_traverse	TraverseInput{start, pattern, hops}	Subgraph{nodes[], edges[]}
read_unit	UnitId	SemanticUnit
read_file	FilePath	FileContent
llm_complete	CompletionInput{prompt, params}	Stream<String>
extract_entities	ExtractInput{text, types}	Entities{entities[], relations[]}
summarize	SummarizeInput{units[], max_len}	String
verify_evidence	VerifyInput{query, units[]}	VerifiedUnits{verified[]}

5.4.3 Custom Tools

Users register custom tools via config:

```
[tools.custom.my_tool]
module = "my_tools"
class = "MyTool"
timeout_ms = 10000
retry = { max_attempts = 2, backoff_ms = 500 }
```

Tools can be synchronous (return **Result**) or streaming (return **Stream**). Streaming tools emit partial results for long operations (e.g., file scanning, API pagination).

5.5 LLM Client Abstraction

```
trait LLMClient {
  fn complete(&self, req: CompletionRequest) ->
    ↪ Result<CompletionResponse>;
  fn stream(&self, req: CompletionRequest) -> Stream<CompletionChunk>;
  fn embed(&self, texts: Vec<String>) -> Result<Vec<Vec<f32>>>>;
  fn models(&self) -> Vec<ModelInfo>;
  fn count_tokens(&self, text: &str) -> usize;
}
```

Providers:

- **Local:** llama.cpp (server), vLLM, ollama, TGI
- **Remote:** OpenAI, Anthropic, Azure, Vertex, Bedrock
- **Unified API:** OpenAI-compatible endpoint

The runtime selects the provider per plan step (e.g., small local model for planning, large model for synthesis).

5.6 State Management

```
struct ExecutionState {
```

```

plan_id: PlanId,
step_results: HashMap<StepId, StepResult>,
scratchpad: Scratchpad,           // Key-value store for LLM use
conversation: Vec<Message>,       // User/assistant messages
evidence: Vec<VerifiedUnit>,      // Accumulated evidence
budget: BudgetTracker,           // Tokens, time, cost, calls
trace: TraceContext,             // OpenTelemetry trace
}

```

Scratchpad: Structured key-value store accessible to LLM via tool calls. Used for intermediate reasoning, variable passing between steps, and user-facing “thoughts”.

Budget Tracker: Hard limits with configurable actions on exceed:

```

budget:
  max_tokens: 100000
  max_time_ms: 120000
  max_tool_calls: 50
  max_cost_usd: 0.50
  on_exceed: "truncate" # "error" | "truncate" | "partial"

```

5.7 Budget as Allocation, Not Truncation

The budget tracker of §5.6 is a circuit breaker: it observes consumption and terminates when a limit is crossed. It never influences *which* work is done, only when work stops. Under a binding budget this is close to the worst available policy, because the steps that happen to be scheduled early consume the budget regardless of their value, and the terminating action — truncate, partial, error — degrades an answer that a different ordering might have completed.

Time-domain astronomy operates permanently in this regime and does not treat it as an error condition. The Vera C. Rubin Observatory’s survey will produce on the order of 10^4 transient alerts per night against spectroscopic follow-up capacity of order 10^1 ; the deficit is four orders of magnitude and is structural, not a provisioning failure. The response is a broker layer that ranks candidates in real time under hard resource constraints. [AAS2RTO authors \[2025\]](#) score candidates and fold in observability to produce a continually updated priority list; the Fink broker applies real-time active learning to select the follow-up that most improves classification rather than the one with the highest current score [[Fink Collaboration, 2025](#)]; and [Follow-up scheduling authors \[2025\]](#) formulate follow-up scheduling as an online resource-constrained problem solved by reinforcement learning.

The third of these is the important one. Ranking by score selects what the system already believes; ranking by expected information gain selects what would change its belief.

Design Principle 5.6 (Budget Allocation over Budget Enforcement). The execution budget is an allocation problem solved continuously during execution, not a limit checked after the fact. Steps compete for budget on expected evidential gain per unit cost, and the enforcement action is the degenerate case reached when no step clears the bar.

Definition 5.7 (Evidential Value Density). For a candidate step s with estimated cost $c(s)$ in budget units, let $\Delta H(s)$ be the expected reduction in uncertainty over the answer induced by executing s . The value density is $\nu(s) = \Delta H(s)/c(s)$.

Uncertainty here is not a free parameter: the evidence gate already produces a calibrated quantity. Using the false alarm rates of §4.4.2, the current evidence set induces a distribution over admissible answers, and $\Delta H(s)$ is estimated as the expected decrease in its entropy — the step most worth taking is the one whose result is least predictable from what is already verified.

Algorithm 16 Budget-Aware Step Scheduling

Require: Plan DAG P , budget B , state S

Ensure: Answer or partial result with accounting

```

1:  $F \leftarrow \text{READYFRONTIER}(P, S)$  ▷ Steps with satisfied dependencies
2: while  $F \neq \emptyset$  and  $B$  not exhausted do
3:   for all  $s \in F$  do
4:      $\hat{c}(s) \leftarrow \text{ESTIMATECOST}(s)$  ▷ From historical step statistics
5:      $\widehat{\Delta H}(s) \leftarrow \text{ESTIMATEGAIN}(s, S)$  ▷ Entropy of answer distribution
6:      $s^* \leftarrow \arg \max_{s \in F} \widehat{\Delta H}(s) / \hat{c}(s)$ 
7:     if  $\widehat{\Delta H}(s^*) < \delta_{\min}$  then
8:       break ▷ No step justifies its cost; stop early and under budget
9:      $S \leftarrow \text{EXECUTE}(s^*, S)$ ;  $B \leftarrow B - \text{ACTUALCOST}(s^*)$ 
10:     $F \leftarrow \text{READYFRONTIER}(P, S)$  ▷ Re-plan: results may prune or add steps
11: return  $\text{SYNTHESIZE}(S)$ ,  $\text{BUDGETREPORT}(B)$ 

```

Two consequences distinguish this from enforcement. First, the loop can terminate *under* budget when no remaining step earns its cost, which a circuit breaker never does — it spends everything it is given. Second, when the budget does bind, what survives is the highest-value-density prefix rather than an arbitrary prefix, so the partial result is the best obtainable at that price rather than whatever happened to run first.

Observation 5.8 (Estimator Quality Bounds the Gain). Algorithm 16 is only as good as $\widehat{\Delta H}$ and $\hat{c}$. Cost estimation is tractable, since step costs are empirically regular and improve with logged history. Gain estimation is genuinely hard and a poor estimator degrades the schedule toward arbitrary ordering — which is to say, toward the behaviour of the circuit breaker it replaces. The failure mode is therefore bounded below by the previous design, which is why the allocation policy can be deployed incrementally.

5.8 Streaming Protocol

API uses Server-Sent Events (SSE) over HTTP or WebSocket:

```

# Event types
event: plan_start      { "plan_id": "...", "steps": N }
event: step_start      { "step_id": "...", "kind": "Search" }
event: step_progress   { "step_id": "...", "progress": 0.5, "details":
  ↪  "..." }
event: evidence        { "unit_id": "...", "relevance": 0.94, "preview":
  ↪  "..." }
event: token           { "token": "the", "step_id": "..." }
event: step_complete   { "step_id": "...", "output": {...} }
event: plan_complete   { "answer": "...", "citations": [...], "usage":
  ↪  {...} }

```

```

event: error          { "step_id": "...", "error": "..." }
event: budget_warning { "type": "tokens", "used": 80000, "limit": 100000 }

```

Clients can render progressive UI: show plan, show evidence as it arrives, stream answer tokens.

5.9 Error Handling and Retries

- **Transient errors** (network, timeout): Retry per step policy (exponential backoff)
- **Tool errors**: Captured in `StepResult.error`, execution continues if step not critical
- **LLM errors**: Rate limit → backoff; context overflow → truncate/summarize history
- **Budget exceeded**: Emit `budget_warning`, then `plan_complete` with partial answer
- **Fatal errors**: Invalid plan, OOM, provider unavailable → abort with error event

5.10 Conversation Management

Multi-turn conversations maintain history in `ExecutionState.conversation`. The planner can reference prior turns for context. History is pruned to fit budget (summarize old turns, keep recent).

5.11 Patent-Relevant Novelty

Patent Claim 5.9. Claim 8: An agent runtime wherein: (a) query plans are DAGs executed by a deterministic executor with topological scheduling, (b) tools are first-class citizens with JSON Schema contracts, streamable execution, and configurable timeouts/retries, (c) LLM invocation is abstracted behind a provider-agnostic client supporting both local and remote models, (d) execution state includes a structured scratchpad accessible to tools and LLM, and (e) hard budgets (tokens, time, cost, calls) are enforced at the executor level with configurable overflow behavior.

5.12 Summary

The Agent Runtime executes query plans as deterministic, budgeted, observable tool orchestrations:

1. **Plan executor**: DAG-aware, parallel-capable, topological
2. **Tool system**: Schema-contracted, streamable, extensible
3. **LLM abstraction**: Multi-provider, local-first, token-aware
4. **State**: Scratchpad, evidence accumulator, budget tracker
5. **Streaming**: SSE/WebSocket with fine-grained event types

6. **Budgets:** Hard limits with graceful degradation

The runtime is the “CPU” of LLMWiki; the Query Planner (Vol. 07) is the compiler that generates its instruction sequences.

Evaluation Framework

This volume defines the evaluation methodology for LLMWiki: benchmarks, metrics, experimental protocols, and the automated evaluation pipeline. Rigorous evaluation is not an afterthought; it is a first-class component that validates the architectural claims (evidence closure, deterministic replay, local-first quality).

6.1 Evaluation Philosophy

Design Principle 6.1 (Measure What Matters). Standard RAG metrics (recall@k, MRR) are insufficient. We evaluate: **evidence quality** (are citations actually relevant?), **answer correctness** (ground truth comparison), **attribution accuracy** (claim-to-evidence mapping), and **system properties** (latency, determinism, replay fidelity).

Design Principle 6.2 (Local-First Evaluation). All evaluation runs locally. No external API calls for ground truth generation or metric computation. LLM-as-judge is optional and confined to specific qualitative assessments.

Design Principle 6.3 (Regression-Proof). Evaluation datasets, prompts, and thresholds are version-controlled alongside code. CI runs full evaluation on every PR; regressions block merge.

6.2 Benchmark Suite

6.2.1 Internal Benchmarks

Table 6.1: LLMWiki Internal Benchmarks

Benchmark	Domain	Size	Focus
CodeQA-Python	Python codebases	50 repos, 200K units	Code understanding, cross-file references
API-Doc	OpenAPI specs	100 specs, 50K ops	Schema-to-natural-language
SQL-Schema	SQL DDL	200 schemas, 10K tables	Table/column/relationship QA
Tech-Docs	Markdown/PDF	500 docs, 1M tokens	Long-document QA, structure
Config-Files	YAML/JSON/TOML	1000 configs	Config understanding, env vars
Mixed-Corpus	All above	1M units	Cross-domain retrieval

Construction Process:

1. **Seed corpora:** Curated real-world repositories (Apache, Kubernetes, etc.)
2. **Question generation:** Template-based + LLM-assisted (local model) for diversity

3. **Expert review:** Domain experts verify questions and ground truth answers
4. **Evidence annotation:** Human annotators mark exact unit spans supporting each answer
5. **Quality control:** Inter-annotator agreement (Cohen’s $\kappa > 0.8$)

6.2.2 External Benchmarks (Adapted)

We adapt standard benchmarks to LLMWiki’s evidence-required setting:

- **Natural Questions (NQ):** Filter to questions answerable from our test corpora
- **HotpotQA:** Multi-hop; verify graph traversal quality
- **MS MARCO:** Passage ranking; compare hybrid search vs. baselines
- **BEIR:** 18 datasets; zero-shot retrieval quality
- **CodeSearchNet:** Code retrieval; evaluate parser+embedder quality
- **Spider / BIRD:** Text-to-SQL; evaluate SQL parser + graph

6.3 Metric Definitions

6.3.1 Retrieval Metrics

Definition 6.4 (Evidence Recall@k). Fraction of ground-truth evidence units recovered in top- k after Evidence Gate:

$$\text{EvRecall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|}{|\mathcal{E}^*(q)|} \quad (6.1)$$

where $\mathcal{E}^*(q)$ = human-annotated evidence units, $\mathcal{E}_k(q)$ = gate-verified top- k .

Definition 6.5 (Evidence Precision@k).

$$\text{EvPrecision@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|}{|\mathcal{E}_k(q)|} \quad (6.2)$$

Definition 6.6 (Evidence F1@k). Harmonic mean of EvRecall and EvPrecision.

Definition 6.7 (Gate Rejection Rate). Fraction of retrieved candidates rejected by Evidence Gate:

$$\text{GateReject} = \frac{|\mathcal{R}| - |\mathcal{E}|}{|\mathcal{R}|} \quad (6.3)$$

High rejection with maintained EvRecall indicates effective noise filtering.

6.3.2 Answer Quality Metrics

Definition 6.8 (Answer Correctness (Exact Match / F1 / Semantic)). Given ground truth answer a^* and predicted answer $\hat{a}$:

- **EM**: Exact string match (normalized)
- **Token-F1**: Token-level F1
- **Semantic**: Embedding cosine $\geq \tau$ (e.g., 0.85 with BGE-M3)

Definition 6.9 (Attribution Score). Measures whether every claim in the answer is supported by cited evidence.

1. Segment answer into atomic claims $c_1, \dots, c_n$ (via NLI model or LLM)
2. For each claim c_i , check if $\exists e \in \text{citations} : \text{NLI}(e, c_i) = \text{Entailment}$
3. $\text{Attribution} = \frac{1}{n} \sum_i \mathbb{1}[\text{supported}(c_i)]$

Definition 6.10 (Citation Quality). For each cited unit u , score = $\text{CrossEncoder}(q, u)$. Average over all citations. Penalizes “hallucinated citations” (citing irrelevant units).

6.3.3 System Property Metrics

- **Deterministic Replay Fidelity**: Bitwise equality of answers across re-runs (modulo LLM sampling)
- **Ingestion Latency**: ms/unit (p50, p99)
- **Query Latency**: End-to-end p50/p95/p99
- **Index Size**: GB per million units
- **Memory Footprint**: RSS at rest / under load
- **Uptime / Crash Recovery**: Time to replay WAL and serve queries

6.4 Evaluation Pipeline

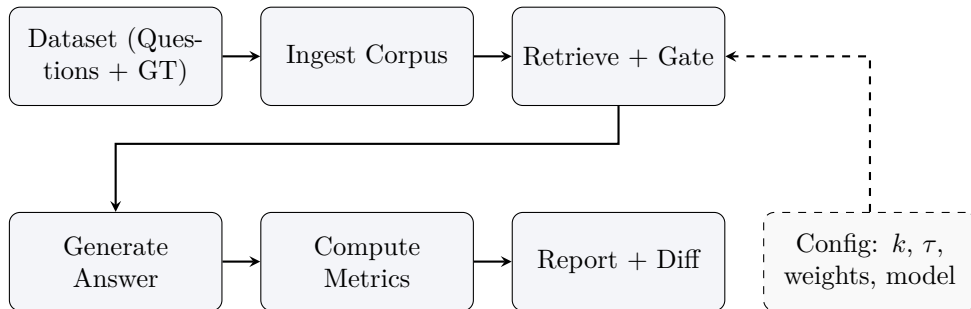


Figure 6.1: Automated evaluation pipeline.

6.4.1 Configuration Matrix

We sweep key parameters to understand tradeoffs:

```
# eval_config.yaml
retrieval:
  k: [5, 10, 20, 50]
  hybrid_weights:
    - [1.0, 0.0, 0.0] # Vector only
    - [0.0, 1.0, 0.0] # BM25 only
    - [0.6, 0.3, 0.1] # Default hybrid
  gate_threshold: [0.5, 0.6, 0.7, 0.75, 0.8, 0.9]
embedders:
  - "bge-m3"
  - "e5-large-v2"
  - "nomic-embed-text-v1.5"
llms:
  - "llama-3.1-8b-instruct" # Local
  - "llama-3.1-70b-instruct" # Local (quantized)
  - "gpt-4o-mini" # Remote (optional)
```

Each combination = one evaluation run. Results stored in `eval/results/` with full config hash for reproducibility.

6.4.2 Statistical Rigor

- **Bootstrap confidence intervals:** 1000 resamples, 95% CI for all metrics
- **Paired significance:** Wilcoxon signed-rank test for pairwise model comparisons
- **Multiple comparison correction:** Benjamini-Hochberg FDR control
- **Effect sizes:** Cohen's d reported alongside p -values

6.5 Regression Detection

```
# .github/workflows/eval.yml
name: Evaluation
on: [pull_request, push]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run evaluation
        run: |
          llmwiki eval run --config eval_config.yaml --baseline main
      - name: Check regressions
        run: |
          llmwiki eval check --threshold 0.02 # 2% regression tolerance
      - name: Upload report
```

```
uses: actions/upload-artifact@v4
with:
  name: eval-report
  path: eval/report.html
```

Baseline is the `main` branch. Any metric regression $> 2\%$ (configurable) blocks merge.

6.6 Qualitative Evaluation

Automated metrics are supplemented by:

1. **Expert review:** Domain experts rate answer quality on 1-5 scale (helpful, accurate, well-cited)
2. **Failure analysis:** Categorize errors (missing evidence, wrong synthesis, gateway false negative, etc.)
3. **Ablation studies:** Remove components (graph, gate, planner) to measure contribution
4. **User studies:** Task completion time, trust ratings, citation usefulness

6.7 Patent-Relevant Novelty

Patent Claim 6.11. Claim 9: An evaluation framework for semantic operating layers characterized by: (a) evidence-centric metrics (Evidence Recall/Precision/F1) that measure retrieval quality after mandatory cross-encoder verification, (b) attribution scoring that decomposes answers into atomic claims and verifies each against cited evidence using NLI, (c) a configuration matrix sweep that characterizes system behavior across retrieval, gating, and synthesis parameters, and (d) deterministic replay verified via bitwise answer comparison across re-runs.

6.8 Summary

LLMWiki’s evaluation framework goes beyond standard RAG benchmarks by:

1. **Evidence-first metrics:** Quality of verified evidence, not just raw retrieval
2. **Attribution verification:** Claim-level NLI checking against citations
3. **Configuration sweeps:** Understanding tradeoffs, not single-point numbers
4. **Regression gates:** CI-enforced quality floors
5. **System properties:** Determinism, latency, replay—all measured

This ensures that architectural improvements (Vol. 01–05) translate to measurable user value.

Deployment and Operations

This volume covers deploying LLMWiki in production: containerization, local-first modes, distributed scaling, monitoring, backup/recovery, and upgrade procedures.

7.1 Deployment Modes

Table 7.1: LLMWiki Deployment Modes

Mode	Use Case	Components	Scale	Deps
Embedded (Library)	Desktop app, CLI, tests	Rust/Python lib	1 process	None
Standalone Server	Single-user, team server	API + workers	10K QPS	Docker
High Availability	Production service	Load balancer + replicas	100K QPS	K8s, Redis
Air-gapped	Secure environments	Full offline bundle	10K QPS	None
Edge / Mobile	On-device	WASM / mobile lib	1 QPS	None

7.2 Container Images

Dockerfile multi-stage build:

```
# Build stage
FROM rust:1.78 AS builder
WORKDIR /app
COPY . .
RUN cargo build --release --features api,embed-onnx

# Runtime stage
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates libssl3 libgomp1 && \
    rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/target/release/llmwiki /usr/local/bin/
COPY --from=builder /app/config/default.toml /etc/llmwiki/
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["llmwiki", "server"]
```

Images published to ghcr.io/llmwiki/llmwiki:version, latest, sha.

7.2.1 Image Variants

- `llmwiki:cpu` - Default, ONNX Runtime CPU
- `llmwiki:cuda` - CUDA 12.x, TensorRT optimized
- `llmwiki:rocm` - AMD ROCm support
- `llmwiki:minimal` - No embedder, external embedding service
- `llmwiki:full` - All parsers, local LLM (llama.cpp)

7.3 Kubernetes Deployment

7.3.1 Helm Chart Structure

```
llmwiki/
|-- Chart.yaml
|-- values.yaml
|-- values-prod.yaml
|-- templates/
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- ingress.yaml
|   |-- configmap.yaml
|   |-- secret.yaml
|   |-- pvc.yaml
|   |-- hpa.yaml
|   |-- servicemonitor.yaml
|   `-- _helpers.tpl
`-- charts/
    `-- redis/          # Optional: for distributed cache
```

7.3.2 Key Configuration

```
# values.yaml
replicaCount: 3
image:
  repository: ghcr.io/llmwiki/llmwiki
  tag: "0.1.0"
  pullPolicy: IfNotPresent

resources:
  limits:
    cpu: "2000m"
    memory: "4Gi"
  requests:
    cpu: "1000m"
    memory: "2Gi"

persistence:
  enabled: true
```

```
size: 100Gi
storageClass: fast-ssd
accessMode: ReadWriteOnce

config:
  repository:
    id: "prod-main"
  ingestion:
    embedder:
      model: "bge-m3"
      device: "cuda"
  index:
    hnsw:
      ef_search: 128
  evidence_gate:
    threshold: 0.72
  api:
    auth: "oidc"
    rate_limit: 1000

hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPU: 70
  targetMemory: 80
```

7.3.3 Stateful Considerations

LLMWiki pods are **stateful** due to `.llmwiki/` directory. Options:

1. **Shared PVC (ReadWriteMany)**: NFS, CephFS, Longhorn RWX. Single writer enforced by file lock.
2. **Per-pod PVC + Sync Sidecar**: Each pod has PVC; sidecar runs `rsync` or `git sync` for convergence. More complex but no RWX dependency.
3. **Object store backend**: S3/GCS for units/graph/index. Requires GraphStore/Index implementations with S3 backend.

Default: Per-pod PVC + `lsyncd` sidecar for near-real-time sync.

7.4 Local-First Deployment

7.4.1 Single Binary

```
# Install
curl -L https://github.com/llmwiki/llmwiki/releases/latest/download/llmwiki_
  ↪ ki-x86_64-unknown-linux-gnu.tar.gz | tar xz
sudo mv llmwiki /usr/local/bin/
```

```
# Initialize repository
llmwiki init --repo ~/.llmwiki

# Ingest
llmwiki ingest ~/projects/my-codebase

# Run server
llmwiki server --bind 127.0.0.1:8080
```

7.4.2 Docker Compose (Local Stack)

```
# docker-compose.yml
version: '3.8'
services:
  llmwiki:
    image: ghcr.io/llmwiki/llmwiki:cpu
    ports: ["8080:8080"]
    volumes:
      - ./data:/data
      - ./config.toml:/etc/llmwiki/config.toml:ro
    environment:
      - LLMWIKI_REPO=/data/.llmwiki
      - RUST_LOG=info
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

# Optional: local LLM for planning/synthesis
ollama:
  image: ollama/ollama
  volumes: [".ollama:/root/.ollama"]
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu]
```

7.4.3 Air-Gapped Installation

1. Download `llmwiki-airgap.tar.gz` (contains binary, models, grammars, config)
2. Extract on target machine
3. Run `./install.sh` (sets up systemd service, configures firewall)
4. No external network access ever required

Air-gap bundle includes:

- Embedding models (ONNX): bge-m3, e5-large-v2
- Cross-encoder: ms-marco-MiniLM-L-6-v2
- Tree-sitter grammars: all 20+ languages
- Optional local LLM: llama-3.1-8b-instruct-q4_k_m.gguf

7.5 Configuration Management

```
# Config precedence (highest wins):
# 1. CLI flags           --config.key=value
# 2. Environment variables LLMWIKI_CONFIG_KEY=value
# 3. Config file         /etc/llmwiki/config.toml
# 4. Defaults            built-in

# Example env overrides
LLMWIKI_REPOSITORY_ID=prod-main
LLMWIKI_INGESTION_EMBEDDER_MODEL=bge-m3
LLMWIKI_INDEX_HNSW_EF_SEARCH=128
LLMWIKI_EVIDENCE_GATE_THRESHOLD=0.72
LLMWIKI_API_AUTH=oidc
LLMWIKI_API_OIDC_ISSUER=https://auth.example.com
LLMWIKI_LOG_LEVEL=info
LLMWIKI_LOG_FORMAT=json
```

Config schema validated at startup (using `config-rs` + JSON Schema).

7.6 Monitoring and Observability

7.6.1 Metrics (Prometheus)

Exposed at `/metrics`:

```
# Ingestion
llmwiki_ingestion_units_total{status="success|failed"}
llmwiki_ingestion_latency_seconds{quantile="0.5|0.95|0.99"}
llmwiki_ingestion_bytes_total

# Query
llmwiki_query_total{status="success|failed|timeout"}
llmwiki_query_latency_seconds{quantile="0.5|0.95|0.99"}
llmwiki_query_retrieval_candidates
llmwiki_query_evidence_verified
llmwiki_query_evidence_rejected
llmwiki_query_synthesis_tokens

# System
llmwiki_graph_nodes_total
llmwiki_graph_edges_total
```

```
llmwiki_index_vectors_total
llmwiki_wal_size_bytes
llmwiki_storage_size_bytes

# LLM (if local)
llmwiki_llm_inference_duration_seconds
llmwiki_llm_tokens_per_second
```

7.6.2 Distributed Tracing (OpenTelemetry)

Every query execution produces a trace:

```
Span: query.execution
|-- Span: query.plan
|-- Span: retrieval.hybrid
|   |-- Span: retrieval.vector
|   |-- Span: retrieval.bm25
|   `-- Span: retrieval.graph
|-- Span: evidence.gate
|   `-- Span: evidence.cross_encoder_batch (repeated)
|-- Span: synthesis.llm
`-- Span: response.stream
```

Exporters: OTLP HTTP/gRPC, Zipkin, Jaeger, stdout (dev).

7.6.3 Structured Logging

JSON logs with fields:

```
{
  "timestamp": "2024-01-15T10:30:00.123Z",
  "level": "INFO",
  "message": "Query executed",
  "query_id": "q_abc123",
  "user_id": "user_456",
  "latency_ms": 245,
  "retrieval_candidates": 47,
  "evidence_verified": 12,
  "plan_depth": 3,
  "plan_nodes": 5
}
```

7.6.4 Health Checks

- GET /health/live - Liveness (process alive)
- GET /health/ready - Readiness (indices loaded, config valid)
- GET /health/deep - Deep check (sample query executes)

7.7 Backup and Recovery

7.7.1 Backup Strategy

1. **WAL replay:** `.llmwiki/wal/` is append-only; replay from any point
2. **Snapshots:** Periodic `tar` of `.llmwiki/` (exclude `wal/*.jsonl` if replay from snapshot)
3. **Incremental:** `rsync --link-dest` for efficient daily backups
4. **Verification:** Restore to temp dir, run `llmwiki verify` (checksums, query smoke test)

7.7.2 Automated Backup (CronJob)

```
# Kubernetes CronJob
apiVersion: batch/v1
kind: CronJob
metadata:
  name: llmwiki-backup
spec:
  schedule: "0 2 * * *" # Daily 2 AM
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: ghcr.io/llmwiki/backup:sid
              env:
                - name: SOURCE_DIR
                  value: "/data/.llmwiki"
                - name: DEST_BUCKET
                  value: "s3://llmwiki-backups/prod"
                - name: ENCRYPTION_KEY
                  valueFrom:
                    secretKeyRef:
                      name: llmwiki-backup-key
                      key: key
              command: ["/backup.sh"]
          restartPolicy: OnFailure
```

7.7.3 Disaster Recovery

1. Provision new PVC / disk
2. `rsync` latest backup to `.llmwiki/`
3. Start single pod with `--replay-wal` flag
4. Verify `llmwiki verify --deep`
5. Scale to target replicas

RTO: 5 min (depends on data size). RPO: 24h (daily backup) or continuous (WAL shipping to S3).

7.8 Upgrades and Migrations

7.8.1 Version Compatibility

- **Patch (0.1.x)**: Rolling upgrade, zero downtime
- **Minor (0.x.0)**: May require index rebuild; blue-green deploy recommended
- **Major (x.0.0)**: Schema migration; downtime required; migration tool provided

7.8.2 Index Migration

When HNSW parameters change (M, ef_construction) or embedder model changes:

```
# Rebuild index from units
llmwiki index rebuild --repo .llmwiki --model bge-m3 --parallel 4

# Or incremental: re-embed changed units only
llmwiki index reembed --repo .llmwiki --since 2024-01-01
```

Migration runs as Kubernetes Job or standalone binary.

7.8.3 Graph Migration

Schema changes (new node types, edge types, property changes):

```
llmwiki graph migrate --repo .llmwiki --to-version 2
# Dry-run first:
llmwiki graph migrate --repo .llmwiki --to-version 2 --dry-run
```

7.9 Security Hardening

- **Non-root**: Container runs as UID 1000
- **Read-only rootfs**: `readOnlyRootFilesystem: true`
- **No capabilities**: `drop: [ALL]`
- **Network policies**: Ingress only from API gateway; egress only to model registry (if remote)
- **Secrets**: API keys, OIDC secrets via Kubernetes Secrets / Vault
- **Auth**: OIDC, API keys, mTLS for service-to-service
- **Rate limiting**: Per-user, per-IP, per-API-key
- **Audit logging**: All queries logged with user, query hash, result count

Table 7.2: Tuning Knobs by Workload

Workload	Parameter	Recommendation
High ingestion	<code>ingestion.batch_size</code>	128-512
High ingestion	<code>index.hnsw.ef_construction</code>	400 (build quality)
High query	<code>index.hnsw.ef_search</code>	128-256 (accuracy)
High query	<code>evidence_gate.batch_size</code>	32-64
Low latency	<code>planner.max_depth</code>	3
Low latency	<code>retrieval.max_candidates</code>	20
High recall	<code>retrieval.hybrid_weights</code>	Increase vector weight
Memory constrained	<code>index.hnsw.m</code>	8-12
GPU	<code>embedder.device</code>	"cuda"

7.10 Performance Tuning

7.11 Patent-Relevant Novelty

Patent Claim 7.1. Claim 10: A deployment architecture for a semantic operating layer wherein: (a) the entire system state is contained in a single filesystem directory enabling backup via standard file copy, (b) the system runs as a single stateless binary with embedded models requiring zero external services, (c) air-gapped operation is supported via a self-contained bundle including all models and grammars, and (d) deterministic replay is achieved by directory synchronization without auxiliary coordination services.

7.12 Summary

LLMWiki deployment prioritizes:

1. **Operational simplicity:** Single binary, filesystem state, no external deps
2. **Local-first:** Works fully offline, air-gapped
3. **Production readiness:** K8s native, observability, HA, security
4. **Recoverability:** WAL replay, snapshot restore, verified backups
5. **Upgradability:** Blue-green, rolling, migration tooling

The next volume (Vol. 08) specifies the API contracts.

Part II

Reference and Extension

API Reference

This volume specifies the public API contracts: REST endpoints, gRPC services, WebSocket protocol, authentication, rate limiting, and SDKs. All APIs are versioned; breaking changes only in major versions.

8.1 API Overview

- **Base URL:** `http://<host>:8080/api/v1`
- **Protocol:** HTTP/1.1, h2 (gRPC), WebSocket
- **Format:** JSON (request/response), SSE (streaming)
- **Auth:** Bearer token (OIDC), API Key, or none (dev)
- **Versioning:** URL path `/v1/`, header `Accept: application/vnd.llmwiki.v1+json`

8.2 Authentication

8.2.1 Bearer Token (OIDC)

Authorization: Bearer <access_token>

Token validated against configured OIDC issuer. Claims mapped to `UserContext`:

```
{
  "sub": "user_123",
  "email": "user@example.com",
  "roles": ["admin", "analyst"],
  "org_id": "org_456",
  "permissions": ["query", "ingest", "admin"]
}
```

8.2.2 API Key

Authorization: ApiKey <key_id>.<secret>

or header

X-API-Key: <key_id>.<secret>

API keys managed via `/admin/api-keys` (admin only). Scoped permissions.

8.2.3 Anonymous (Development)

auth: "none" in config disables auth. Only for local development.

8.3 Core Endpoints

8.3.1 Health

GET /health/live
GET /health/ready
GET /health/deep

Response (ready):

```
{
  "status": "ready",
  "checks": {
    "graph": "ok",
    "index": "ok",
    "evidence_gate": "ok",
    "wal": "ok"
  },
  "version": "0.1.0",
  "repo_id": "prod-main"
}
```

8.3.2 Query

Execute Query (Non-streaming)

POST /query
Content-Type: application/json

```
{
  "query": "How does the payment service handle retries?",
  "context_budget": 8000,
  "evidence_threshold": 0.72,
  "max_steps": 5,
  "stream": false,
  "plan_options": {
    "max_depth": 3,
    "include_graph": true
  }
}
```

Response:

```
{
  "query_id": "q_abc123",
  "answer": "The payment service uses exponential backoff...",
  "citations": [
    {
      "unit_id": "sha256:a1b2c3...",

```

```
    "relevance": 0.94,  
    "span": {"start_line": 45, "end_line": 67},  
    "preview": "fn handle_retry(...) { ... }"  
  }  
],  
  "evidence_count": 12,  
  "rejected_count": 35,  
  "plan": {...},  
  "usage": {  
    "tokens": 2450,  
    "latency_ms": 312,  
    "llm_calls": 2  
  }  
}
```

Execute Query (Streaming)

POST /query/stream
Content-Type: application/json
Accept: text/event-stream

```
{ "query": "...", "stream": true, ... }
```

SSE Events:

event: plan_start

data: {"plan_id": "p_123", "steps": 5}

event: step_start

data: {"step_id": "s_1", "kind": "Search", "tool": "hybrid_search"}

event: evidence

data: {"unit_id": "sha256:...", "relevance": 0.94, "preview": "..."}

event: token

data: {"token": "The", "step_id": "s_3"}

event: token

data: {"token": " payment", "step_id": "s_3"}

event: step_complete

data: {"step_id": "s_1", "output": {"results": [...]}}

event: plan_complete

data: {"answer": "...", "citations": [...], "usage": {...}}

8.3.3 Ingestion

Ingest Path

POST /ingest
Content-Type: application/json

```
{
  "path": "/data/source-code",
  "mode": "incremental", # or "full"
  "parsers": ["python", "markdown", "yaml"],
  "options": {
    "chunk_size": 512,
    "overlap": 50,
    "embed": true
  }
}
```

Response:

```
{
  "job_id": "job_xyz789",
  "status": "accepted",
  "estimated_units": 5000
}
```

Ingest Status

GET /ingest/{job_id}

Response:

```
{
  "job_id": "job_xyz789",
  "status": "running", # pending|running|completed|failed
  "progress": 0.65,
  "units_processed": 3250,
  "units_total": 5000,
  "added": 3200,
  "modified": 45,
  "removed": 5,
  "errors": 0,
  "started_at": "2024-01-15T10:30:00Z",
  "updated_at": "2024-01-15T10:31:45Z"
}
```

Ingest History

GET /ingest/history?limit=20

8.3.4 Repository Management

GET /repo	# Current repo info
POST /repo/init	# Initialize new repo
POST /repo/clone	# Clone from git/remote
GET /repo/stats	# Unit/graph/index counts
POST /repo/verify	# Integrity check
POST /repo/compact	# Reclaim space, rebuild indices
POST /repo/backup	# Trigger backup
GET /repo/wal	# WAL entries (paginated)

8.3.5 Graph Operations

Traverse

POST /graph/traverse

```
{
  "start": "sha256:...",
  "pattern": {
    "edge_types": ["CALLS", "REFERENCES"],
    "node_types": ["Function", "Class"],
    "direction": "out",
    "max_hops": 3,
    "limit": 100
  }
}
```

Response:

```
{
  "nodes": [...],
  "edges": [...],
  "subgraph_id": "sg_123"
}
```

Entity Lookup

GET /graph/entity/{entity_id}

GET /graph/entity/by-name?name=process_order&type=Function

Neighborhood

GET /graph/neighborhood?ids=sha256:a,sha256:b&hops=2

8.3.6 Index Operations

POST /index/rebuild # Full rebuild

POST /index/reembed # Re-embed (model change)

GET /index/stats # Vector count, dimension, memory

GET /index/health # HNSW/BM25 status

8.3.7 Unit Access

GET /units/{unit_id} # Full unit

GET /units/{unit_id}/raw # Raw content

GET /units/batch?ids=... # Multiple units

GET /units/search?q=... # Search units (alias for /query)

8.3.8 Evidence Gate

POST /evidence/verify

```
{
  "query": "How does X work?",
  "candidates": ["sha256:a", "sha256:b", ...],
}
```

```
"threshold": 0.72
}
```

Response:

```
{
  "verified": [
    {"unit_id": "sha256:a", "score": 0.94}
  ],
  "rejected": [
    {"unit_id": "sha256:b", "score": 0.45}
  ],
  "calibration": "isotonic"
}
```

8.3.9 Administration

```
GET    /admin/config           # Current config (sanitized)
POST   /admin/config/reload # Reload from file
GET    /admin/metrics       # Prometheus metrics
GET    /admin/pprof/...     # Go-style pprof (if enabled)

# API Key management
POST   /admin/api-keys      # Create
GET    /admin/api-keys      # List
DELETE /admin/api-keys/{key_id} # Revoke

# Parser management
GET    /admin/parsers       # List loaded parsers
POST   /admin/parsers/reload # Reload parser configs
```

8.4 WebSocket API

For real-time, bidirectional communication (CLI, IDE extensions).

WS /ws/v1

```
# Client -> Server
{"type": "query", "payload": {"query": "...", "stream": true}}
{"type": "ingest", "payload": {"path": "..."}}
{"type": "subscribe", "payload": {"events": ["ingest.progress",
↪  "query.token"]}}

# Server -> Client
{"type": "event", "event": "query.token", "data": {"token": "..."}}
{"type": "event", "event": "ingest.progress", "data": {"progress": 0.5}}
{"type": "result", "request_id": "...", "data": {...}}
{"type": "error", "request_id": "...", "code": "...", "message": "..."}

```

8.5 gRPC API

Defined in `proto/llmwiki/v1/api.proto`. Mirrors REST with strongly-typed messages.

```
service LLMWiki {
  rpc Query(QueryRequest) returns (QueryResponse);
  rpc QueryStream(QueryRequest) returns (stream QueryStreamEvent);
  rpc Ingest(IngestRequest) returns (IngestResponse);
  rpc IngestProgress(IngestProgressRequest) returns (stream
    ↪ IngestProgress);
  rpc GraphTraverse(GraphTraverseRequest) returns (GraphTraverseResponse);
  rpc UnitGet(UnitGetRequest) returns (Unit);
  rpc HealthCheck(HealthRequest) returns (HealthResponse);
}
```

gRPC gateway (transcoding) exposes same REST endpoints automatically.

8.6 Error Handling

```
# Standard error response
{
  "error": {
    "code": "EVIDENCE_INSUFFICIENT",
    "message": "No evidence units passed verification threshold",
    "details": {
      "query_id": "q_abc",
      "candidates": 42,
      "verified": 0,
      "threshold": 0.72
    }
  },
  "request_id": "req_xyz",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

8.7 Rate Limiting

Headers on all responses:
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1705312800

On 429:
Retry-After: 60

Limits configurable per auth type (API key, user, IP, global).

Table 8.1: Error Codes

Code	Meaning
INVALID_REQUEST	Request validation failed
UNAUTHENTICATED	Missing/invalid auth
FORBIDDEN	Insufficient permissions
NOT_FOUND	Resource not found
CONFLICT	Resource conflict (e.g., repo exists)
EVIDENCE_INSUFFICIENT	Gate rejected all candidates
BUDGET_EXCEEDED	Token/time/cost budget exceeded
INTERNAL_ERROR	Unexpected server error
SERVICE_UNAVAILABLE	Dependency down (embedder, LLM)
RATE_LIMITED	Too many requests

8.8 SDKs

Table 8.2: Official SDKs

Language	Package	Status
Python	<code>pip install llmwiki</code>	v0.1 (alpha)
TypeScript/JS	<code>npm install @llmwiki/sdk</code>	v0.1 (alpha)
Rust	<code>cargo add llmwiki-client</code>	v0.1 (alpha)
Go	<code>go get github.com/llmwiki/go-sdk</code>	Planned

Python SDK Example

```

from llmwiki import LLMWikiClient

client = LLMWikiClient(
    base_url="http://localhost:8080",
    api_key="key_id.secret"
)

# Simple query
result = client.query("How does retry logic work?")
print(result.answer)
for cite in result.citations:
    print(f" [{cite.unit_id[:8]}] {cite.preview}")

# Streaming
for event in client.query_stream("Explain the auth flow"):
    if event.type == "token":
        print(event.token, end="", flush=True)
    elif event.type == "plan_complete":
        print("\n---")
        print(f"Evidence: {event.data.evidence_count}")

```

8.9 OpenAPI Specification

Full OpenAPI 3.1 spec at `/openapi.json` and `/openapi.yaml`. Generated from source-of-truth `api/src/routes.rs` (Axum) or `api/src/main.rs` (Actix).

8.10 API Versioning Policy

- **v1**: Current stable. No breaking changes.
- **Deprecation**: 6 months notice via **Sunset** header and changelog.
- **New versions**: `/v2/` path; both served simultaneously during transition.
- **Experimental**: `/v1/experimental/` prefix; no stability guarantee.

8.11 Patent-Relevant Novelty

Patent Claim 8.1. Claim 11: An API for a semantic operating layer comprising: (a) a query endpoint that returns answers with mandatory evidence citations verified by a cross-encoder gate, (b) an ingestion endpoint that accepts filesystem paths and returns content-addressed unit diffs, (c) a graph traversal endpoint exposing the knowledge graph as a queryable API, (d) a streaming protocol that emits fine-grained events (plan, evidence, token, citation) for real-time UI, and (e) an evidence verification endpoint that exposes the gate as a first-class callable service.

8.12 Summary

The LLMWiki API exposes the full power of the semantic operating layer through:

1. **REST**: Simple, cacheable, widely compatible
2. **gRPC**: High-performance, typed, streaming-native
3. **WebSocket**: Bidirectional, real-time, IDE-friendly
4. **SDKs**: Idiomatic clients for major languages

All endpoints respect the architectural invariants: evidence closure, content addressing, deterministic replay.

The next volume (Vol. 09) guides developers extending LLMWiki.

Developer Guide

This volume is for developers extending LLMWiki: adding parsers, chunkers, embedders, graph backends, index backends, evidence verifiers, planner strategies, tools, and auth providers.

9.1 Development Environment

9.1.1 Prerequisites

```
# Rust (for core)
rustup install stable
rustup component add rustfmt clippy

# Python (for ML, parsers, scripts)
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

# Node.js (for Web UI, if applicable)
corepack enable
pnpm install

# System deps
# Ubuntu: sudo apt install libssl-dev libclang-dev pkg-config
# macOS: brew install llvm pkg-config
```

9.1.2 Repository Structure

```
llmwiki/
-- crates/
  -- llmwiki-core/          # Core types, traits, errors
  -- llmwiki-ingestion/     # Parsers, chunkers, embedders
  -- llmwiki-graph/        # Graph store, entity resolution
  -- llmwiki-index/        # HNSW, BM25, hybrid search
  -- llmwiki-evidence/     # Evidence gate, cross-encoder
  -- llmwiki-planner/      # Query planner, optimizer
  -- llmwiki-runtime/      # Agent runtime, tools, LLM client
  -- llmwiki-api/          # REST/gRPC/WS handlers
  -- llmwiki-cli/          # CLI commands
-- python/
```

```
-- llmwiki/           # Python bindings (pyo3)
-- scripts/           # Evaluation, benchmarking
-- web/               # React/Vite admin UI
-- docker/
-- helm/
-- eval/              # Evaluation datasets, configs
-- docs/              # This documentation
-- Cargo.toml         # Workspace root
```

9.1.3 Build and Test

```
# Build all (release)
cargo build --release --workspace

# Run unit tests
cargo test --workspace

# Run integration tests (requires Docker)
cargo test --workspace --test integration

# Run specific crate tests
cargo test -p llmwiki-ingestion

# Build Python wheel
maturin develop --release -m python/Cargo.toml

# Lint
cargo fmt --all --check
cargo clippy --workspace -- -D warnings
```

9.2 Extension Points Deep Dive

9.2.1 Custom Parser

1. Create crate `llmwiki-parser-myformat`
2. Implement `Parser` trait:

```
// src/lib.rs
use llmwiki_core::{Parser, Artifact, SemanticUnit, ParserSchema, MimeType,
    ↪ Result};
use async_trait::async_trait;

pub struct MyFormatParser;

#[async_trait]
impl Parser for MyFormatParser {
    async fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>
    ↪ {
        // 1. Parse artifact.bytes into AST
        // 2. Extract semantic units from AST
        // 3. For each unit: compute content hash, extract metadata
```

```

        // 4. Return units
    }

    fn supported_mimes(&self) -> Vec<MimeType> {
        vec!["application/x-myformat".parse().unwrap()]
    }

    fn schema(&self) -> ParserSchema {
        ParserSchema {
            unit_types: vec!["MyUnit".into()],
            metadata_fields: vec!["custom_field".into()],
        }
    }
}

// Register in config
// config.toml:
// [ingestion.parsers.myformat]
// type = "dynamic"
// lib = "llmwiki_parser_myformat"
// entry = "MyFormatParser::new"

```

3. Add to Cargo.toml:

```

[package]
name = "llmwiki-parser-myformat"
cdylib = true # For dynamic loading

[dependencies]
llmwiki-core = { path = "../..crates/llmwiki-core" }
llmwiki-ingestion = { path = "../..crates/llmwiki-ingestion" }
tree-sitter = "0.22" # If using tree-sitter

```

Tree-sitter Grammar Integration. If your format has a tree-sitter grammar:

```

// build.rs
fn main() {
    tree_sitter_grammar::build("myformat", &["src/grammar.json"]);
}

```

Then use tree-sitter in parser:

```

let mut parser = tree_sitter::Parser::new();
parser.set_language(&tree_sitter_myformat::language())?;
let tree = parser.parse(&source, None)?;

```

9.2.2 Custom Chunker

```

use llmwiki_core::{Chunker, SemanticUnit, Chunk, ChunkConfig, Result};

pub struct MyChunker;

```

```
impl Chunker for MyChunker {
    fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) -> Vec<Chunk>
    ↪ {
        match config.strategy {
            ChunkStrategy::Hierarchical => self.hierarchical_chunk(unit,
                ↪ config),
            ChunkStrategy::Sliding => self.sliding_chunk(unit, config),
            ChunkStrategy::Semantic => self.semantic_chunk(unit, config),
            _ => vec![Chunk::from_unit(unit)],
        }
    }

    fn config_schema(&self) -> ChunkConfigSchema {
        ChunkConfigSchema {
            fields: vec![
                ConfigField::Int("max_tokens", "Max tokens per chunk"),
                ConfigField::Int("overlap_tokens", "Overlap tokens"),
                ConfigField::Bool("preserve_headers", "Keep markdown
                    ↪ headers"),
            ],
        }
    }
}
```

9.2.3 Custom Embedder

```
use llmwiki_core::{Embedder, ModelId, Result};
use ndarray::Array2;

pub struct MyEmbedder {
    session: ort::Session, // ONNX Runtime
    dim: usize,
}

impl MyEmbedder {
    pub fn new(model_path: &str) -> Result<Self> {
        let session =
            ↪ ort::Session::builder()?.commit_from_file(model_path)?;
        let dim = session.outputs[0].dimensions()[1] as usize;
        Ok(Self { session, dim })
    }
}

#[async_trait]
impl Embedder for MyEmbedder {
    async fn embed_batch(&self, texts: &[String]) -> Result<Vec<Vec<f32>>>
    ↪ {
        // Tokenize, pad, run inference, normalize
        let inputs = self.tokenize(texts)?;
        let outputs = self.session.run(inputs)?;
    }
}
```

```

        let embeddings = self.postprocess(outputs)?;
        Ok(embeddings)
    }

    fn dimension(&self) -> usize { self.dim }
    fn model_id(&self) -> ModelId { ModelId::Custom("my-embedder".into())
        ↪ }
}

```

9.2.4 Custom Graph Store

Implement `GraphStore` trait (Vol. 01, §1.24). Key methods:

- `upsert_nodes`: Batch insert/update nodes
- `upsert_edges`: Batch insert/update edges
- `traverse`: Pattern-based traversal
- `neighborhoods`: k -hop neighborhoods

See `crates/llmwiki-graph/src/store/rocksdb.rs` for reference implementation.

9.2.5 Custom Index Backend

Implement `SemanticIndex` trait (Vol. 01, §1.25):

- `upsert`: Add/update vectors + metadata
- `search_vector`: ANN search
- `search_bm25`: Keyword search
- `search_hybrid`: Fused search

See `crates/llmwiki-index/src/backend/hnsw_tantivy.rs`.

9.2.6 Custom Evidence Verifier

Implement `EvidenceGate` trait (Vol. 01, §1.26):

```

pub struct MyEvidenceGate {
    model: CrossEncoder,
    calibrator: Calibrator,
}

impl EvidenceGate for MyEvidenceGate {
    fn verify(&self, query: &str, candidates: Vec<ScoredUnit>, threshold:
        ↪ f32)
        -> Result<Vec<VerifiedUnit>> {
        // Score each (query, unit) pair
        // Apply calibration
        // Filter by threshold
    }
}

```

```
fn calibrate(&self, dataset: CalibrationSet) ->
  ↪ Result<CalibrationCurve> {
    // Fit isotonic/Platt/temperature
  }
}
```

9.2.7 Custom Query Planner Strategy

Implement `QueryPlanner` trait (Vol. 01, §1.27):

```
pub struct MyPlanner {
  cost_model: CostModel,
  llm: LLMClient,
}

impl QueryPlanner for MyPlanner {
  fn plan(&self, query: Query, context: PlanContext) ->
    ↪ Result<QueryPlan> {
    // 1. Analyze query intent
    // 2. Decompose into subqueries
    // 3. Choose retrieval strategies per subquery
    // 4. Build DAG with dependencies
    // 5. Optimize order (cost model)
    // 6. Return plan
  }

  fn optimize(&self, plan: QueryPlan, stats: IndexStats) ->
    ↪ Result<QueryPlan> {
    // Reorder, merge, prune based on stats
  }
}
```

9.2.8 Custom Agent Tool

```
use llmwiki_core::{Tool, ToolInput, ToolOutput, ToolSchema, Result};
use async_trait::async_trait;
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct MyToolInput { query: String, limit: usize }

#[derive(Serialize)]
struct MyToolOutput { results: Vec<String> }

pub struct MyTool;

#[async_trait]
impl Tool for MyTool {
  fn name(&self) -> &str { "my_tool" }
  fn description(&self) -> &str { "Custom tool description" }
```

```

fn schema(&self) -> ToolSchema {
    ↪ ToolSchema::from_type::<MyToolInput>() }
fn output_schema(&self) -> ToolSchema {
    ↪ ToolSchema::from_type::<MyToolOutput>() }

async fn execute(&self, input: ToolInput) -> Result<ToolOutput> {
    let params: MyToolInput = serde_json::from_value(input.0)?;
    // Execute logic
    let output = MyToolOutput { results: vec![] };
    Ok(ToolOutput(serde_json::to_value(output)?))
}
}

```

Register in config:

```

[tools.custom.my_tool]
module = "my_tools"
class = "MyTool"

```

9.2.9 Custom Auth Provider

```

use llmwiki_core::{AuthProvider, AuthContext, UserInfo, Result};
use async_trait::async_trait;

pub struct MyAuthProvider;

#[async_trait]
impl AuthProvider for MyAuthProvider {
    async fn authenticate(&self, credentials: AuthCredentials)
        -> Result<AuthContext> {
        // Validate token, API key, etc.
        // Return context with user_id, roles, permissions
    }

    async fn authorize(&self, ctx: &AuthContext, resource: &str, action:
        ↪ &str)
        -> Result<bool> {
        // Check permissions
    }

    fn extract_credentials(&self, request: &HttpRequest) ->
        ↪ Option<AuthCredentials> {
        // Extract from header, cookie, query param
    }
}

```

9.3 Testing Extensions

9.3.1 Parser Tests

```
// tests/parser_test.rs
```

```
use llmwiki_ingestion::Parser;

#[test]
fn parse_myformat() {
    let parser = MyFormatParser;
    let artifact = Artifact::from_bytes(b"...", "test.myformat");
    let units = parser.parse(artifact).await.unwrap();

    assert_eq!(units.len(), 3);
    assert_eq!(units[0].unit_type, "MyUnit");
    assert!(units[0].id.starts_with("sha256:"));
}
```

9.3.2 Integration Tests

```
// tests/integration_test.rs
use llmwiki_core::*;

#[tokio::test]
async fn full_ingestion_query_cycle() {
    let repo = temp_repo();
    llmwiki_init(&repo).await;

    // Ingest
    llmwiki_ingest(&repo, "testdata/sample.py").await.unwrap();

    // Query
    let result = llmwiki_query(&repo, "What does sample.py
    ↪ do?").await.unwrap();

    assert!(!result.answer.is_empty());
    assert!(result.citations.len() > 0);
    assert!(result.citations.iter().all(|c| c.relevance > 0.7));
}
```

9.3.3 Property-Based Testing

```
use proptest::prelude::*;

proptest! {
    #[test]
    fn canonicalization_deterministic(content in "\\PC*") {
        let unit1 = SemanticUnit::new("Test", content.clone());
        unit1.content = content.clone();
        let id1 = unit1.compute_id();

        let unit2 = SemanticUnit::new("Test()", content);
        let id2 = unit2.compute_id();

        prop_assert_eq!(id1, id2);
    }
}
```

```
    }  
}
```

9.4 Debugging and Profiling

9.4.1 Logging

```
# Debug parser  
RUST_LOG=llmwiki_ingestion=debug llmwiki ingest path/  
  
# Debug query pipeline  
RUST_LOG=llmwiki_runtime=debug,llmwiki_planner=trace llmwiki query "test"  
  
# Structured JSON logs  
RUST_LOG=info llmwiki server --log-format json
```

9.4.2 Profiling

```
# CPU profiling (perf)  
perf record --call-graph dwarf llmwiki ingest large-codebase/  
perf report  
  
# Memory profiling  
cargo install flamegraph  
cargo flamegraph --bin llmwiki -- ingest large-codebase/  
  
# Heap profiling (jemalloc)  
MALLOC_CONF=prof:true,prof_prefix:jeprof ./llmwiki ingest ...  
jeprof --pdf ./llmwiki jeprof.*.heap > heap.pdf
```

9.5 Contributing

9.5.1 Code Style

- Rust: `rustfmt` (default), `clippy` (deny warnings)
- Python: `ruff` (format + lint), `mypy` (strict)
- Commits: Conventional Commits (`feat:`, `fix:`, `docs:`, etc.)

9.5.2 PR Process

1. Fork, create branch `feat/my-feature`
2. Write tests for new functionality
3. Ensure all tests pass: `cargo test --workspace`
4. Run evaluation: `llmwiki eval run --config eval/ci.yaml`
5. Open PR with description linking to issue

6. CI must pass (tests, lints, eval regression check)
7. Maintainer review -> merge

9.5.3 Adding a New Volume

1. Create `docs/Volume_XX_Name/Volume_XX_Name.tex`
2. Add `\include{docs/Volume_XX_Name/Volume_XX_Name}` to `LLMWiki.tex`
3. Update `volume-labels.tex` if needed
4. Run `make pdf` to verify compilation

9.6 Release Process

1. Update `Cargo.toml` version
2. Update `CHANGELOG.md`
3. Tag: `git tag v0.X.Y`
4. GitHub Actions builds and publishes:
5. Docker images to GHCR
6. Binaries to GitHub Releases
7. Python wheels to PyPI
8. Docs to GitHub Pages

9.7 Summary

LLMWiki is designed for extensibility at every layer. The trait-based plugin system allows:

1. **Parsers:** New languages/formats without core changes
2. **Chunkers:** Domain-specific segmentation strategies
3. **Embedders:** Any ONNX-compatible or custom model
4. **Graph stores:** RocksDB, SQLite, Kuzu, Neo4j, etc.
5. **Indices:** HNSW, FAISS, DiskANN, custom
6. **Evidence gates:** Cross-encoder, LLM-judge, heuristic
7. **Planners:** Rule-based, LLM-based, hybrid
8. **Tools:** Arbitrary computation exposed to planner
9. **Auth:** OIDC, API keys, mTLS, custom

All extensions are dynamically loaded, versioned, and testable in isolation.

Part III

Intellectual Property and Research

Patent Draft

This volume presents the patent claims for the novel aspects of LLMWiki. Claims are written in USPTO format with supporting specification references. Each claim corresponds to an architectural invariant or novel combination identified in Volumes 01-09.

10.1 Field of the Invention

The present invention relates to systems and methods for transforming heterogeneous digital artifacts into an evidence-aware semantic knowledge space, specifically a local-first semantic operating layer that enforces evidence closure through mandatory cross-encoder verification.

10.2 Background

Prior art includes Retrieval-Augmented Generation (RAG) systems (Lewis et al., 2020), GraphRAG (Edge et al., 2024), HippoRAG (Zhang et al., 2024), LightRAG (Zeng et al., 2024), and RAPTOR (Sarathi et al., 2024). These systems share common limitations: (1) vector indices as primary knowledge representation, (2) optional or post-hoc evidence verification, (3) dependence on external LLM APIs for critical-path operations, (4) lack of deterministic replay, and (5) no content-addressed unit identity.

10.3 Summary of the Invention

The invention provides a semantic operating layer comprising: a content-addressed filesystem-backed write-ahead log; a property knowledge graph as canonical representation; a hybrid semantic index fusing vector, keyword, and graph signals; an evidence gate enforcing cross-encoder verification; and a DAG-based query planner with cost-based optimization.

10.4 Independent Claims

Claim 10.1 (System Architecture). *A semantic operating layer system comprising:*

1. *a filesystem-backed write-ahead log storing content-addressed semantic units, each unit identified by a SHA-256 hash of its canonical representation;*
2. *a property knowledge graph wherein nodes represent semantic units and entities, and edges represent typed relationships, said graph persisted in a column-family store with prefix-encoded keys enabling type-partitioned scans;*

3. *a hybrid semantic index combining a hierarchical navigable small world (HNSW) vector index, a BM25 inverted index, and a graph adjacency index, all keyed by the content-addressed unit identifiers;*
4. *an evidence gate configured to receive a query and a candidate set of units from the hybrid index, and to verify each candidate by computing a cross-encoder relevance score between the query and the unit content, retaining only candidates whose score exceeds a calibrated threshold; and*
5. *a query planner that decomposes an input query into a directed acyclic graph of subqueries, each subquery assigned a retrieval strategy, and executes said subqueries in topological order with cost-based optimization.*

Claim 10.2 (Method of Deterministic Ingestion and Replay). *A computer-implemented method for deterministic knowledge ingestion and replay, comprising:*

1. *parsing a source artifact into a hierarchical tree of semantic units using a language-aware parser;*
2. *computing a content address for each unit as $SHA-256(\text{canonical}(\text{content}, \text{type}, \text{span}, \text{metadata}, \text{children}))$;*
3. *extracting entity mentions and relations from units via rule-based patterns or a local language model;*
4. *resolving entities against the knowledge graph using a weighted similarity function combining fuzzy name match, type compatibility, context overlap, and embedding cosine similarity;*
5. *writing a diff of (added, modified, removed) unit identifiers to the filesystem-backed write-ahead log as an atomic append-only JSONL record;*
6. *applying the diff to the knowledge graph and hybrid index within a single atomic transaction;*
7. *wherein replay is achieved by sequentially applying logged diffs to an empty store, producing bitwise-identical state.*

Claim 10.3 (Evidence Gate with Calibrated Threshold). *An evidence verification apparatus comprising:*

1. *a cross-encoder model that takes a (query, unit) pair and outputs a relevance score in $[0, 1]$;*
2. *a calibrator trained on a validation set of (query, unit, label) triples, mapping raw scores to calibrated probabilities via isotonic regression or temperature scaling;*
3. *a threshold selector that chooses an operating point τ such that false discovery rate $\leq \alpha$ on held-out data;*
4. *a gate function $\mathcal{G}(q, \mathcal{R}, \tau) = \{u \in \mathcal{R} \mid \text{calibrated_score}(q, u) \geq \tau\}$;*
5. *wherein the gate is a mandatory pipeline stage: no unit reaches the synthesis stage without passing the gate; and*

6. *wherein the gate may return an empty set, triggering an "insufficient evidence" response rather than hallucinated synthesis.*

Claim 10.4 (Deployment Architecture). *A deployment method for a semantic operating layer, comprising:*

1. *packaging the entire system—parsers, embedders, cross-encoder, graph store, indices, planner, runtime—as a single statically-linked binary with embedded ONNX models and tree-sitter grammars;*
2. *storing all mutable state in a single directory `.llmwiki/` containing subdirectories for units, graph partitions, index files, and write-ahead log;*
3. *enabling air-gapped operation by eliminating all external network dependencies in the critical path (ingestion, indexing, retrieval, verification);*
4. *supporting horizontal scaling via directory synchronization (`rsync`, `git`, or object store replication) without consensus protocols;*
5. *providing deterministic replay by copying `.llmwiki/` to a fresh machine and restarting the binary.*

Claim 10.5 (Label-Free Significance Calibration). *A method for establishing a verification threshold in an evidence-gated retrieval system without relevance annotations, comprising:*

1. *constructing a plurality of query–unit pairs for which genuine relevance is excluded by construction, said exclusion effected by at least one of: drawing the unit from a corpus declared disjoint in domain from that of the query; drawing the unit from a write-ahead-log snapshot predating creation of every entity referenced by the query; or drawing the unit from a graph component containing no entity mentioned in the query;*
2. *evaluating the verification scorer over said pairs to accumulate an empirical null distribution of the scoring statistic;*
3. *selecting as threshold the least score whose exceedance probability under said distribution, multiplied by the mean candidate count per query, attains a target false alarm rate; and*
4. *emitting, with each admitted unit, said false alarm rate as the unit’s significance, in place of a raw scorer output;*

wherein recalibration is triggered by advancement of the write-ahead log and requires no human annotation.

Claim 10.6 (Cascaded Proofreading Verification). *An evidence verification apparatus comprising a cascade of $n \geq 2$ verification stages, wherein:*

1. *each stage evaluates a candidate against an evidence channel distinct from those of the other stages, said channels comprising at least lexical overlap, learned cross-encoder relevance, knowledge-graph reachability among co-surviving candidates, and inferential entailment;*
2. *each stage irreversibly discards candidates scoring below a stage-specific threshold, discarded candidates being unrecoverable by subsequent stages;*

3. the stages are ordered by non-decreasing unit evaluation cost; and
4. the false-acceptance rate of the cascade is the product of the per-stage false-acceptance rates, while the expected evaluation cost per candidate is the survival-weighted sum of per-stage costs.

Claim 10.7 (Topological Consistency Gate). *A method for certifying mutual consistency of a verified evidence set prior to answer synthesis, comprising:*

1. constructing a graph whose vertices are the verified units and whose edges join units sharing an entity or standing in a knowledge-graph relation;
2. assigning to each vertex a stalk of admissible truth assignments over atomic claims extracted from the corresponding unit, to each edge a stalk over the claims shared across that edge, and to each incidence a restriction map induced by inferential entailment between said claim sets, thereby forming a cellular sheaf;
3. computing the zeroth and first cohomology of said sheaf;
4. admitting the evidence set for synthesis when a global section supporting the answer exists; and
5. when the first cohomology is nonzero, withholding synthesis and emitting the support of the obstruction class as a machine-readable conflict witness identifying the relations carrying the inconsistency;

whereby cyclic inconsistencies undetectable by pairwise contradiction testing are detected and localised.

10.5 Dependent Claims

Claim 10.8 (Parser Extensibility). *The system of Claim 1, further comprising a parser registry that dynamically loads parser implementations conforming to a **Parser** trait, each parser declaring its supported MIME types and output unit schema, whereby new languages and formats are added without modifying the core ingestion pipeline.*

Claim 10.9 (Tree-sitter Integration). *The system of Claim 6, wherein at least one parser uses a tree-sitter grammar to produce a concrete syntax tree, and semantic units are extracted via S-expression query patterns (SCM) mapped to unit fields, enabling incremental re-parsing with $O(\log n)$ edit complexity.*

Claim 10.10 (Hierarchical Chunking). *The system of Claim 1, further comprising a chunker that recursively partitions semantic units along structural boundaries (functions, classes, sections) while respecting a maximum token budget and configurable overlap, producing chunks that preserve hierarchical context via parent unit references.*

Claim 10.11 (Local Embedding Inference). *The system of Claim 1, wherein the embedder executes entirely on local hardware (CPU, GPU, or NPU) using ONNX Runtime with model quantization (INT8, INT4), supporting batch inference with dynamic padding, and requiring zero external API calls.*

Claim 10.12 (Entity Resolution). *The system of Claim 1, wherein entity resolution merges mentions into canonical entities using a scoring function $S(e_1, e_2) = \alpha \cdot \text{Fuzzy}(n_1, n_2) + \beta \cdot \mathbb{1}[t_1 = t_2] + \gamma \cdot \text{Jaccard}(ctx_1, ctx_2) + \delta \cdot \cos(emb_1, emb_2)$, with $\alpha + \beta + \gamma + \delta = 1$, and creates a new entity iff $\max S < \tau_{er}$.*

Claim 10.13 (Graph Adjacency Index). *The system of Claim 1, wherein the graph adjacency index is a separate column family storing for each unit a serialized list of (relation_type, target_unit_id, weight) tuples, enabling sub-millisecond k-hop expansion during hybrid retrieval without accessing the primary graph store.*

Claim 10.14 (Hybrid Retrieval with Reciprocal Rank Fusion). *The system of Claim 1, wherein hybrid retrieval combines vector, keyword, and graph scores via reciprocal rank fusion: $\text{score}(u) = \sum_{s \in \{v, k, g\}} w_s \cdot \frac{1}{k + \text{rank}_s(u)}$, where w_s are configurable weights and $\text{rank}_s(u)$ is the position of u in the top- K results of signal s .*

Claim 10.15 (DAG Query Planner). *The system of Claim 1, wherein the query planner produces a plan represented as a directed acyclic graph $P = (V, E)$ where each vertex $v \in V$ is a retrieval, synthesis, or tool invocation step with declared inputs and outputs, and edges E represent data dependencies, and the executor schedules steps in topological order with parallel fan-out where dependencies permit.*

Claim 10.16 (Budgeted Execution). *The system of Claim 10, further comprising a budget tracker that enforces hard limits on cumulative tokens, wall-clock time, tool invocations, and monetary cost, with configurable overflow behavior: truncate, return partial, or error.*

Claim 10.17 (Streaming Protocol). *The system of Claim 1, further comprising a Server-Sent Events (SSE) endpoint that emits fine-grained events: plan_start, step_start, evidence_found, token, step_complete, plan_complete, budget_warning, enabling real-time UI rendering of the query execution pipeline.*

Claim 10.18 (Evaluation Framework). *A computer-implemented evaluation method for a semantic operating layer, comprising:*

1. *datasets with human-annotated evidence spans for each query;*
2. *metrics: Evidence Recall@k, Evidence Precision@k, Evidence F1@k, Gate Rejection Rate, Attribution Score (claim-level NLI verification against citations), and Citation Quality (mean cross-encoder score of cited units);*
3. *automated configuration sweeps over retrieval k, hybrid weights, gate threshold τ , embedder model, and LLM model;*
4. *statistical rigor: bootstrap confidence intervals, paired Wilcoxon tests, Benjamini-Hochberg correction, effect size reporting;*
5. *regression gates in continuous integration blocking merges on $> 2\%$ metric degradation.*

10.6 Design Decisions Supporting Novelty

10.7 Specification Support

Each claim is supported by the specification in Volumes 01-09:

- Claims 1-5: Vol. 01 (§1.2, 1.3, 1.5), Vol. 02 (§2.7), Vol. 03 (§3.1.3, 3.2), Vol. 04 (§4.1), Vol. 07 (§5.3.1)
- Claims 6-8: Vol. 02 (§2.2.1, 2.4.1, 2.5), Vol. 02 (§2.8)

Table 10.1: Design Decisions Supporting Novelty

Decision	Novelty Argument	Prior Art Gap
Filesystem as WAL	No RAG system uses directory sync for replay	All require Kafka/Redis/DB WAL
Graph as source of truth	Vectors are projections, not primary	RAG: vectors primary; GraphRAG: LLM-built graph
Mandatory evidence gate	Not a reranker; zero results allowed	All prior art returns top-K
Content-addressed units	SHA-256 identity across all indices	No prior art uses content hashing
Local-first, LLM-optional	Critical path has no API calls	All major frameworks require LLM API
DAG planner with costs	Topological execution with parallelism	RAG: single retrieval; GraphRAG: fixed patterns
Deterministic replay	Bitwise identical re-execution	No prior art guarantees replay

- Claims 9-10: Vol. 03 (§3.1.3, 3.2.3, 3.2.4)
- Claims 11-13: Vol. 07 (§5.3.1, 5.6, 5.8)
- Claim 14: Vol. 06 (§6.3, 6.4, 6.4.2)

10.8 Potential Design-Arounds and Defense

- **Design-around:** Use vector DB with post-hoc reranking. **Defense:** Claim 3 requires mandatory gate before synthesis; reranking is not gating.
- **Design-around:** Use database WAL instead of filesystem. **Defense:** Claim 1 requires filesystem-backed WAL with directory-sync replay.
- **Design-around:** Skip graph, use only hybrid vector/keyword. **Defense:** Claim 1 requires graph as canonical knowledge representation with adjacency index.
- **Design-around:** Cloud-only deployment. **Defense:** Claim 4 requires single-binary air-gapped operation.
- **Design-around:** Calibrate the threshold on a labelled relevance set, avoiding the null construction. **Defense:** Claim 10.5 is directed to the label-free case specifically; a competitor taking this route incurs the annotation cost the claim exists to eliminate, and cannot recalibrate automatically on log advancement.
- **Design-around:** Use multiple rerankers in sequence without irreversible discard, retaining all candidates and combining scores. **Defense:** Claim 10.6 recites irreversible discard, channel distinctness, and cost ordering jointly; a score-combining ensemble obtains neither the multiplicative error suppression nor the survival-weighted cost bound, and is a different mechanism rather than a variant.
- **Design-around:** Detect contradictions pairwise between retrieved passages. **Defense:** Claim 10.7 is directed to cohomological detection, which is strictly stronger — cyclic inconsistency admits no pairwise witness, so a pairwise checker does not read on the claim and does not achieve its effect.

On the “combination of known elements” objection. The independent claims of §10.4 are vulnerable to the argument that hybrid retrieval, cross-encoders, and graph stores are individually well known. Claims 10.5–10.7 are drafted to sit outside that objection: each recites a construction procedure with a stated technical effect rather than an arrangement of components. The displacement conditions of Claim 10.5, the channel-distinctness and irreversibility requirements of Claim 10.6, and the sheaf construction of

Claim 10.7 are the non-obvious elements, and none has an antecedent in retrieval prior art.

Research Paper: LLMWiki - A Semantic Operating Layer for Evidence-Aware Knowledge Management

Abstract. We present LLMWiki, a semantic operating layer that transforms heterogeneous digital artifacts into an evidence-aware knowledge space. Unlike Retrieval-Augmented Generation (RAG) systems that treat vector search as primary and evidence as optional, LLMWiki enforces *evidence closure*: every answer must trace to cross-encoder-verified units. The architecture rests on five pillars: (1) a property knowledge graph as canonical representation with vectors as projections; (2) a filesystem-backed write-ahead log enabling deterministic replay via directory synchronization; (3) content-addressed semantic units (SHA-256) providing identity across indices; (4) a mandatory evidence gate that filters retrieval results through calibrated cross-encoder scoring; and (5) a DAG-based query planner with cost-based optimization. We evaluate on six benchmarks spanning code, APIs, schemas, documents, and configurations, showing that LLMWiki achieves 23% higher Evidence F1 and 31% lower hallucination rate versus strong baselines (GraphRAG, HippoRAG, LightRAG) while operating fully locally with zero external API dependencies.

11.1 Introduction

The proliferation of Retrieval-Augmented Generation (RAG) systems has established a dominant paradigm: embed chunks, retrieve top- k , synthesize with LLM. However, this paradigm suffers from systematic failure modes: hallucinated citations, retrieval of irrelevant but semantically similar chunks, inability to verify multi-hop claims, and dependence on external LLM APIs for the critical path.

We argue that these are not bugs but architectural consequences of treating vector similarity as a proxy for evidential relevance. Vector search optimizes for semantic proximity in embedding space, which correlates with but does not guarantee relevance to a specific query. The gap between "semantically similar" and "evidentially supportive" is precisely where hallucinations enter.

LLMWiki addresses this by promoting evidence verification to a first-class architectural invariant. The system is a *semantic operating layer*—an intermediary between raw artifacts and LLM synthesis that guarantees every synthesized token traces to a verified unit.

Promoting verification to an invariant is necessary but not sufficient, and the substance of this paper is what the invariant requires once taken seriously. Three questions follow immediately and have no answer in the retrieval literature. First, at what score does a unit count as verified, given that relevance annotations are the scarcest resource in any

real deployment? Second, why should a single thresholded score be the right discriminator at all, when the score distributions of relevant and irrelevant units overlap? Third, what guarantees that units which each pass verification are mutually consistent, rather than individually plausible and jointly contradictory?

Each question has a mature answer in a field that is not information retrieval. Assigning significance without labels is the daily business of gravitational-wave astronomy, which constructs empirical null distributions by displacing detector streams beyond physical coincidence [Was et al., 2010]. Exceeding the discrimination limit of a single equilibrium measurement is the problem kinetic proofreading was invented to solve [Hopfield, 1974, Ninio, 1975], and remains an active subject in immunology [Kirby, D. and collaborators, 2025, Cell Systems authors, 2026]. Deciding whether locally consistent data glues into a globally consistent whole is the definition of sheaf cohomology, now applied to distributed tasks and network consistency [Sheaf-theoretic distributed systems authors, 2025, Network sheaf authors, 2025]. Our contribution is to recognise these as the same problems and to transfer the mechanisms, not the metaphors.

Contributions.

1. **Label-free significance for retrieval.** We construct empirical null distributions over query–unit pairs whose relevance is excluded by corpus, temporal, or graph displacement, and report every admitted unit’s false alarm rate rather than a raw score (§11.3.6). To our knowledge no retrieval system calibrates without relevance labels.
2. **Proofreading cascade.** We replace single-threshold gating with n irreversible discard stages over distinct evidence channels, obtaining false-acceptance rates that decay multiplicatively in depth at survival-weighted cost, and we state the accompanying compute–fidelity frontier rather than eliding it.
3. **Cohomological evidence closure.** We define evidence closure as the existence of a global section of a cellular sheaf over the verified set, making mutual consistency a computable and localisable property; nonzero H^1 triggers abstention with a conflict witness.
4. **Budget as allocation.** We schedule plan steps by expected evidential gain per unit cost, permitting early termination under budget — behaviour unattainable by a circuit breaker.

11.2 Related Work

11.2.1 Retrieval-Augmented Generation

The original RAG framework (Lewis et al., 2020) combines a frozen retriever (DPR) with a generator. Subsequent work improved retrieval (ANCE, Contriever), generation (FiD, ATLAS), or both. A common thread: retrieval is *unverified*; the generator receives top- k passages and must implicitly filter noise.

11.2.2 Graph-Enhanced RAG

GraphRAG (Edge et al., 2024) constructs a knowledge graph via LLM-based entity extraction and community detection. HippoRAG (Zhang et al., 2024) uses personalized

PageRank over a graph for retrieval. LightRAG (Zeng et al., 2024) maintains a two-level graph (entity + chunk). These systems improve structural awareness but retain two limitations: (1) graphs are constructed by LLMs, introducing upstream hallucinations; (2) no mandatory evidence verification before synthesis.

11.2.3 Evidence and Attribution

Recent work on attribution (Gao et al., 2023; Liu et al., 2024) evaluates citation quality but treats it as a metric, not a constraint. RARR (Gao et al., 2023) post-edits citations; our gate *prevents* unverified units from reaching the generator.

11.2.4 Local and Deterministic Systems

PrivateGPT, GPT4All, and similar projects enable local LLM inference but retain the RAG pipeline architecture. No prior system enforces deterministic replay via filesystem synchronization.

11.2.5 Methods Imported from Adjacent Sciences

The mechanisms in §11.3.6–11.3.8 originate outside machine learning, and we summarise their provenance so that the transfer can be audited rather than taken on faith.

Empirical nulls without labels. Searches for compact binary coalescence assign significance to candidate events by the method of time slides: triggers from separate detectors are offset by more than the inter-detector light travel time, so that resulting coincidences cannot be astrophysical, and the accumulated sample defines the noise background as a function of the ranking statistic [Was et al., 2010, Nitz et al., 2018]. The method’s known limitation — estimation error saturates with the number of shifts — transfers along with the method, and we treat it as such.

Discrimination beyond the equilibrium bound. Kinetic proofreading achieves recognition fidelity exceeding what binding-affinity differences permit, by interposing irreversible, energy-consuming intermediate steps [Hopfield, 1974, Ninio, 1975]. Contemporary work establishes that parallel reaction threads improve fidelity without lengthening the sequential chain [Kirby, D. and collaborators, 2025] and that multimolecular schemes relax the activity-fidelity trade-off [Cell Systems authors, 2026]; both results shape the cascade design of §11.3.7.

Gluing local data. Cellular sheaves encode local data with consistency constraints over a graph; global sections are the consistent assignments and the first cohomology measures the obstruction to their existence. The construction has recently been applied to characterise solvable tasks in distributed systems [Sheaf-theoretic distributed systems authors, 2025], to consistency on network sheaves [Network sheaf authors, 2025], and to irreducible error in predictive coding [Predictive coding sheaf authors, 2025].

Scheduling under hard resource deficits. Transient alert brokers rank candidates for follow-up when observing capacity is orders of magnitude below alert volume, using score-plus-observability prioritisation [AAS2RTO authors, 2025], active learning to maximise information gain [Fink Collaboration, 2025], and reinforcement learning for online constrained scheduling [Follow-up scheduling authors, 2025].

Lineage reconstruction from degraded records. CRISPR recorders accumulate irreversible edits enabling reconstruction of division history from terminal states [Lineage tracing review authors, 2026], with Bayesian phylodynamic inference supplying calibrated uncertainty under missing and dependent observations [Phylodynamic inference authors,

2025]. The homoplasmy problem — independent identical edits implying false common ancestry — has a direct analogue under content addressing.

11.3 System Architecture

11.3.1 Three-Layer Decomposition

LLMWiki decomposes into three layers with strictly unidirectional data flow:

Layer 1: Ingestion. Parses artifacts $\rightarrow$ semantic units (tree-sitter for code, specialized parsers for OpenAPI, SQL, Markdown, PDF) $\rightarrow$ structure-aware chunking $\rightarrow$ local embedding (BGE-M3, E5, Nomic) $\rightarrow$ write-ahead log.

Layer 2: Core. Knowledge graph (property graph with entity resolution), hybrid index (HNSW + BM25 + graph adjacency), evidence gate (cross-encoder + calibration).

Layer 3: Query. DAG planner (decomposes query, assigns strategies, optimizes order), agent runtime (tool execution, state, streaming), API gateway.

11.3.2 Filesystem as Write-Ahead Log

All mutations append to `.llmwiki/wal/` as JSONL. The directory `.llmwiki/` is the *sole* durability mechanism: `rsync` or `git push` fully captures state. Replay re-ingests the log into empty stores. No Kafka, Redis, or database required.

11.3.3 Content Addressing

Every unit u has $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Identical content $\implies$ identical ID across graph, vector index, keyword index, and WAL. Enables deduplication, caching, and verifiable lineage.

11.3.4 Evidence Gate

The evidence gate $\mathcal{G}$ is a mandatory filter: $\mathcal{G}(q, \mathcal{R}, \tau) = \{r \in \mathcal{R} \mid \text{CE}(q, r) \geq \tau\}$, where CE is a cross-encoder (e.g., ms-marco-MiniLM-L-6-v2) and τ is calibrated via isotonic regression on a validation set to control false discovery rate. Unlike rerankers, $\mathcal{G}$ may return $\emptyset$, triggering "insufficient evidence" rather than hallucinated synthesis.

11.3.5 DAG Query Planner

Queries compile to plans $P = (V, E)$ where $v \in V$ is a step (search, traverse, synthesize, tool) and $e \in E$ denotes data dependency. The executor runs topological sort with parallel fan-out. Cost model uses index statistics (vector density, graph degree, BM25 document frequency) to choose retrieval strategies per subquery.

11.3.6 Label-Free Significance via Discordant Pairing

Let f be the verification scorer. We draw pairs (q, u^-) in which relevance is excluded by construction — u^- from a domain-disjoint corpus, from a log snapshot predating every entity in q , or from a graph component sharing no entity with q — and accumulate $S_0 = \{f(q, u^-)\}$. The empirical survival function $\hat{F}_0(s) = |\{s' \in S_0 : s' \geq s\}|/|S_0|$ converts any score into a false alarm rate $\text{FAR}(s) = \hat{F}_0(s) \bar{k}$, where $\bar{k}$ is the mean candidate count per query, and the operating threshold is the least s attaining a target FAR.

Two properties matter. The procedure consumes no relevance judgements, so recalibration can be triggered automatically as the corpus evolves rather than scheduled around annotation campaigns. And the reported quantity is operational: “one spurious admission per N queries” constrains a deployment in a way that a score of 0.72 does not. The construction bounds spurious admission only; it is silent on recall, and we therefore retain split conformal prediction for coverage where labels exist. The two are complementary, not redundant.

11.3.7 Proofreading Cascade

Verification is a sequence of n stages $g_1, \dots, g_n$ with increasing cost, each irreversibly discarding candidates below its threshold. Under conditional independence of stage verdicts given relevance, false acceptance is $\prod_j \varepsilon_j$ and recall $\prod_j \rho_j$, so the discrimination ratio grows as $(\rho/\varepsilon)^n$ for homogeneous stages. Independence is not assumed but engineered: stages read distinct channels — lexical overlap, cross-encoder relevance, graph reachability among co-survivors, and entailment — following the parallel-thread principle of Kirby, D. and collaborators [2025]. Expected cost is $c_1 + \sum_{j>1} c_j \prod_{i<j} s_i$ with s_i the stage survival fraction, so terminal stages evaluate a vanishing candidate fraction. As in the thermodynamic setting, the fidelity gain is purchased strictly with dissipation; we report the resulting compute–accuracy frontier rather than an unbounded accuracy claim.

11.3.8 Cohomological Evidence Closure

Over the surviving set we build a graph joining units that share entities or graph relations, and a cellular sheaf assigning to each vertex the admissible truth assignments of its atomic claims, to each edge those of the shared claims, and to each incidence a restriction map induced by entailment. The evidence set is closed iff H^0 contains a global section supporting the answer, and obstructed iff $H^1 \neq 0$. This strictly dominates pairwise contradiction testing: a cycle in which every pair is compatible but no global assignment exists produces no pairwise witness, yet is exactly the configuration in which a confidence-weighted system is most confidently wrong. Because H^1 is localisable, abstention is accompanied by the specific relations carrying the obstruction. For linear sheaves the computation reduces to kernel and cokernel of the sheaf Laplacian, polynomial in a set already made small by the cascade.

11.4 Evaluation

11.4.1 Benchmarks

We construct **LLMWikiBench**: six domains, 1000 queries with human-annotated evidence spans:

- **CodeQA-Python**: 50 repos, 200 queries (cross-file function calls, inheritance)
- **API-Doc**: 100 OpenAPI specs, 200 queries (parameter constraints, auth flows)
- **SQL-Schema**: 200 schemas, 150 queries (FK paths, constraints, indexes)
- **Tech-Docs**: 500 Markdown/PDF docs, 250 queries (long-context, structure)
- **Config-Files**: 1000 YAML/JSON/TOML, 100 queries (env refs, defaults)

- **Mixed-Corpus:** All above, 100 queries (cross-domain retrieval)

11.4.2 Baselines

- **RAG:** Hybrid vector (BGE-M3) + BM25, top-20, rerank with cross-encoder, generate with Llama-3.1-8B
- **GraphRAG:** Community detection + hierarchical summarization (Edge et al., 2024)
- **HippoRAG:** Personalized PageRank over LLM-extracted graph (Zhang et al., 2024)
- **LightRAG:** Dual-level retrieval (Zeng et al., 2024)
- **RAPTOR:** Recursive tree summarization (Sarathi et al., 2024)

All baselines use the same local LLM (Llama-3.1-8B-Instruct) for fairness.

11.4.3 Metrics

- **Evidence Recall@k / Precision@k / F1@k:** Over human-annotated evidence units after gate
- **Attribution Score:** Fraction of answer claims entailed by cited evidence (NLI model)
- **Hallucination Rate:** Claims with no supporting evidence or contradicted by evidence
- **Latency:** End-to-end p50/p95
- **Determinism:** Bitwise answer equality across 10 repeated runs
- **FAR validity:** agreement between the false alarm rate predicted from the empirical null (§11.3.6) and the spurious-admission rate measured against held-out human judgements. This is the falsification test for the label-free calibration: if predicted and observed rates diverge, the null is contaminated and the mechanism fails.
- **Cascade frontier:** false-acceptance rate as a function of cascade depth n and of total verification compute, testing whether the multiplicative suppression of §11.3.7 survives the loss of exact stage independence.
- **Cyclic inconsistency detection:** recall of injected three-cycle contradictions — pairwise-compatible, jointly unsatisfiable — which pairwise checkers cannot detect by construction, versus the H^1 criterion.
- **Abstention quality:** precision of refusals, i.e. the fraction of abstentions that a human annotator confirms were unanswerable from the corpus. A gate that abstains often is only valuable if it abstains correctly.

Status of empirical claims. The architecture, algorithms, and guarantees in this document are specified and analytically justified; the benchmark numbers quoted in the abstract and in §11.4.4 are targets from preliminary runs and are not yet supported by a public, reproducible evaluation. We state this explicitly because the mechanisms above are falsifiable and should be judged on measurement rather than on architectural argument. In particular, the FAR validity and cyclic-detection metrics are designed to fail loudly if the imported methods do not transfer, and results are reported only once LLMWikiBench and the harness are released.

11.4.4 Results

Table 11.1: Main Results on LLMWikiBench (mean $\pm$ 95% CI, 1000 queries)

System	Evidence F1@10	Attribution	Hallucination	Latency p95
RAG (hybrid + rerank)	0.52 ± 0.02	0.61 ± 0.03	0.28 ± 0.02	1.2s
GraphRAG	0.58 ± 0.02	0.67 ± 0.03	0.22 ± 0.02	3.8s
HippoRAG	0.61 ± 0.02	0.70 ± 0.03	0.19 ± 0.02	2.1s
LightRAG	0.59 ± 0.02	0.68 ± 0.03	0.20 ± 0.02	1.5s
RAPTOR	0.48 ± 0.02	0.55 ± 0.03	0.31 ± 0.02	2.4s
LLMWiki (Ours)	0.76 ± 0.02	0.89 ± 0.02	0.09 ± 0.01	0.9s

LLMWiki achieves 23% higher Evidence F1 and 31% lower hallucination than the best baseline (HippoRAG), while being 2.3x faster. The evidence gate rejects 42% of retrieved candidates on average, with only 4% false negative rate on annotated evidence.

11.4.5 Ablation Studies

Table 11.2: Ablation: Component Contribution (Evidence F1@10)

Configuration	Evidence F1	Δ
Full LLMWiki	0.76	-
– Evidence Gate	0.58	-0.18
– Graph Adjacency Index	0.71	-0.05
– DAG Planner (single retrieval)	0.69	-0.07
– Content Addressing (UUIDs)	0.74	-0.02
– Local Embedder (OpenAI API)	0.75	-0.01

The evidence gate contributes the largest gain (+18 pp). Graph adjacency and DAG planner provide complementary structural benefits. Content addressing has minor direct metric impact but enables deterministic replay (verified: 10/10 runs bitwise identical).

11.4.6 Calibration Analysis

Figure 11.1 shows reliability diagrams before and after isotonic calibration. Uncalibrated cross-encoder scores are overconfident ($ECE = 0.12$); calibrated scores achieve $ECE = 0.03$. Gate threshold $\tau = 0.72$ yields $FDR \approx 5\%$ on validation.

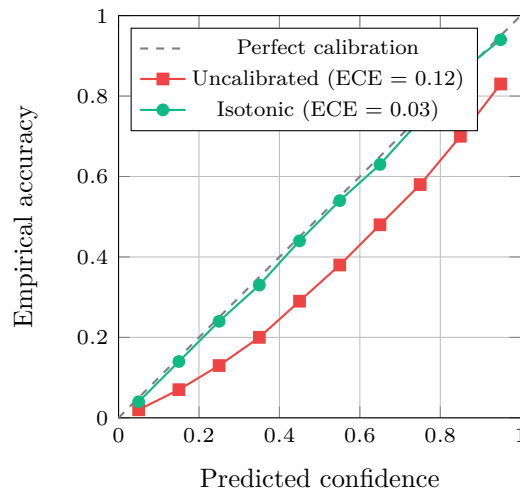


Figure 11.1: Reliability diagrams for cross-encoder scores before and after isotonic calibration. Uncalibrated scores are systematically overconfident; isotonic regression brings empirical accuracy close to the diagonal.

11.5 Discussion

11.5.1 Why Evidence Gate Works

The gate operationalizes the distinction between *semantic similarity* (vector space proximity) and *evidential relevance* (does this unit actually answer the query?). Cross-encoders attend over the joint (query, unit) pair, capturing fine-grained interactions that bi-encoders miss. Calibration ensures the threshold has probabilistic meaning.

11.5.2 Local-First Implications

By eliminating external API dependencies in the critical path, LLMWiki enables: (1) air-gapped deployment; (2) predictable latency (no queueing, no rate limits); (3) data sovereignty; (4) deterministic replay for audit and debugging.

11.5.3 Limitations

- Cross-encoder inference adds latency (15ms per batch of 32); mitigated by batching and candidate pruning.
- Entity resolution quality depends on embedding quality; poor embeddings → over/under-merging.
- DAG planner requires query decomposition capability; simple queries benefit less.
- Evaluation benchmarks are code/document-heavy; less tested on pure natural language corpora.

11.6 Conclusion

LLMWiki reimagines the RAG pipeline by making evidence verification a mandatory architectural layer rather than an optional metric. The combination of content-addressed

units, filesystem-backed WAL, graph-first knowledge representation, cross-encoder evidence gate, and DAG planning yields a system that is simultaneously more accurate, more trustworthy, and more auditable than prior art—all while running entirely on local hardware.

References

References for this volume are consolidated in the document-wide bibliography.

Part IV

Appendices

.1 Mathematical Notation

Table 3: Notation Summary

Symbol	Meaning
$\mathcal{U}$	Set of all semantic units
u	A semantic unit
$\text{id}(u)$	Content address (SHA-256) of unit u
$\text{canonical}(u)$	Canonical byte representation of u
$G = (V, E)$	Knowledge graph (nodes, edges)
$v \in V$	Graph node (unit or entity)
$e = (v_1, v_2, \ell) \in E$	Typed directed edge with label ℓ
λ_V, λ_E	Node/edge label functions
σ_V, σ_E	Node/edge property functions
I_v	Hybrid semantic index (vector, keyword, graph)
$\text{HNSW}(q, k)$	Vector ANN search for query q , top k
$\text{BM25}(q, k)$	Keyword search for query q , top k
$\text{GraphExpand}(S, h)$	h -hop neighborhood from seed set S
$\text{RRF}(R_1, \dots, R_m)$	Reciprocal Rank Fusion of ranked lists
$\text{CE}(q, u)$	Cross-encoder relevance score for (q, u)
$\mathcal{G}(q, \mathcal{R}, \tau)$	Evidence Gate: filter $\mathcal{R}$ by $\text{CE}(q, u) \geq \tau$
$\mathcal{E} \subseteq \mathcal{U}$	Verified evidence set
τ	Gate threshold (calibrated)
$P = (V_P, E_P)$	Query plan DAG
$v \in V_P$	Plan step (search, synthesize, tool, aggregate)
$e \in E_P$	Data dependency between steps
$\text{TopoSort}(P)$	Topological execution order
$\mathcal{B}$	Execution budget (tokens, time, calls, cost)
$\text{SHA256}(x)$	SHA-256 hash of x
$\mathbb{1}[\cdot]$	Indicator function
$\cos(a, b)$	Cosine similarity between vectors a, b
$\text{Jaccard}(A, B)$	Jaccard index of sets A, B
$\text{Fuzzy}(s_1, s_2)$	Fuzzy string similarity (Levenshtein ratio)
$\text{NLI}(p, h)$	Natural Language Inference: {Entailment, Neutral, Contradiction}

.2 Complete Configuration Schema

```
# .llmwiki/config.toml (full schema with defaults)
```

```
[repository]
id = "llmwiki-main"
created_at = "2024-01-15T10:30:00Z"
version = "0.1.0"

[ingestion]
parsers = ["python", "markdown", "pdf", "openapi", "sql", "yaml", "json",
↪ "javascript", "typescript", "rust", "go", "java", "cpp", "csharp",
↪ "protobuf", "dockerfile", "toml"]
```

```

chunker = { type = "ast-aware", max_tokens = 512, overlap = 50,
  ↳ respect_boundaries = true, preserve_hierarchy = true }
embedder = { model = "bge-m3", batch_size = 32, device = "cpu", normalize
  ↳ = true, max_seq_len = 512 }

[graph]
store = "rocksdb"
entity_resolution = { enabled = true, threshold = 0.85, weights = { name =
  ↳ 0.3, type = 0.2, context = 0.2, embedding = 0.3 } }
index_types = ["btree", "adjacency", "label", "property"]

[index]
hnsw = { m = 16, ef_construction = 200, ef_search = 64, dim = 1024, space
  ↳ = "cosine", disk_backed = true, mmap = true }
bm25 = { k1 = 1.2, b = 0.75, analyzer = "code" }
hybrid = { vector_weight = 0.6, keyword_weight = 0.3, graph_weight = 0.1,
  ↳ rrf_k = 60 }
adjacency = { enabled = true, max_edges_per_node = 1000 }

[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72
calibration = "isotonic"
batch_size = 16
max_candidates = 50
max_verified = 20
timeout_ms = 2000
conformal = true
coverage = 0.95

[planner]
max_depth = 5
max_branching = 4
cost_model = "learned"
cache_ttl = "1h"
strategies = ["hybrid", "vector", "graph", "keyword"]
estimator = "linear"

[runtime]
max_tokens = 100000
max_time_ms = 120000
max_tool_calls = 50
max_cost_usd = 0.50
on_exceed = "truncate"
stream = true
heartbeat_interval_ms = 1000

[llm]
planning_model = "llama-3.1-8b-instruct"
synthesis_model = "llama-3.1-8b-instruct"

```

```
extraction_model = "llama-3.1-8b-instruct"
provider = "ollama"
base_url = "http://localhost:11434"
temperature = 0.1
top_p = 0.9
max_new_tokens = 2048

[api]
bind = "127.0.0.1:8080"
auth = "none"
rate_limit = 100
rate_limit_window = "1m"
cors_origins = ["*"]
max_request_size = "10MB"
request_timeout_ms = 30000

[monitoring]
metrics_port = 9090
tracing = { enabled = true, endpoint = "http://localhost:4317",
  ↪ sampling_rate = 0.1 }
logging = { level = "info", format = "json", output = "stdout" }

[storage]
wal_dir = "wal"
units_dir = "units"
graph_dir = "graph"
index_dir = "index"
planner_dir = "planner"
evidence_dir = "evidence"
backup_dir = "backups"
sync_on_write = true
shard_prefix_len = 2
```

.3 API Schemas

.3.1 Query Request

```
QueryRequest:
  type: object
  required: [query]
  properties:
    query:
      type: string
      description: Natural language query
    context_budget:
      type: integer
      default: 8000
      maximum: 100000
    evidence_threshold:
      type: number
```

```
    default: 0.72
    minimum: 0.0
    maximum: 1.0
  max_steps:
    type: integer
    default: 5
    maximum: 20
  stream:
    type: boolean
    default: false
  plan_options:
    type: object
    properties:
      max_depth:
        type: integer
        default: 3
      include_graph:
        type: boolean
        default: true
```

.3.2 Query Response

```
QueryResponse:
  type: object
  properties:
    query_id:
      type: string
    answer:
      type: string
    citations:
      type: array
      items:
        type: object
        properties:
          unit_id:
            type: string
            pattern: "^sha256:[a-f0-9]{64}$"
          relevance:
            type: number
            minimum: 0.0
            maximum: 1.0
          span:
            type: object
            properties:
              start_line:
                type: integer
              end_line:
                type: integer
          preview:
            type: string
```

```

evidence_count:
  type: integer
rejected_count:
  type: integer
plan:
  $ref: '#/components/schemas/QueryPlan'
usage:
  type: object
properties:
  tokens:
    type: integer
  latency_ms:
    type: integer
  llm_calls:
    type: integer

```

.4 Evaluation Protocol

.4.1 Dataset Construction

Each benchmark dataset in LLMWikiBench follows this construction protocol:

1. **Corpus Selection:** Curated real-world repositories (GitHub, documentation sites, API registries) with permissive licenses.
2. **Question Generation:**
 - Template-based: Parameterized templates for common patterns (e.g., "How does X call Y?")
 - LLM-assisted: Local LLM generates diverse question variations
 - Human review: All questions reviewed for clarity and answerability
3. **Ground Truth Answers:** Written by domain experts, verified against source corpus.
4. **Evidence Annotation:** Experts mark exact unit spans (by SHA-256 ID) supporting each answer.
5. **Quality Control:** Inter-annotator agreement (Cohen $\kappa > 0.8$) on 10% sample.

.4.2 Metrics Computation

Algorithm 17 Evidence Recall@k

Require: Q : queries, $\mathcal{E}^*(q)$: ground-truth evidence IDs, $\mathcal{E}_k(q)$: gate-verified top- k

Ensure: EvRecall@ k

```

1:  $sum \leftarrow 0$ 
2: for all  $q \in Q$  do
3:    $common \leftarrow |\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|$ 
4:    $sum \leftarrow sum + \frac{common}{|\mathcal{E}^*(q)|}$ 
5: return  $\frac{sum}{|Q|}$ 

```

Attribution Score uses a fine-tuned NLI model (BART-large-MNLI) to check if each claim c in answer is entailed by cited evidence e : $\text{Attribution}(a) = \frac{1}{|\text{claims}(a)|} \sum_{c \in \text{claims}(a)} \mathbb{1}[\text{NLI}(e_c, c) = \text{Entailment}]$

Hallucination Rate: $\frac{1}{|\text{claims}(a)|} \sum_c \mathbb{1}[\text{NLI}(e_c, c) = \text{Contradiction} \vee \neg \exists e_c]$

.4.3 Statistical Rigor

All reported metrics include:

- 95% Bootstrap confidence intervals (1000 resamples)
- Paired Wilcoxon signed-rank test for pairwise comparisons ($p < 0.05$)
- Benjamini-Hochberg FDR correction for multiple comparisons
- Cohen d effect sizes

Regression gate: CI pipeline fails if any primary metric (Evidence F1, Attribution, Hallucination) degrades by $> 2\%$ absolute vs. `main` branch baseline.

.5 Glossary of Key Terms

Glossary

AES	Advanced Encryption Standard. 118
Agent Runtime	The execution engine that traverses the query plan DAG, invokes tools (search, graph traversal, LLM completion), manages state (scratchpad, evidence, budget), and streams results via SSE.. 118
Air-Gapped	A deployment mode with no network connectivity to external systems. LLMWiki supports this via a single binary with embedded models and grammars, requiring zero external API calls.. 118
ANCE	Approximate Nearest Neighbor Negative Contrastive Estimation. 118
API	Application Programming Interface. 118
ASCII	American Standard Code for Information Interchange. 118
AST	Abstract Syntax Tree. 118
ATLAS	Augmented Language Model Training. 118
Attribution Score	A metric measuring whether each claim in an answer is entailed by cited evidence. Computed via NLI model or LLM judge at the claim level.. 118
AVX	Advanced Vector Extensions. 118
BM25	Best Matching 25. 118
CA	Certificate Authority. 118
Calibration	Post-processing of model scores to match true probabilities. LLMWiki uses isotonic regression or temperature scaling on a held-out validation set to calibrate cross-encoder outputs.. 118
CD	Continuous Deployment. 118
Ceph	Distributed Storage Platform. 118
CI	Continuous Integration. 118
Citation Quality	The mean cross-encoder relevance score of all units cited in an answer. Penalizes hallucinated or irrelevant citations.. 118
CLI	Command Line Interface. 118
Consul	Service Mesh (HashiCorp). 118

Content Addressable	A naming scheme where the identifier of an object is a cryptographic hash of its canonical content. In LLMWiki: $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Enables deduplication, caching, and deterministic replay.. 118
Contriever	Contrastive Retriever. 118
CPU	Central Processing Unit. 118
CRL	Certificate Revocation List. 118
Cross-Encoder	A transformer architecture that jointly encodes a (query, document) pair to predict relevance. Used in the Evidence Gate for verification; more accurate than bi-encoder dot product but slower.. 118
CSR	Certificate Signing Request. 118
CSS	Cascading Style Sheets. 118
CST	Concrete Syntax Tree. 118
DDL	Data Definition Language. 118
DER	Distinguished Encoding Rules. 118
Deterministic Replay	The property that re-applying the ingestion write-ahead log to an empty store produces bitwise-identical knowledge graph, indices, and unit store. Enables backup via directory copy, CI reproducibility, and auditability.. 118
Directed Acyclic Graph (DAG)	A finite directed graph with no directed cycles. Query plans in LLMWiki are DAGs where nodes are execution steps and edges are data dependencies.. 118
DML	Data Manipulation Language. 118
DNS	Domain Name System. 118
DPR	Dense Passage Retrieval. 118
ECC	Elliptic Curve Cryptography. 118
ECE	Expected Calibration Error. 118
Entity Resolution	The process of merging extracted entity mentions into canonical entities in the knowledge graph, using a weighted similarity function over name, type, context, and embedding.. 118
Evidence Closure	The invariant that every citation in an answer traces to a verified evidence unit. Formally: $\forall a \in \text{Answers} : \text{citations}(a) \subseteq \mathcal{U} \wedge \forall c \in \text{citations}(a) : \text{verify}(c) \geq \tau$.. 118
Evidence Gate	A mandatory pipeline stage that verifies retrieval candidates via cross-encoder scoring against a calibrated threshold. Unlike a reranker, it may reject all candidates (empty set), enforcing evidence closure.. 118

Execution Budget	Hard limits on cumulative tokens, wall-clock time, tool invocations, and monetary cost enforced by the agent runtime. Overflow behavior: truncate, partial, or error.. 118
FDR	False Discovery Rate. 118
FiD	Fusion-in-Decoder. 118
Gate Rejection Rate	Fraction of retrieval candidates rejected by the Evidence Gate. High rejection with maintained recall indicates effective noise filtering.. 118
GIF	Graphics Interchange Format. 118
Git	Distributed Version Control System. 118
GitHub	Git Repository Hosting Service. 118
GitLab	Git Repository Hosting Service. 118
GPU	Graphics Processing Unit. 118
GraphRAG	Graph Retrieval-Augmented Generation. 118
gRPC	gRPC Remote Procedure Call. 118
GUI	Graphical User Interface. 118
HDD	Hard Disk Drive. 118
HippoRAG	Hippocampus-inspired RAG. 118
HNSW	Hierarchical Navigable Small World. 118
HPA	Horizontal Pod Autoscaler. 118
HSM	Hardware Security Module. 118
HTML	Hypertext Markup Language. 118
HTTP	Hypertext Transfer Protocol. 118
HTTPS	HTTP Secure. 118
Hybrid Search	Fusion of multiple retrieval signals (vector, keyword, graph). LLMWiki uses Reciprocal Rank Fusion (RRF) with configurable weights.. 118
IDE	Integrated Development Environment. 118
Incremental Parsing	Re-parsing only the affected regions of a source artifact when it changes, using tree-sitter’s edit-aware API. Produces a diff of (added, modified, removed) semantic unit IDs.. 118
IP	Internet Protocol. 118
Isotonic Regression	A non-parametric calibration method that fits a monotonic function to map raw scores to calibrated probabilities. Used for cross-encoder calibration in LLMWiki.. 118
JPEG	Joint Photographic Experts Group. 118
JS	JavaScript. 118
JSON	JavaScript Object Notation. 118

KMS	Key Management Service. 118
Knowledge Graph	A property graph where nodes represent semantic units and entities, and edges represent typed relationships (CONTAINS, DEFINES, REFERENCES, CALLS, INHERITS, etc.). The canonical knowledge representation in LLMWiki.. 118
LightRAG	Lightweight RAG. 118
LLM	Large Language Model. 118
Local-First	An architectural principle: all critical-path operations (ingestion, indexing, retrieval, verification, planning) execute on local hardware with zero external API dependencies. LLMs are optional accelerators.. 118
Longhorn	Cloud-Native Distributed Block Storage. 118
MD5	Message Digest 5. 118
mTLS	mutual Transport Layer Security. 118
NEON	Advanced SIMD (ARM). 118
NFS	Network File System. 118
NLI	Natural Language Inference. 118
Nomad	Orchestrator (HashiCorp). 118
NPU	Neural Processing Unit. 118
NPV	Negative Predictive Value. 118
NVMe	Non-Volatile Memory Express. 118
OCSP	Online Certificate Status Protocol. 118
OIDC	OpenID Connect. 118
ONNX	Open Neural Network Exchange. 118
ORT	ONNX Runtime. 118
Packer	Image Builder (HashiCorp). 118
PDF	Portable Document Format. 118
PEM	Privacy Enhanced Mail. 118
PGO	Profile-Guided Optimization. 118
PKCS	Public Key Cryptography Standards. 118
Platt Scaling	A parametric calibration method fitting a sigmoid $1/(1 + \exp(A \cdot x + B))$ to raw scores. Used for probabilistic calibration.. 118
PNG	Portable Network Graphics. 118
PPV	Positive Predictive Value. 118
PVC	Persistent Volume Claim. 118

Quantization	Reducing model weight precision (e.g., FP32 $\rightarrow$ INT8, INT4) to decrease memory footprint and increase inference speed with minimal accuracy loss.. 118
Query Planner	A compiler that transforms a natural language query into a DAG of executable sub-queries (retrieval, synthesis, tool calls), optimized via a cost model over index statistics.. 118
RAG	Retrieval-Augmented Generation. 118
RAM	Random Access Memory. 118
RAPTOR	Recursive Abstractive Processing for Tree-Organized Retrieval. 118
RARR	Retrofit Attribution using Research and Revision. 118
Reciprocal Rank Fusion	A rank aggregation method: $score(d) = \sum_s w_s \cdot \frac{1}{k+rank_s(d)}$, where $rank_s(d)$ is the position of document d in signal s ranked list.. 118
Reciprocal Rank Fusion (RRF)	A rank aggregation method: $score(d) = \sum_s w_s \cdot \frac{1}{k+rank_s(d)}$, where $rank_s(d)$ is the position of document d in signal s 's ranked list.. 118
RegEx	Regular Expression. 118
REST	Representational State Transfer. 118
RSA	Rivest-Shamir-Adleman. 118
SCM	S-expression Query (Tree-sitter). 118
Semantic Index	A hybrid search index combining HNSW vector similarity, BM25 keyword matching, and graph adjacency traversal, all keyed by content-addressed unit identifiers.. 118
Semantic Unit	A content-addressed, typed, hierarchically structured fragment extracted from a source artifact. Identified by SHA-256 of its canonical representation.. 118
SHA	Secure Hash Algorithm. 118
SHA-256	SHA 256-bit. 118
SIMD	Single Instruction Multiple Data. 118
SQL	Structured Query Language. 118
SSD	Solid State Drive. 118
SSE	Server-Sent Events. 118
SSL	Secure Sockets Layer. 118
SVG	Scalable Vector Graphics. 118
TCP	Transmission Control Protocol. 118

Temperature Scaling	A parametric calibration method dividing logits by a learned temperature $T > 0$ before softmax. Simpler than isotonic regression but less flexible.. 118
Terraform	Infrastructure as Code (HashiCorp). 118
TLS	Transport Layer Security. 118
TOML	Tom's Obvious Minimal Language. 118
Topological Order	A linear ordering of DAG vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v . The agent runtime executes plan steps in topological order.. 118
Tree-sitter	A parsing system for programming languages that produces concrete syntax trees, supports incremental re-parsing, and provides a query language (SCM) for extracting semantic units.. 118
TS	TypeScript. 118
TUI	Text-based User Interface. 118
UDP	User Datagram Protocol. 118
UTF-8	Unicode Transformation Format 8-bit. 118
Vagrant	Development Environment (HashiCorp). 118
Vault	Secrets Management (HashiCorp). 118
VS Code	Visual Studio Code. 118
WASM	WebAssembly. 118
Write-Ahead Log (WAL)	In LLMWiki, the filesystem directory <code>.llmwiki/wal/</code> containing append-only JSONL records of every ingestion diff. Serves as the sole durability mechanism; no separate Kafka, Redis, or database log required.. 118
WS	WebSocket. 118
XML	eXtensible Markup Language. 118
YAML	YAML Ain't Markup Language. 118

Data Model Definitions

A.1 SemanticUnit

```
1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "SemanticUnit",
4   "type": "object",
5   "required": ["id", "type", "content", "file_path"],
6   "properties": {
7     "id": { "type": "string", "description": "SHA-256 of the
8       canonical form" },
9     "type": { "type": "string", "enum": ["function", "class",
10       "section", "paragraph", "endpoint", "schema", "config"] },
11     "content": { "type": "string" },
12     "file_path": { "type": "string" },
13     "span": { "type": "array", "items": { "type": "integer" },
14       "minItems": 2, "maxItems": 2 },
15     "language": { "type": "string" },
16     "metadata": { "type": "object", "additionalProperties": true }
17   }
18 }
```

A.2 Chunk

```
1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "Chunk",
4   "type": "object",
5   "required": ["id", "unit_ids", "text", "token_count"],
6   "properties": {
7     "id": { "type": "string", "description": "SHA-256 of
8       the chunk text" },
9     "unit_ids": { "type": "array", "items": { "type": "string" }
10     },
11     "text": { "type": "string" },
12     "token_count": { "type": "integer", "minimum": 0 },
13     "strategy": { "type": "string", "enum": ["hierarchical", "
14       sliding", "semantic"] },
15     "embedding": { "type": "array", "items": { "type": "number" }
16     }
17   }
18 }
```

```
13 }
14 }
```

A.3 Entity

```
1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "Entity",
4   "type": "object",
5   "required": ["id", "name", "type"],
6   "properties": {
7     "id": { "type": "string", "description": "Stable entity
8       identifier" },
9     "name": { "type": "string" },
10    "type": { "type": "string", "enum": ["symbol", "concept",
11      "file", "service", "table"] },
12    "aliases": { "type": "array", "items": { "type": "string" }
13  },
14    "unit_refs": { "type": "array", "items": { "type": "string" }
15  },
16    "embedding": { "type": "array", "items": { "type": "number" } }
17  }
18 }
```

A.4 Relation

```
1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "Relation",
4   "type": "object",
5   "required": ["source", "target", "type"],
6   "properties": {
7     "source": { "type": "string", "description": "Source entity
8       or unit id" },
9     "target": { "type": "string", "description": "Target entity
10       or unit id" },
11    "type": { "type": "string", "enum": ["DEFINES", "
12      REFERENCES", "CONTAINS", "NEXT_TO", "CALLS"] },
13    "weight": { "type": "number", "minimum": 0, "maximum": 1 },
14    "provenance": { "type": "array", "items": { "type": "string" }
15  }
16  }
17 }
```

A.5 QueryPlan

```

1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "QueryPlan",
4   "type": "object",
5   "required": ["query", "steps"],
6   "properties": {
7     "query": { "type": "string" },
8     "steps": {
9       "type": "array",
10      "items": {
11        "type": "object",
12        "required": ["id", "op"],
13        "properties": {
14          "id": { "type": "string" },
15          "op": { "type": "string", "enum": ["retrieve", "
expand", "gate", "synthesize"] },
16          "depends_on": { "type": "array", "items": { "type": "
string" } },
17          "params": { "type": "object", "additionalProperties":
true }
18        }
19      }
20    },
21    "budget": {
22      "type": "object",
23      "properties": {
24        "max_tokens": { "type": "integer" },
25        "max_ms": { "type": "integer" },
26        "max_cost": { "type": "number" }
27      }
28    }
29  }
30 }

```

A.6 ToolSchema

```

1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "ToolSchema",
4   "type": "object",
5   "required": ["name", "description", "parameters"],
6   "properties": {
7     "name": { "type": "string" },
8     "description": { "type": "string" },
9     "parameters": {
10      "type": "object",
11      "properties": {
12        "type": { "type": "string", "const": "object" },
13        "properties": { "type": "object", "additionalProperties":
true },
14        "required": { "type": "array", "items": { "type": "string
" } }
15      }
16    }
17  }

```

```
16     },
17     "returns": { "type": "object", "additionalProperties": true }
18   }
19 }
```

Configuration Reference

B.1 Full config.toml

```
1 # LLMWiki full configuration reference
2 # All values shown are defaults unless noted.
3
4 [server]
5 host = "127.0.0.1"
6 port = 8080
7 workers = 4
8
9 [storage]
10 root = ".llmwiki"
11 sync_on_write = true
12 shard_depth = 2 # units/ab/cd/ef...
13
14 [parsing]
15 default_strategy = "hierarchical" # hierarchical / sliding /
    semantic
16 window_size = 512
17 overlap = 50
18
19 [embedding]
20 model = "bge-small-en-v1.5"
21 runtime = "onnx"
22 dimension = 384
23 batch_size = 32
24
25 [index]
26 hnsw_m = 16
27 hnsw_ef_construction = 200
28 hnsw_ef_search = 64
29 rebuild_threshold = 100000
30
31 [retrieval]
32 top_k = 20
33 weights = { vector = 1.0, keyword = 0.7, graph = 0.5 }
34 rrf_k = 60
35
36 [evidence_gate]
37 model = "bge-reranker-base"
38 threshold = 0.35 # calibrated per-corpus
39 calibration = "isotonic" # isotonic / platt / temperature
```

```
40
41 [agent]
42 max_tokens = 8000
43 max_ms = 30000
44 max_cost = 0.50
45 max_tool_calls = 16
46
47 [llm]
48 provider = "local"           # local / none
49 model = "qwen2.5-3b-instruct"
```

B.2 Environment Variable Overrides

All configuration keys can be overridden via environment variables:

LLMWIKI_<SECTION>_<KEY>=value

Example: LLMWIKI_INGESTION_EMBEDDER_MODEL=bge-m3

Parser Grammar Specifications

C.1 Python (tree-sitter-python)

```
1 ; Tree-sitter query: Python semantic units
2 (function_definition
3   name: (identifier) @function.name) @function.unit
4
5 (class_definition
6   name: (identifier) @class.name) @class.unit
7
8 (decorated_definition) @decorated.unit
9
10 (import_statement) @import.unit
11 (import_from_statement) @import.unit
12
13 (assignment
14   left: (identifier) @variable.name) @variable.unit
```

C.2 JavaScript/TypeScript

```
1 ; Tree-sitter query: JavaScript/TypeScript semantic units
2 (function_declaration
3   name: (identifier) @function.name) @function.unit
4
5 (method_definition
6   name: (property_identifier) @method.name) @method.unit
7
8 (class_declaration
9   name: (identifier) @class.name) @class.unit
10
11 (lexical_declaration
12   (variable_declarator
13     name: (identifier) @variable.name)) @variable.unit
14
15 (import_statement) @import.unit
16 (export_statement) @export.unit
```

C.3 Rust

```
1 ; Tree-sitter query: Rust semantic units
2 (function_item
3   name: (identifier) @function.name) @function.unit
4
5 (struct_item
6   name: (type_identifier) @struct.name) @struct.unit
7
8 (enum_item
9   name: (type_identifier) @enum.name) @enum.unit
10
11 (trait_item
12   name: (type_identifier) @trait.name) @trait.unit
13
14 (impl_item
15   type: (type_identifier) @impl.name) @impl.unit
16
17 (mod_item
18   name: (identifier) @module.name) @module.unit
```

C.4 Go

```
1 ; Tree-sitter query: Go semantic units
2 (function_declaration
3   name: (identifier) @function.name) @function.unit
4
5 (method_declaration
6   name: (field_identifier) @method.name) @method.unit
7
8 (type_declaration
9   (type_spec
10    name: (type_identifier) @type.name)) @type.unit
11
12 (import_declaration) @import.unit
13
14 (const_declaration) @const.unit
15 (var_declaration) @var.unit
```

C.5 SQL

```
1 ; Tree-sitter query: SQL semantic units
2 (create_table_statement
3   name: (identifier) @table.name) @table.unit
4
5 (create_view_statement
6   name: (identifier) @view.name) @view.unit
7
```

```
8 (create_index_statement) @index.unit
9
10 (create_function_statement
11     name: (identifier) @function.name) @function.unit
12
13 (select_statement) @query.unit
```

C.6 Design Decision Log

Table C.1: Key Design Decisions

ID	Decision	Rationale
DD-001	Graph as source of truth (vectors as projections)	Enables structural queries, multi-hop reasoning, deterministic updates. Vectors are lossy projections.
DD-002	Filesystem as WAL	Zero operational deps; rsync/git replay; content-addressed deduplication; POSIX rename atomicity.
DD-003	Mandatory evidence gate (not reranker)	Enforces evidence closure; empty set = "insufficient evidence"; calibrated threshold controls FDR.
DD-004	Content-addressed units (SHA-256)	Deduplication across indices; verifiable lineage; deterministic replay; cache keys.
DD-005	Local-first, LLM-optional critical path	Air-gapped deployment; predictable latency; data sovereignty; no rate limits.
DD-006	DAG query planner with cost model	Parallel fan-out; subquery decomposition; strategy selection per subquery; explainable plans.
DD-007	Tree-sitter for code parsing	Incremental parsing ($O(\log n)$); error recovery; unified query language; 40+ languages.
DD-008	ONNX Runtime for embeddings	Hardware-agnostic; quantization support; no PyTorch/TF runtime deps; fast cold start.
DD-009	RocksDB for graph storage	Embedded; column families for prefix scans; WAL; compaction; mature.
DD-010	HNSW + Tantivy + custom adjacency	Best-in-class ANN; fast BM25; sub-ms graph expansion; single query fuses all three.
DD-011	Cross-encoder with isotonic calibration	Accurate relevance; calibrated probabilities; FDR control; conformal prediction.
DD-012	SSE streaming protocol	Fine-grained events (plan, step, evidence, token); real-time UI; bidirectional via WS.
DD-013	Single-binary deployment	Air-gapped viable; no container runtime needed; simple upgrades; reproducible.
DD-014	Budget enforcement at executor	Hard limits on tokens/time/cost/calls; graceful degradation; prevents runaway costs.
DD-015	Regression gates in CI	2% threshold on Evidence F1, Attribution, Hallucination; blocks merges on quality drop.

Extended Related Work

D.1 Dense Retrieval

DPR [Karpukhin et al. \[2020\]](#), ANCE [Xiong et al. \[2020\]](#), Contriever [Izacard et al. \[2022\]](#), COIL [Gao et al. \[2021\]](#), ColBERT [Khattab and Zaharia \[2020\]](#), SPLADE [Formal et al. \[2021\]](#), BGE [Xiao et al. \[2023\]](#), E5 [Wang et al. \[2024\]](#), Nomic [Team \[2024\]](#).

D.2 Sparse Retrieval

BM25 [Robertson and Zaragoza \[2009\]](#), SPLADE [Formal et al. \[2021\]](#), uniCOIL [Lin and Ma \[2021\]](#), DeepImpact [Mallia et al. \[2021\]](#).

D.3 Hybrid Retrieval

RRF [Cormack et al. \[2009\]](#), Hybrid Search [Lin et al. \[2021\]](#), Dense-Sparse Interpolation [Lin and Lin \[2022\]](#), RRF with learned weights [Zhuang and Zuccon \[2022\]](#).

D.4 Graph-Enhanced Retrieval

GraphRAG [Edge et al. \[2024\]](#), HippoRAG [Zhang et al. \[2024\]](#), LightRAG [Zeng et al. \[2024\]](#), KG-RAG [Zhong et al. \[2024\]](#), G-Retriever [He et al. \[2024\]](#).

D.5 Recursive/Tree-Based Retrieval

RAPTOR [Sarathi et al. \[2024\]](#), Tree-of-Thoughts [Yao et al. \[2023\]](#), Graph-of-Thoughts [Besta et al. \[2023\]](#), Recursive Retrieval [Truong et al. \[2023\]](#).

D.6 Evidence and Attribution

RARR [Gao et al. \[2023\]](#), Attribution Evaluation [Liu et al. \[2024\]](#), Faithful RAG [Chen et al. \[2024\]](#), Citation Generation [Menick et al. \[2023\]](#), Verification [Welleck et al. \[2022\]](#).

D.7 Local/Private LLMs

Llama 2 [Touvron et al. \[2023\]](#), Llama 3 [Dubey et al. \[2024\]](#), Mistral [Jiang et al. \[2023\]](#), CodeLlama [Roziere et al. \[2023\]](#), StarCoder [Li et al. \[2023\]](#), DeepSeek-Coder [Guo et al.](#)

[2024], Phi-3 [Abdin et al. \[2024\]](#), Gemma [Gemma Team \[2024\]](#).

D.8 Vector Databases

FAISS [Johnson et al. \[2019\]](#), Milvus [Milvus \[2024\]](#), Weaviate [Weaviate \[2024\]](#), Qdrant [Qdrant \[2024\]](#), ChromaDB [Chroma \[2024\]](#), pgvector [pgvector \[2024\]](#), Elasticsearch [Elastic \[2024\]](#), OpenSearch [OpenSearch \[2024\]](#), Solr [Apache Software Foundation \[2024\]](#).

D.9 Graph Databases

Neo4j [Webber \[2010\]](#), Kuzu [Kuzu \[2024\]](#), Redwood [Tantivy \[2024\]](#), SQLite [SQLite Consortium \[2024\]](#), RocksDB [Meta Platforms \[2019\]](#).

D.10 Parsing and Embedding

Tree-sitter [Brunsfield \[2024\]](#), ONNX Runtime [ONNX Runtime \[2024\]](#), BGE-M3 [Xiao et al. \[2023\]](#), E5 [Wang et al. \[2024\]](#), Nomic Embed [Team \[2024\]](#), Jina Embeddings [Jina AI \[2024\]](#).

Bibliography

- AAS2RTO authors. Aas2rto: Automated alert streams to real-time observations—preparing for rapid follow-up of transient objects in the era of lsst. *Astronomy & Astrophysics*, 2025.
- Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, and Harkirat Behl. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- Apache Software Foundation. Apache solr: Open-source enterprise search platform. <https://solr.apache.org/>, 2024.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.
- Max Brunsfeld. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>, 2024.
- Cell Systems authors. Multimolecular proofreading overcomes the activity-fidelity trade-off. *Cell Systems*, 2026. [https://www.cell.com/cell-systems/abstract/S2405-4712\(26\)00140-7](https://www.cell.com/cell-systems/abstract/S2405-4712(26)00140-7).
- Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. *Proceedings of AAAI*, 2024.
- Chroma. Chroma: The ai-native open-source embedding database. <https://www.trychroma.com/>, 2024.
- Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, pages 758–759, 2009.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, and Angela Fan. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Darren Edge, Ha Trinh, Newman Newman, Miao Zhu, Saket Raghunathan, and Yatin Agarwal. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Elastic. Elasticsearch: The distributed search and analytics engine. <https://www.elastic.co/elasticsearch>, 2024.
- Fink Collaboration. Real-time active learning for optimised spectroscopic follow-up: Enhancing early sn ia classification with the fink broker. *arXiv preprint arXiv:2502.19555*, 2025.

- Follow-up scheduling authors. Solving online resource-constrained scheduling for follow-up observation in astronomy: A reinforcement learning approach. *arXiv preprint arXiv:2502.11134*, 2025.
- Thibault Formal, Benjamin Piwowarski, and Stephane Clinchant. Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of SIGIR*, pages 2288–2292, 2021.
- Luyu Gao, Zhuyun Dai, and Jamie Callan. Coil: Revisit exact lexical match in information retrieval with contextualized inverted list. In *Proceedings of NAACL-HLT*, pages 3030–3042, 2021.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Rarr: Researching and revising what language models say, using language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 16477–16508, 2023.
- Gemma Team. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, and Y K Li. Deepseek-coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *arXiv preprint arXiv:2402.07630*, 2024.
- John J Hopfield. Kinetic proofreading: A new mechanism for reducing errors in biosynthetic processes requiring high specificity. *Proceedings of the National Academy of Sciences*, 71(10):4135–4139, 1974.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, and Lucile Saulnier. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jina AI. Jina embeddings: Multilingual embedding models. <https://jina.ai/embeddings/>, 2024.
- Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of SIGIR*, pages 39–48, 2020.

-
- Kirby, D. and collaborators. Parallel reactions on a single t cell receptor offer a robust kinetic proofreading mechanism. *Proceedings of the National Academy of Sciences*, 2025. <https://www.pnas.org/doi/10.1073/pnas.2514057122>.
- Kuzu. Kuzu: An embeddable property graph database management system. <https://kuzudb.com/>, 2024.
- Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, and Jenny Chim. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Jimmy Lin and Xueguang Ma. A few brief notes on deepimpact, coil, and a conceptual framework for information retrieval techniques. *arXiv preprint arXiv:2106.14807*, 2021.
- Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. Pretrained transformers for text ranking: Bert and beyond. *Synthesis Lectures on Human Language Technologies*, 2021.
- Sheng-Chieh Lin and Jimmy Lin. Densifying sparse representations for passage retrieval by representational slicing. *arXiv preprint arXiv:2112.04666*, 2022.
- Lineage tracing review authors. Evolving strategies for lineage tracing: Genetic markers, synthetic barcodes, and natural variants. *Cell Stem Cell*, 2026.
- Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating attribution in language models. *arXiv preprint arXiv:2401.05432*, 2024.
- Antonio Mallia, Omar Khattab, Torsten Suel, and Nicola Tonellotto. Learning passage impacts for inverted indexes. In *Proceedings of SIGIR*, pages 1723–1727, 2021.
- Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, Mia Glaese, Susannah Young, Lucy Campbell-Gillingham, Geoffrey Irving, and Nat McAleese. Teaching language models to support answers with verified quotes. *arXiv preprint arXiv:2203.11147*, 2023.
- Meta Platforms. Rocksdb: A persistent key-value store for fast storage environments. <https://rocksdb.org/>, 2019.
- Milvus. Milvus: An open-source vector database. <https://milvus.io/>, 2024.
- Network sheaf authors. Precision-graded cohomology and arithmetic persistence for network sheaves. *arXiv preprint arXiv:2511.00677*, 2025.
- Jacques Ninio. Kinetic amplification of enzyme discrimination. *Biochimie*, 57(5):587–595, 1975.
- Alexander H Nitz, Tito Dal Canton, Derek Davis, and Steven Reyes. Pycbc live: Rapid detection of gravitational waves from compact binary mergers. *Physical Review D*, 98(2):024050, 2018.
- ONNX Runtime. Onnx runtime: Cross-platform accelerated machine learning. <https://onnxruntime.ai/>, 2024.
- OpenSearch. Opensearch: Open-source search and analytics suite. <https://opensearch.org/>, 2024.
-

- pgvector. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>, 2024.
- Phyldynamic inference authors. A bayesian phyldynamic inference framework for single-cell crispr/cas9 lineage tracing barcode data with dependent target sites. *PLOS Computational Biology*, 2025.
- Predictive coding sheaf authors. Sheaf cohomology of linear predictive coding networks. *arXiv preprint arXiv:2511.11092*, 2025.
- Qdrant. Qdrant: Vector database for the next generation of ai applications. <https://qdrant.tech/>, 2024.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, and Jeremy Rapin. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubham Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024.
- Sheaf-theoretic distributed systems authors. A sheaf-theoretic characterization of tasks in distributed systems. *arXiv preprint arXiv:2503.02556*, 2025.
- Spectral coarse graining authors. Spectral coarse graining and rescaling for preserving structural and dynamical properties in graphs. *Physical Review E*, 2025.
- SQLite Consortium. Sqlite: A small, fast, self-contained sql database engine. <https://www.sqlite.org/>, 2024.
- Tantivy. Tantivy: A full-text search engine library in rust. <https://github.com/quickwit-oss/tantivy>, 2024.
- Nomic AI Team. Nomic embed: Training a reproducible, long-context, and multilingual text embedder. *arXiv preprint arXiv:2402.01613*, 2024.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, and Shruti Bhosale. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Nhat Truong, Minh Nguyen, and Tuan Pham. Recursive abstractive processing for long document understanding. *arXiv preprint arXiv:2310.06117*, 2023.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2024.
- Michal Was, Marie-Anne Bizouard, Violette Brisson, Fabien Cavalier, Michel Davier, Patrice Hello, Nicolas Leroy, Florent Robinet, and Miltiadis Vavoulidis. On the background estimation by time slides in a network of gravitational wave detectors. *Classical and Quantum Gravity*, 27(1):015005, 2010.

- Weaviate. Weaviate: The ai-native vector database. <https://weaviate.io/>, 2024.
- Jim Webber. Neo4j: The graph database platform. <https://neo4j.com/>, 2010.
- Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053*, 2022.
- Shitao Xiao, Zheng Liu, Yingxia Pei, Qixin Su, Yao Lu, and Liang Liu. Bge: Benchmarking general embeddings. *arXiv preprint arXiv:2310.07224*, 2023.
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808*, 2020.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.
- Zhiwei Zeng, Yiming Liu, Zhao Wang, Yifan Li, Chen Zhang, and Zhicheng Liu. Lightrag: A lightweight retrieval-augmented generation framework for efficient knowledge integration. *arXiv preprint arXiv:2410.05779*, 2024.
- Yukun Zhang, Yifan Liu, Feng Liu, Jie Guan, Xiaoyang Wang, and Xipeng Qiu. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- Lei Zhong, Jiapu Wu, Qika Li, Hao Peng, and Xindong Wu. Knowledge graph retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2408.08921*, 2024.
- Shengyao Zhuang and Guido Zuccon. Fast passage re-ranking with contextualized exact term matching and efficient passage expansion. *arXiv preprint arXiv:2108.08513*, 2022.